OFFICIAL DOCUMENTATION · OFFLINE REFERENCE

Ollama Documentation

Official documentation: guide, capabilities, CLI, API reference and Python library

SOURCE	docs.ollama.com and github.com/ollama/ollama (docs/), github.com/ollama/ollama-python
COVERS	Running models, vision, embeddings, structured outputs, CLI, Modelfile, context length, Linux install, GPU, troubleshooting, REST API, Python client

Index

Page numbers match the page counter at the bottom of each page. Every entry is a clickable link.

Parts at a glance

Get started	2 topics	**6**
Capabilities	7 topics	**12**
More information	10 topics	**33**
API reference	6 topics	**69**
Python library	1 topic	**117**

Detailed contents

Get started 6

1 Ollama documentation 6
- 1.1 Explore models 6
- 1.2 Documentation 6
- 1.3 Community 6

2 Quickstart 7
- 2.1 Get started 7
- 2.2 Connect a desktop app 7
- 2.3 Use a coding agent 7
- 2.4 Build an application 8
- 2.5 Run a model locally 11
- 2.6 Next steps 11

Capabilities 12

3 Streaming 12
- 3.1 Key streaming concepts 12
- 3.2 Handling streamed chunks 12

4 Thinking 13
- 4.1 Discover a model's thinking controls 13
- 4.2 Enable thinking in API calls 13
- 4.3 Stream the reasoning trace 14

5 Structured Outputs 15
- 5.1 Generating structured JSON 15
- 5.2 Generating structured JSON with a schema 15
- 5.3 Example: Extract structured data 16
- 5.4 Example: Vision with structured outputs 16
- 5.5 Tips for reliable structured outputs 17

6 Vision 18
- 6.1 Quick start 18
- 6.2 Usage with Ollama's API 18

7 Embeddings 19
- 7.1 Generate embeddings 19
- 7.2 Generate a batch of embeddings 19
- 7.3 Tips 20

8 Tool calling 21

8.1 Calling a single tool 21
8.2 Parallel tool calling 22
8.3 Multi-turn tool calling (Agent loop) 24
8.4 Tool calling with streaming 25
8.5 Using functions as tools with Ollama Python SDK 26

9 Decision 28
9.1 Setup 28
9.2 Ask from the terminal 28
9.3 Choose a label 29
9.4 Judge an image 29
9.5 Use a cloud model 30
9.6 Question types 31
9.7 Ask multiple questions 31

More information 33

10 CLI Reference 33
10.1 Run a model 33
10.2 Launch integrations 33
10.3 Generate embeddings 34
10.4 Ask a decision model 34
10.5 Speak with a speech model 34
10.6 Download a model 34
10.7 Remove a model 34
10.8 List models 35
10.9 Sign in to Ollama 35
10.10 Sign out of Ollama 35
10.11 Create a customized model 35
10.12 List running models 35
10.13 Stop a running model 35
10.14 Start Ollama 35

11 Modelfile Reference 36
11.1 Table of Contents 36
11.2 Format 36
11.3 Examples 36
11.4 Instructions 37
11.5 Notes 41

12 Context length 42
12.1 Setting context length 42

13 Linux 43
13.1 Install 43
13.2 Manual install 43
13.3 Customizing 44
13.4 Updating 45
13.5 Installing specific versions 45
13.6 Viewing logs 45
13.7 Uninstall 45

14 macOS 47
14.1 System Requirements 47
14.2 Filesystem Requirements 47
14.3 Troubleshooting 47
14.4 Uninstall 47

15 Docker 48

15.1 CPU only 48
15.2 Nvidia GPU 48
15.3 AMD GPU 49
15.4 Vulkan Support 49
15.5 Run model locally 49
15.6 Try different models 49

16 Importing a Model 50
16.1 Table of Contents 50
16.2 Importing a model from Safetensors weights 50
16.3 Importing a GGUF model 50
16.4 Sharing your model on ollama.com 50

17 FAQ 52
17.1 How can I upgrade Ollama? 52
17.2 How can I view the logs? 52
17.3 Is my GPU compatible with Ollama? 52
17.4 How can I specify the context window size? 52
17.5 How can I tell if my model was loaded onto the GPU? 52
17.6 How do I configure Ollama server? 53
17.7 How do I use Ollama behind a proxy? 54
17.8 Does Ollama send my prompts and answers back to ollama.com? 54
17.9 How do I disable Ollama's cloud features? 54
17.10 How can I expose Ollama on my network? 55
17.11 How can I use Ollama with a proxy server? 55
17.12 How can I use Ollama with ngrok? 55
17.13 How can I use Ollama with Cloudflare Tunnel? 55
17.14 How can I allow additional web origins to access Ollama? 55
17.15 Where are models stored? 56
17.16 How can I use Ollama in Visual Studio Code? 56
17.17 How do I use Ollama with GPU acceleration in Docker? 56
17.18 Why is networking slow in WSL2 on Windows 10? 56
17.19 How can I preload a model into Ollama to get faster response times? 56
17.20 How do I keep a model loaded in memory or make it unload immediately? 57
17.21 How do I manage the maximum number of requests the Ollama server can queue? 57
17.22 How does Ollama handle concurrent requests? 57
17.23 How does Ollama load models on multiple GPUs? 58
17.24 How can I enable Flash Attention? 58
17.25 How can I set the quantization type for the K/V cache? 58
17.26 Where can I find my Ollama Public Key? 59
17.27 How can I stop Ollama from starting when I login to my computer? 59

18 Hardware support 61
18.1 Nvidia 61
18.2 AMD Radeon 62
18.3 Metal (Apple GPUs) 64
18.4 Vulkan GPU Support 64

19 Troubleshooting 65
19.1 LLM libraries 65
19.2 Installing older or pre-release versions on Linux 66
19.3 Linux tmp noexec 66
19.4 Linux docker 66
19.5 NVIDIA GPU Discovery 66
19.6 AMD GPU Discovery 67
19.7 Multiple AMD GPUs 67
19.8 Windows Terminal Errors 68

API reference 69

20 Introduction 69
- 20.1 Get started 69
- 20.2 Base URLs 69
- 20.3 Ollama API example 69
- 20.4 Libraries 69
- 20.5 Versioning 70

21 Streaming 71
- 21.1 Disabling streaming 71
- 21.2 When to use streaming vs non-streaming 71

22 Usage 72
- 22.1 Example response 72

23 Errors 73
- 23.1 Status codes 73
- 23.2 Error messages 73
- 23.3 Errors that occur while streaming 73

24 OpenAI compatibility 74
- 24.1 Direct cloud access 74
- 24.2 Local server usage 74
- 24.3 Endpoints 77
- 24.4 Models 81

25 API 83
- 25.1 Endpoints 83
- 25.2 Conventions 83
- 25.3 Generate a completion 83
- 25.4 Generate a chat completion 92
- 25.5 Create a Model 104
- 25.6 Check if a Blob Exists 106
- 25.7 Push a Blob 107
- 25.8 List Local Models 107
- 25.9 Show Model Information 108
- 25.10 Copy a Model 109
- 25.11 Delete a Model 110
- 25.12 Pull a Model 110
- 25.13 Push a Model 111
- 25.14 Generate Embeddings 112
- 25.15 List Running Models 114
- 25.16 Generate Embedding 115
- 25.17 Version 116

Python library 117

26 Ollama Python Library 117
- 26.1 Prerequisites 117
- 26.2 Install 117
- 26.3 Usage 117
- 26.4 Streaming responses 117
- 26.5 Cloud Models 117
- 26.6 Custom client 118
- 26.7 Async client 119
- 26.8 API 119
- 26.9 Errors 121

Ollama documentation

Source: https://docs.ollama.com/index

Use open models in your desktop apps and coding agents, or build them into your application.

Use your apps

Connect Claude or ChatGPT on your desktop, or launch a coding agent from your terminal.

Build an application

Call cloud models with Ollama, OpenAI, or Anthropic APIs.

Explore models

Choose cloud models or run models on your computer.

Cloud models

Run models without a local GPU.

Local models

Download Ollama and run a model on your computer.

Browse the model library for coding, vision, embeddings, and reasoning models.

Documentation

- Integrations — connect an app, editor, or agent.
- API reference — use Ollama's API or compatible OpenAI and Anthropic clients.
- Python library and JavaScript library — use Ollama from your application.

Community

Join the conversation on Discord or Reddit.

Quickstart

Source: https://docs.ollama.com/quickstart

Use Ollama in desktop apps and coding agents, or build an application with an API key.

Get started

Download Ollama for macOS, Windows, or Linux. Open the app, or get started from your terminal:

```
ollama
```

Follow the setup prompts. Sign in to use cloud models, or choose a local model.

Connect a desktop app

On macOS, open Ollama and select **Apps**. Connect **Claude** or **ChatGPT (Desktop)**. Follow the prompts to install or restart the app.

Choose your Ollama models in **Settings → Apps**.

- Claude Desktop — use Ollama models in Claude.
- ChatGPT Desktop — use Ollama models in Codex mode. Regular Chat and voice use your usual ChatGPT connection.

Use a coding agent

From your project directory, launch your agent:

Claude Code

```
ollama launch claude
```

Ollama offers to install Claude Code if needed. See Claude Code for details.

Codex CLI

Install Codex CLI first, then launch it:

```
ollama launch codex
```

See Codex CLI for setup.

OpenCode

```
ollama launch opencode
```

Ollama offers to install OpenCode if needed. See OpenCode for details.

Choose a model when prompted, then try:

```
Explain how this repository is organized.
```

For more tools, see Integrations. To connect an agent directly with an API key, see Claude Code.

Build an application

Use cloud models with an API key, or run models locally without one. Cloud requests do not require an Ollama installation.

1. Create an API key

For local models, skip this step and select **Local** under Send a request.

For cloud models, sign in or create an account, then create an API key.

Set your key in the terminal:

macOS / Linux

```
```

Windows

```
$env:OLLAMA_API_KEY = "your_api_key"
```

Keep your key on your application's server, outside browser code and source control.

2. Send a request

Make your first request to Ollama

Cloud

These examples use the `gemma4:31b` cloud model.

Ollama API

```
curl https://ollama.com/api/chat \
  -H "Authorization: Bearer $OLLAMA_API_KEY" \
  -H "Content-Type: application/json" \
  -d '{
    "model": "gemma4:31b",
    "messages": [
      {
        "role": "user",
        "content": "Say hello in one sentence."
      }
    ],
    "stream": false
  }'
```

Read the answer from `message.content`. See Ollama's API and libraries.

OpenAI Chat Completions

```
curl https://ollama.com/v1/chat/completions \
  -H "Authorization: Bearer $OLLAMA_API_KEY" \
  -H "Content-Type: application/json" \
  -d '{
    "model": "gemma4:31b",
    "messages": [
      {
```

```
        "role": "user",
        "content": "Say hello in one sentence."
      }
    ]
  }'
```

Read the answer from `choices[0].message.content`. See OpenAI compatibility for client setup and supported features.

OpenAI Responses

```
curl https://ollama.com/v1/responses \
  -H "Authorization: Bearer $OLLAMA_API_KEY" \
  -H "Content-Type: application/json" \
  -d '{
    "model": "gemma4:31b",
    "input": "Say hello in one sentence."
  }'
```

Read the `output_text` content blocks inside `output`. The OpenAI Python client exposes the text as `response.output_text`.

Responses requests are stateless: include conversation history in each request. `previous_response_id` and `conversation` aren't supported. See Responses compatibility.

Anthropic Messages

```
curl https://ollama.com/v1/messages \
  -H "Authorization: Bearer $OLLAMA_API_KEY" \
  -H "Content-Type: application/json" \
  -H "anthropic-version: 2023-06-01" \
  -d '{
    "model": "gemma4:31b",
    "max_tokens": 1024,
    "messages": [
      {
        "role": "user",
        "content": "Say hello in one sentence."
      }
    ]
  }'
```

Read text blocks from `content`. Direct cloud requests use bearer authentication. See Anthropic compatibility for client setup and supported features.

OpenAI and Anthropic compatibility each cover a subset of the original API. See Cloud for models and usage limits.

Local

Run Gemma 4 E2B on your computer. No API key required.

> **Note:** The model download is about 7.2 GB. We recommend 8 GB of available VRAM, or unified memory on a Mac. Larger context windows need more memory. With less VRAM, Ollama can use system RAM, but responses may be slower.

Download Ollama and open the app. On Linux, start the server with `ollama serve` if it is not already running.

Download the model:

```
ollama pull gemma4:e2b
```

Send a request to your local server:

Ollama API

```
curl http://localhost:11434/api/chat \
  -H "Content-Type: application/json" \
  -d '{
    "model": "gemma4:e2b",
    "messages": [
      {
        "role": "user",
        "content": "Say hello in one sentence."
      }
    ],
    "stream": false
  }'
```

OpenAI Chat Completions

```
curl http://localhost:11434/v1/chat/completions \
  -H "Content-Type: application/json" \
  -d '{
    "model": "gemma4:e2b",
    "messages": [
      {
        "role": "user",
        "content": "Say hello in one sentence."
      }
    ]
  }'
```

Read the answer from `choices[0].message.content`. See OpenAI compatibility for client setup and supported features.

OpenAI Responses

```
curl http://localhost:11434/v1/responses \
  -H "Content-Type: application/json" \
  -d '{
    "model": "gemma4:e2b",
    "input": "Say hello in one sentence."
  }'
```

Read the `output_text` content blocks inside `output`. The OpenAI Python client exposes the text as `response.output_text`.

Responses requests are stateless: include conversation history in each request. `previous_response_id` and `conversation` aren't supported. See Responses compatibility.

Anthropic Messages

```
curl http://localhost:11434/v1/messages \
  -H "Content-Type: application/json" \
  -H "anthropic-version: 2023-06-01" \
  -d '{
    "model": "gemma4:e2b",
    "max_tokens": 1024,
    "messages": [
      {
        "role": "user",
        "content": "Say hello in one sentence."
```

```
      }
    ]
  }'
```

Read text blocks from `content`. See Anthropic compatibility for client setup and supported features.

Run a model locally

Download Ollama, then run:

```
ollama run gemma4:e2b
```

Ollama downloads the model and starts a chat on your computer. Type `/bye` to leave.

Next steps

Add tool calling, stream responses, or browse more models.

Streaming

Source: https://docs.ollama.com/capabilities/streaming

Streaming allows you to render text as it is produced by the model.

Streaming is enabled by default through the REST API, but disabled by default in the SDKs.

To enable streaming in the SDKs, set the `stream` parameter to `True`.

Key streaming concepts

1. Chatting: Stream partial assistant messages. Each chunk includes the `content` so you can render messages as they arrive.
2. Thinking: Thinking-capable models emit a `thinking` field alongside regular content in each chunk. Detect this field in streaming chunks to show or hide reasoning traces before the final answer arrives.
3. Tool calling: Watch for streamed `tool_calls` in each chunk, execute the requested tool, and append tool outputs back into the conversation.

Handling streamed chunks

> **Note:** It is necessary to accumulate the partial fields in order to maintain the history of the conversation. This is particularly important for tool calling where the thinking, tool call from the model, and the executed tool result must be passed back to the model in the next request.

Python

```python
from ollama import chat

stream = chat(
  model='qwen3',
  messages=[{'role': 'user', 'content': 'What is 17 × 23?'}],
  stream=True,
)

in_thinking = False
content = ''
thinking = ''
for chunk in stream:
  if chunk.message.thinking:
    if not in_thinking:
      in_thinking = True
      print('Thinking:\n', end='', flush=True)
    print(chunk.message.thinking, end='', flush=True)
    # accumulate the partial thinking
    thinking += chunk.message.thinking
  elif chunk.message.content:
    if in_thinking:
      in_thinking = False
      print('\n\nAnswer:\n', end='', flush=True)
    print(chunk.message.content, end='', flush=True)
    # accumulate the partial content
    content += chunk.message.content

  # append the accumulated fields to the messages for the next request
  new_messages = [{ role: 'assistant', thinking: thinking, content: content }]
```

Thinking

Source: https://docs.ollama.com/capabilities/thinking

Thinking-capable models emit a `thinking` field that separates their reasoning trace from the final answer.

Use this capability to audit model steps, animate the model *thinking* in a UI, or hide the trace entirely when you only need the final response.

See the full list of thinking models.

Discover a model's thinking controls

Thinking controls vary by model. Use `/api/show` to discover the values a model supports and the value Ollama uses by default:

```
curl http://localhost:11434/api/show -d '{
  "model": "gpt-oss"
}'
```

The response includes a top-level `thinking` object:

```
{
  "thinking": {
    "values": ["low", "medium", "high"],
    "default": "medium"
  }
}
```

- `values` can contain booleans (`true` or `false`) for on/off controls. It can also contain model-defined strings for named levels.
- `default` is used when `think` is not set.
- `values: [false]` means the model does not support thinking.
- If `thinking` is omitted, the model has no thinking metadata. The model might conduct thinking based on its existing behavior.

Enable thinking in API calls

Set the `think` field on a chat or generate request:

- `true`: request thinking output.
- `false`: request no thinking output, if the model permits it.
- `null`: use the model default.
- A string: select a supported level from `thinking.values`. Use the exact value from `/api/show`. Numbers are not supported.

If the model resolves named levels from `/api/show` metadata, Ollama applies supported names exactly. Unsupported names use the model default.

The reasoning output and answer use separate fields. Chat returns `message.thinking` and `message.content`. Generate returns `thinking` and `response`.

cURL

```
curl http://localhost:11434/api/chat -d '{
  "model": "qwen3",
  "messages": [{
    "role": "user",
    "content": "How many letter r are in strawberry?"
  }],
  "think": true,
  "stream": false
}'
```

Python

```python
from ollama import chat

response = chat(
  model='qwen3',
  messages=[{'role': 'user', 'content': 'How many letter r are in strawberry?'}],
  think=True,
  stream=False,
)

print('Thinking:\n', response.message.thinking)
print('Answer:\n', response.message.content)
```

Stream the reasoning trace

Thinking streams interleave reasoning tokens before answer tokens. Detect the first `thinking` chunk to render a "thinking" section, then switch to the final reply once `message.content` arrives.

Python

```python
from ollama import chat

stream = chat(
  model='qwen3',
  messages=[{'role': 'user', 'content': 'What is 17 × 23?'}],
  think=True,
  stream=True,
)

in_thinking = False

for chunk in stream:
  if chunk.message.thinking and not in_thinking:
    in_thinking = True
    print('Thinking:\n', end='')

  if chunk.message.thinking:
    print(chunk.message.thinking, end='')
  elif chunk.message.content:
    if in_thinking:
      print('\n\nAnswer:\n', end='')
      in_thinking = False
    print(chunk.message.content, end='')
```

Structured Outputs

Source: https://docs.ollama.com/capabilities/structured-outputs

> **Note:** Ollama's cloud currently does not support structured outputs.

Structured outputs let you enforce a JSON schema on model responses so you can reliably extract structured data, describe images, or keep every reply consistent.

Generating structured JSON

cURL

```
curl -X POST http://localhost:11434/api/chat -H "Content-Type: application/json" -d '{
  "model": "gpt-oss",
  "messages": [{"role": "user", "content": "Tell me about Canada in one line"}],
  "stream": false,
  "format": "json"
}'
```

Python

```
from ollama import chat

response = chat(
  model='gpt-oss',
  messages=[{'role': 'user', 'content': 'Tell me about Canada.'}],
  format='json'
)
print(response.message.content)
```

Generating structured JSON with a schema

Provide a JSON schema to the `format` field.

> **Note:** It is ideal to also pass the JSON schema as a string in the prompt to ground the model's response.

cURL

```
curl -X POST http://localhost:11434/api/chat -H "Content-Type: application/json" -d '{
  "model": "gpt-oss",
  "messages": [{"role": "user", "content": "Tell me about Canada."}],
  "stream": false,
  "format": {
    "type": "object",
    "properties": {
      "name": {"type": "string"},
      "capital": {"type": "string"},
      "languages": {
        "type": "array",
        "items": {"type": "string"}
      }
    },
    "required": ["name", "capital", "languages"]
  }
}'
```

Python

Use Pydantic models and pass `model_json_schema()` to `format`, then validate the response:

```python
from ollama import chat
from pydantic import BaseModel

class Country(BaseModel):
  name: str
  capital: str
  languages: list[str]

response = chat(
  model='gpt-oss',
  messages=[{'role': 'user', 'content': 'Tell me about Canada.'}],
  format=Country.model_json_schema(),
)

country = Country.model_validate_json(response.message.content)
print(country)
```

Example: Extract structured data

Define the objects you want returned and let the model populate the fields:

```python
from ollama import chat
from pydantic import BaseModel

class Pet(BaseModel):
  name: str
  animal: str
  age: int
  color: str | None
  favorite_toy: str | None

class PetList(BaseModel):
  pets: list[Pet]

response = chat(
  model='gpt-oss',
  messages=[{'role': 'user', 'content': 'I have two cats named Luna and Loki...'}],
  format=PetList.model_json_schema(),
)

pets = PetList.model_validate_json(response.message.content)
print(pets)
```

Example: Vision with structured outputs

Vision models accept the same `format` parameter, enabling deterministic descriptions of images:

```python
from ollama import chat
from pydantic import BaseModel
from typing import Literal, Optional

class Object(BaseModel):
  name: str
  confidence: float
  attributes: str

class ImageDescription(BaseModel):
  summary: str
  objects: list[Object]
  scene: str
  colors: list[str]
  time_of_day: Literal['Morning', 'Afternoon', 'Evening', 'Night']
```

```
  setting: Literal['Indoor', 'Outdoor', 'Unknown']
  text_content: Optional[str] = None

response = chat(
  model='gemma4',
  messages=[{
    'role': 'user',
    'content': 'Describe this photo and list the objects you detect.',
    'images': ['path/to/image.jpg'],
  }],
  format=ImageDescription.model_json_schema(),
  options={'temperature': 0},
)

image_description = ImageDescription.model_validate_json(response.message.content)
print(image_description)
```

Tips for reliable structured outputs

- Define schemas with Pydantic (Python) or Zod (JavaScript) so they can be reused for validation.
- Lower the temperature (e.g., set it to `0`) for more deterministic completions.
- Structured outputs work through the OpenAI-compatible API via `response_format`

Vision

Source: https://docs.ollama.com/capabilities/vision

Vision models accept images alongside text so the model can describe, classify, and answer questions about what it sees.

Quick start

```
ollama run gemma4 ./image.png what is in this image?
```

Usage with Ollama's API

Provide an `images` array. SDKs accept file paths, URLs or raw bytes while the REST API expects base64-encoded image data.

cURL

```
# 1. Download a sample image
curl -L -o test.jpg "https://upload.wikimedia.org/wikipedia/commons/3/3a/Cat03.jpg"

# 2. Encode the image
IMG=$(base64 < test.jpg | tr -d '\n')

# 3. Send it to Ollama
curl -X POST http://localhost:11434/api/chat \
-H "Content-Type: application/json" \
-d '{
    "model": "gemma4",
    "messages": [{
    "role": "user",
    "content": "What is in this image?",
    "images": ["'"$IMG"'"]
    }],
    "stream": false
}'
```

Python

```
from ollama import chat
# from pathlib import Path

# Pass in the path to the image
path = input('Please enter the path to the image: ')

# You can also pass in base64 encoded image data
# img = base64.b64encode(Path(path).read_bytes()).decode()
# or the raw bytes
# img = Path(path).read_bytes()

response = chat(
  model='gemma4',
  messages=[
    {
      'role': 'user',
      'content': 'What is in this image? Be concise.',
      'images': [path],
    }
  ],
)

print(response.message.content)
```

Embeddings

Source: https://docs.ollama.com/capabilities/embeddings

Embeddings turn text into numeric vectors you can store in a vector database, search with cosine similarity, or use in RAG pipelines. The vector length depends on the model (typically 384–1024 dimensions).

Generate embeddings

CLI

Generate embeddings directly from the command line:

```
ollama run embeddinggemma "Hello world"
```

You can also pipe text to generate embeddings:

```
echo "Hello world" | ollama run embeddinggemma
```

Output is a JSON array.

cURL

```
curl -X POST http://localhost:11434/api/embed \
  -H "Content-Type: application/json" \
  -d '{
    "model": "embeddinggemma",
    "input": "The quick brown fox jumps over the lazy dog."
  }'
```

Python

```
import ollama

single = ollama.embed(
  model='embeddinggemma',
  input='The quick brown fox jumps over the lazy dog.'
)
print(len(single['embeddings'][0]))  # vector length
```

> **Note:** The `/api/embed` endpoint returns L2-normalized (unit-length) vectors.

Generate a batch of embeddings

Pass an array of strings to `input`.

cURL

```
curl -X POST http://localhost:11434/api/embed \
  -H "Content-Type: application/json" \
  -d '{
    "model": "embeddinggemma",
    "input": [
      "First sentence",
      "Second sentence",
```

```
      "Third sentence"
    ]
  }'
```

Python

```python
import ollama

batch = ollama.embed(
  model='embeddinggemma',
  input=[
    'The quick brown fox jumps over the lazy dog.',
    'The five boxing wizards jump quickly.',
    'Jackdaws love my big sphinx of quartz.',
  ]
)
print(len(batch['embeddings']))  # number of vectors
```

Tips

- Use cosine similarity for most semantic search use cases.
- Use the same embedding model for both indexing and querying.

Tool calling

Source: https://docs.ollama.com/capabilities/tool-calling

Ollama supports tool calling (also known as function calling) which allows a model to invoke tools and incorporate their results into its replies.

Calling a single tool

Invoke a single tool and include its response in a follow-up request.

Also known as "single-shot" tool calling.

cURL

```
curl -s http://localhost:11434/api/chat -H "Content-Type: application/json" -d '{
  "model": "qwen3",
  "messages": [{"role": "user", "content": "What is the temperature in New York?"}],
  "stream": false,
  "tools": [
    {
      "type": "function",
      "function": {
        "name": "get_temperature",
        "description": "Get the current temperature for a city",
        "parameters": {
          "type": "object",
          "required": ["city"],
          "properties": {
            "city": {"type": "string", "description": "The name of the city"}
          }
        }
      }
    }
  ]
}'
```

Generate a response with a single tool result

```
curl -s http://localhost:11434/api/chat -H "Content-Type: application/json" -d '{
  "model": "qwen3",
  "messages": [
    {"role": "user", "content": "What is the temperature in New York?"},
    {
      "role": "assistant",
      "tool_calls": [
        {
          "type": "function",
          "function": {
            "index": 0,
            "name": "get_temperature",
            "arguments": {"city": "New York"}
          }
        }
      ]
    },
    {"role": "tool", "tool_name": "get_temperature", "content": "22°C"}
  ],
  "stream": false
}'
```

Python

Install the Ollama Python SDK:

```
# with pip
pip install ollama -U

# with uv
uv add ollama
```

```
from ollama import chat

def get_temperature(city: str) -> str:
  """Get the current temperature for a city

  Args:
    city: The name of the city

  Returns:
    The current temperature for the city
  """
  temperatures = {
    "New York": "22°C",
    "London": "15°C",
    "Tokyo": "18°C",
  }
  return temperatures.get(city, "Unknown")

messages = [{"role": "user", "content": "What is the temperature in New York?"}]

# pass functions directly as tools in the tools list or as a JSON schema
response = chat(model="qwen3", messages=messages, tools=[get_temperature], think=True)

messages.append(response.message)
if response.message.tool_calls:
  # only recommended for models which only return a single tool call
  call = response.message.tool_calls[0]
  result = get_temperature(**call.function.arguments)
  # add the tool result to the messages
  messages.append({"role": "tool", "tool_name": call.function.name, "content": str(result)})

  final_response = chat(model="qwen3", messages=messages, tools=[get_temperature],
think=True)
  print(final_response.message.content)
```

Parallel tool calling

cURL

Request multiple tool calls in parallel, then send all tool responses back to the model.

```
curl -s http://localhost:11434/api/chat -H "Content-Type: application/json" -d '{
  "model": "qwen3",
  "messages": [{"role": "user", "content": "What are the current weather conditions and
temperature in New York and London?"}],
  "stream": false,
  "tools": [
    {
      "type": "function",
      "function": {
        "name": "get_temperature",
        "description": "Get the current temperature for a city",
        "parameters": {
          "type": "object",
          "required": ["city"],
          "properties": {
            "city": {"type": "string", "description": "The name of the city"}
          }
        }
      }
    },
    {
```

```
      "type": "function",
      "function": {
        "name": "get_conditions",
        "description": "Get the current weather conditions for a city",
        "parameters": {
          "type": "object",
          "required": ["city"],
          "properties": {
            "city": {"type": "string", "description": "The name of the city"}
          }
        }
      }
    }
  ]
}'
```

Generate a response with multiple tool results

```
curl -s http://localhost:11434/api/chat -H "Content-Type: application/json" -d '{
  "model": "qwen3",
  "messages": [
    {"role": "user", "content": "What are the current weather conditions and temperature in
New York and London?"},
    {
      "role": "assistant",
      "tool_calls": [
        {
          "type": "function",
          "function": {
            "index": 0,
            "name": "get_temperature",
            "arguments": {"city": "New York"}
          }
        },
        {
          "type": "function",
          "function": {
            "index": 1,
            "name": "get_conditions",
            "arguments": {"city": "New York"}
          }
        },
        {
          "type": "function",
          "function": {
            "index": 2,
            "name": "get_temperature",
            "arguments": {"city": "London"}
          }
        },
        {
          "type": "function",
          "function": {
            "index": 3,
            "name": "get_conditions",
            "arguments": {"city": "London"}
          }
        }
      ]
    },
    {"role": "tool", "tool_name": "get_temperature", "content": "22°C"},
    {"role": "tool", "tool_name": "get_conditions", "content": "Partly cloudy"},
    {"role": "tool", "tool_name": "get_temperature", "content": "15°C"},
    {"role": "tool", "tool_name": "get_conditions", "content": "Rainy"}
  ],
  "stream": false
}'
```

Python

```python
from ollama import chat

def get_temperature(city: str) -> str:
  """Get the current temperature for a city

  Args:
    city: The name of the city

  Returns:
    The current temperature for the city
  """
  temperatures = {
    "New York": "22°C",
    "London": "15°C",
    "Tokyo": "18°C"
  }
  return temperatures.get(city, "Unknown")

def get_conditions(city: str) -> str:
  """Get the current weather conditions for a city

  Args:
    city: The name of the city

  Returns:
    The current weather conditions for the city
  """
  conditions = {
    "New York": "Partly cloudy",
    "London": "Rainy",
    "Tokyo": "Sunny"
  }
  return conditions.get(city, "Unknown")


messages = [{'role': 'user', 'content': 'What are the current weather conditions and
temperature in New York and London?'}]

# The python client automatically parses functions as a tool schema so we can pass them
directly
# Schemas can be passed directly in the tools list as well
response = chat(model='qwen3', messages=messages, tools=[get_temperature, get_conditions],
think=True)

# add the assistant message to the messages
messages.append(response.message)
if response.message.tool_calls:
  # process each tool call
  for call in response.message.tool_calls:
    # execute the appropriate tool
    if call.function.name == 'get_temperature':
      result = get_temperature(**call.function.arguments)
    elif call.function.name == 'get_conditions':
      result = get_conditions(**call.function.arguments)
    else:
      result = 'Unknown tool'
    # add the tool result to the messages
    messages.append({'role': 'tool',  'tool_name': call.function.name, 'content':
str(result)})

  # generate the final response
  final_response = chat(model='qwen3', messages=messages, tools=[get_temperature,
get_conditions], think=True)
  print(final_response.message.content)
```

Multi-turn tool calling (Agent loop)

An agent loop allows the model to decide when to invoke tools and incorporate their results into its replies.

It also might help to tell the model that it is in a loop and can make multiple tool calls.

Python

```python
from ollama import chat, ChatResponse


def add(a: int, b: int) -> int:
  """Add two numbers"""
  """
  Args:
    a: The first number
    b: The second number

  Returns:
    The sum of the two numbers
  """
  return a + b


def multiply(a: int, b: int) -> int:
  """Multiply two numbers"""
  """
  Args:
    a: The first number
    b: The second number

  Returns:
    The product of the two numbers
  """
  return a * b


available_functions = {
  'add': add,
  'multiply': multiply,
}

messages = [{'role': 'user', 'content': 'What is (11434+12341)*412?'}]
while True:
    response: ChatResponse = chat(
        model='qwen3',
        messages=messages,
        tools=[add, multiply],
        think=True,
    )
    messages.append(response.message)
    print("Thinking: ", response.message.thinking)
    print("Content: ", response.message.content)
    if response.message.tool_calls:
        for tc in response.message.tool_calls:
            if tc.function.name in available_functions:
                print(f"Calling {tc.function.name} with arguments {tc.function.arguments}")
                result = available_functions[tc.function.name](**tc.function.arguments)
                print(f"Result: {result}")
                # add the tool result to the messages
                messages.append({'role': 'tool', 'tool_name': tc.function.name, 'content':
str(result)})
    else:
        # end the loop when there are no more tool calls
        break
  # continue the loop with the updated messages
```

Tool calling with streaming

When streaming, gather every chunk of `thinking`, `content`, and `tool_calls`, then return those fields together with any tool results in the follow-up request.

Python

```python
from ollama import chat


def get_temperature(city: str) -> str:
  """Get the current temperature for a city

  Args:
    city: The name of the city

  Returns:
    The current temperature for the city
  """
  temperatures = {
    'New York': '22°C',
    'London': '15°C',
  }
  return temperatures.get(city, 'Unknown')


messages = [{'role': 'user', 'content': "What is the temperature in New York?"}]

while True:
  stream = chat(
    model='qwen3',
    messages=messages,
    tools=[get_temperature],
    stream=True,
    think=True,
  )

  thinking = ''
  content = ''
  tool_calls = []

  done_thinking = False
  # accumulate the partial fields
  for chunk in stream:
    if chunk.message.thinking:
      thinking += chunk.message.thinking
      print(chunk.message.thinking, end='', flush=True)
    if chunk.message.content:
      if not done_thinking:
        done_thinking = True
        print('\n')
      content += chunk.message.content
      print(chunk.message.content, end='', flush=True)
    if chunk.message.tool_calls:
      tool_calls.extend(chunk.message.tool_calls)
      print(chunk.message.tool_calls)

  # append accumulated fields to the messages
  if thinking or content or tool_calls:
    messages.append({'role': 'assistant', 'thinking': thinking, 'content': content,
'tool_calls': tool_calls})

  if not tool_calls:
    break

  for call in tool_calls:
    if call.function.name == 'get_temperature':
      result = get_temperature(**call.function.arguments)
    else:
      result = 'Unknown tool'
    messages.append({'role': 'tool', 'tool_name': call.function.name, 'content': result})
```

This loop streams the assistant response, accumulates partial fields, passes them back together, and appends the tool results so the model can complete its answer.

Using functions as tools with Ollama Python SDK

The Python SDK automatically parses functions as a tool schema so we can pass them directly. Schemas can still be passed if needed.

```python
from ollama import chat

def get_temperature(city: str) -> str:
  """Get the current temperature for a city

  Args:
    city: The name of the city

  Returns:
    The current temperature for the city
  """
  temperatures = {
    'New York': '22°C',
    'London': '15°C',
  }
  return temperatures.get(city, 'Unknown')

available_functions = {
  'get_temperature': get_temperature,
}
# directly pass the function as part of the tools list
response = chat(model='qwen3', messages=messages, tools=available_functions.values(),
think=True)
```

Decision

Source: https://docs.ollama.com/capabilities/decision

Use System One to classify text, answer yes/no questions, or score text against a rubric.

See the full list of decision models.

Setup

Start Ollama v0.35.0 or later, then download Nimble:

setup.sh

```
ollama pull nimble
```

Nimble, Tev1, Clef, and Clef Flash are decision models. The Clef models can also judge images and require v0.35.1 or later.

Local requests do not need an API key.

Ask from the terminal

`ollama run` answers questions about a state: text or JSON, piped in or given after the model name. Ask each question with `--ask`. It is answered yes or no, unless you follow it with labels to choose from (`--choice`) or levels to score it on (`--level`).

examples.sh

```
# Ask a yes/no question
ollama run nimble --ask "Is a refund being requested?" < ticket.txt

# Choose a label
ollama run nimble --ask "Which team should handle this?" \
  --choice billing="Payments and refunds" \
  --choice technical="Software problems" < ticket.txt

# Score on a scale, from the lowest level to the highest
ollama run nimble --ask "How urgent is this?" \
  --level "Routine" \
  --level "Soon" \
  --level "Immediate" < ticket.txt

# Ask several questions at once
ollama run nimble --ask "Is a refund being requested?" \
  --ask "Which team should handle this?" \
  --choice billing="Payments and refunds" \
  --choice technical="Software problems" \
  --ask "How urgent is this?" \
  --level "Routine" \
  --level "Soon" \
  --level "Immediate" < ticket.txt

# Use a JSON file as the state, and refer to its fields in backticks
ollama run nimble --ask 'Is a refund being requested in `body`?' < ticket.json

# Judge an image with Clef
ollama run clef ./receipt.png --ask "Is this a restaurant receipt?"

# Print the full response
ollama run nimble --ask "Is a refund being requested?" --format json < ticket.txt
```

Each answer prints on its own line. A yes/no answer shows the likelier answer and its probability, a label shows its probability, and a score shows the most likely level, its probability, and the score on the scale:

```
Is a refund being requested?    yes      97%
Which team should handle this?  billing  91%
How urgent is this?             Soon     64%  1.2 / 2
```

Without a state, `ollama run nimble` opens an interactive prompt. Your first message is the state, and after that each line is a yes/no question about it, so you can paste several at once; `/clear` starts over. With `--ask`, each message is a new state to answer those questions about.

Choose a label

Put your input in `state` and describe the possible answers in `criteria`:

choice.sh

```
curl http://localhost:11434/v1/systemone \
  -H 'Content-Type: application/json' \
  -d '{
    "model": "nimble",
    "state": "Our checkout has returned 500 errors since 9am.",
    "questions": {
      "label": {
        "type": "choice",
        "instructions": "Which label fits this ticket?",
        "criteria": {
          "billing": "Payments and refunds",
          "bug": "Software errors",
          "account": "Login and account access"
        }
      }
    }
  }'
```

Example response:

```
{
  "model": "nimble",
  "answers": {
    "label": {
      "type": "choice",
      "choice": "bug",
      "probabilities": {"billing": 0.0125, "bug": 0.9781, "account": 0.0093},
      "confidence": 0.8906
    }
  },
  "usage": {"input_tokens": 174, "output_tokens": 1}
}
```

Judge an image

Clef and Clef Flash read images alongside the state. Send base64-encoded PNG, JPEG, or WebP files in `images`; they are shared by all questions, in request order. URLs and data URLs are not accepted. `state` is still required — it holds the context for the decision.

image.sh

```
curl http://localhost:11434/v1/systemone \
  -H 'Content-Type: application/json' \
  -d '{
    "model": "clef-flash",
```

```
    "state": "A user took this screenshot and wants to know what it shows.",
    "images": ["<base64-encoded image>"],
    "questions": {
      "has_ollama": {
        "type": "noul",
        "instructions": "Does this image contain Ollama?"
      },
      "app": {
        "type": "choice",
        "instructions": "Which application is shown in this screenshot?",
        "criteria": {
          "vscode": "Visual Studio Code",
          "terminal": "A terminal emulator",
          "browser": "A web browser",
          "other": "A different application"
        }
      }
    }
  }'
```

Example response:

```
{
  "model": "clef-flash",
  "answers": {
    "has_ollama": {"type": "noul", "noul": 0.959},
    "app": {
      "type": "choice",
      "choice": "vscode",
      "probabilities": {"vscode": 0.966, "other": 0.017, "browser": 0.010, "terminal":
0.007},
      "confidence": 0.868
    }
  },
  "usage": {"input_tokens": 677, "output_tokens": 0}
}
```

Use a cloud model

Clef also runs in Ollama's cloud with a paid plan or purchased usage credits. Add `:cloud` to the model name to send the request through a local Ollama server, v0.40.5 or later, signed in to ollama.com:

cloud.sh

```
curl http://localhost:11434/v1/systemone \
  -H 'Content-Type: application/json' \
  -d '{
    "model": "clef:cloud",
    "state": "Our checkout has returned 500 errors since 9am.",
    "questions": {
      "urgent": {
        "type": "noul",
        "instructions": "Does this ticket need attention within the hour?"
      }
    }
  }'
```

Or call ollama.com directly with an API key:

cloud-direct.sh

```
curl https://ollama.com/v1/systemone \
  -H "Authorization: Bearer $OLLAMA_API_KEY" \
  -H 'Content-Type: application/json' \
  -d '{
    "model": "clef",
```

```
    "state": "Our checkout has returned 500 errors since 9am.",
    "questions": {
      "urgent": {
        "type": "noul",
        "instructions": "Does this ticket need attention within the hour?"
      }
    }
  }'
```

Cloud requests accept up to 64 questions, named with up to 100 ASCII letters, digits, `_`, `.`, or `-`, and up to 4 images of at most 4 MiB and 16 megapixels each, 8 MiB in total. Input is limited to the model's 65,536-token context window; longer input is rejected. Usage is billed per input token at the listed price.

Question types

Type	Criteria	Result
`choice`	2–26 named options with descriptions	The most likely option and each option's probability
`noul`	Descriptions for `"false"` and `"true"`	The probability of `true`, from 0 to 1
`score`	2–10 descriptions, ordered from lowest to highest	The probability-weighted average of the zero-based levels

For the urgency criteria below:

```
score = 0 × P(Routine) + 1 × P(Soon) + 2 × P(Immediate)
```

For `choice` and `score`, `confidence` measures how strongly the model favors one answer over the others, from 0 to 1. A higher value does not guarantee the answer is correct.

Ask multiple questions

Combine question types in one request. Here, both questions use the same ticket:

multiple.sh

```
curl http://localhost:11434/v1/systemone \
  -H 'Content-Type: application/json' \
  -d '{
    "model": "nimble",
    "state": {"ticket": "I was charged twice. Please refund the extra payment."},
    "questions": {
      "refund": {
        "type": "noul",
        "instructions": "Is the customer requesting a refund?",
        "criteria": {
          "false": "No refund is requested",
          "true": "The customer requests a refund"
        }
      },
      "urgency": {
        "type": "score",
        "instructions": "How urgently does this ticket need a response?",
        "criteria": [
          "Routine: no time pressure",
          "Soon: a customer is inconvenienced",
          "Immediate: a critical service is unavailable"
        ]
      }
    }
  }'
```

This returns a refund probability of `0.9989` and an urgency score of `0.8308` on the 0–2 scale (rounded). Results can vary with the model and configuration.

See the API reference for request fields, limits, errors, and token usage.

CLI Reference

Source: https://docs.ollama.com/cli

Run a model

```
ollama run gemma4
```

Launch integrations

```
ollama launch
```

Configure and launch external applications to use Ollama models. This provides an interactive way to set up and start integrations with supported apps.

Supported integrations

- **OpenCode** - Open-source coding assistant
- **Claude Code** - Anthropic's agentic coding tool
- **Codex** - OpenAI's coding assistant
- **VS Code** - Microsoft's IDE with built-in AI chat
- **Droid** - Factory's AI coding agent

Examples

Launch an integration interactively:

```
ollama launch
```

Launch a specific integration:

```
ollama launch claude
```

Launch with a specific model:

```
ollama launch claude --model qwen3.5
```

Configure without launching:

```
ollama launch droid --config
```

Multiline input

For multiline input, you can wrap text with `"""`:

```
>>> """Hello,
... world!
```

```
... """
I'm a basic program that prints the famous "Hello, world!" message to the console.
```

Multimodal models

```
ollama run gemma4 "What's in this image? /Users/jmorgan/Desktop/smile.png"
```

Generate embeddings

```
ollama run embeddinggemma "Hello world"
```

Output is a JSON array:

```
echo "Hello world" | ollama run nomic-embed-text
```

Ask a decision model

```
ollama run nimble --ask "Is a refund being requested?" < ticket.txt

ollama run nimble --ask "Which team should handle this?" \
  --choice billing="Payments and refunds" \
  --choice technical="Software problems" < ticket.txt

ollama run nimble --ask "How urgent is this?" \
  --level "Routine" \
  --level "Soon" \
  --level "Immediate" < ticket.txt
```

The state is piped in or given after the model name, as text or JSON. Each `--ask` question is answered yes or no, unless it's followed by labels to choose from (`--choice`) or levels to score it on (`--level`). See Decision.

Speak with a speech model

```
ollama run phonon "Hello from Ollama."

ollama run phonon --voice Zoey "Hi, this is Zoey."

ollama run phonon "Save this to a file." > speech.wav
```

The audio plays through your speakers, or is written as a WAV file when the output is redirected. Run it without text to speak each line you type. See Speech.

Download a model

```
ollama pull gemma4
```

Remove a model

```
ollama rm gemma4
```

List models

```
ollama ls
```

Sign in to Ollama

```
ollama signin
```

Sign out of Ollama

```
ollama signout
```

Create a customized model

First, create a `Modelfile`

```
FROM gemma4
SYSTEM """You are a happy cat."""
```

Then run `ollama create`:

```
ollama create -f Modelfile
```

List running models

```
ollama ps
```

Stop a running model

```
ollama stop gemma4
```

Start Ollama

```
ollama serve
```

To view a list of environment variables that can be set run `ollama serve --help`

Modelfile Reference

Source: https://docs.ollama.com/modelfile

A Modelfile is the blueprint to create and share customized models using Ollama.

Table of Contents

- Format
- Examples
- Instructions
- FROM (Required)
  - Build from existing model
  - Build from a Safetensors model
  - Build from a GGUF model
- PARAMETER
  - Valid Parameters and Values
- TEMPLATE
  - Template Variables
- SYSTEM
- LICENSE
- MESSAGE
- CAPABILITY
- Notes

Format

The format of the `Modelfile`:

```
# comment
INSTRUCTION arguments
```

Instruction	Description
`FROM` (required)	Defines the base model to use.
`PARAMETER`	Sets the parameters for how Ollama will run the model.
`TEMPLATE`	The full prompt template to be sent to the model.
`SYSTEM`	Specifies the system message that will be set in the template.
`LICENSE`	Specifies the legal license.
`MESSAGE`	Specify message history.
`REQUIRES`	Specify the minimum version of Ollama required by the model.
`CAPABILITY`	Declare an additional model capability.

Examples

Basic `Modelfile`

An example of a `Modelfile` creating a mario blueprint:

```
FROM llama3.2
# sets the temperature to 1 [higher is more creative, lower is more coherent]
PARAMETER temperature 1
# sets the context window size to 4096, this controls how many tokens the LLM can use as
context to generate the next token
PARAMETER num_ctx 4096

# sets a custom system message to specify the behavior of the chat assistant
SYSTEM You are Mario from super mario bros, acting as an assistant.
```

To use this:

1. Save it as a file (e.g. `Modelfile`)
2. `ollama create choose-a-model-name -f <location of the file e.g. ./Modelfile>`
3. `ollama run choose-a-model-name`
4. Start using the model!

To view the Modelfile of a given model, use the `ollama show --modelfile` command.

```
ollama show --modelfile llama3.2
```

```
# Modelfile generated by "ollama show"
# To build a new Modelfile based on this one, replace the FROM line with:
# FROM llama3.2:latest
FROM /Users/pdevine/.ollama/models/blobs/sha256-
00e1317cbf74d901080d7100f57580ba8dd8de57203072dc6f668324ba545f29
TEMPLATE """{{ if .System }}<|start_header_id|>system<|end_header_id|>

{{ .System }}<|eot_id|>{{ end }}{{ if .Prompt }}<|start_header_id|>user<|end_header_id|>

{{ .Prompt }}<|eot_id|>{{ end }}<|start_header_id|>assistant<|end_header_id|>

{{ .Response }}<|eot_id|>"""
PARAMETER stop "<|start_header_id|>"
PARAMETER stop "<|end_header_id|>"
PARAMETER stop "<|eot_id|>"
PARAMETER stop "<|reserved_special_token"
```

Instructions

FROM (Required)

The `FROM` instruction defines the base model to use when creating a model.

```
FROM <model name>:<tag>
```

Build from existing model

```
FROM llama3.2
```

Model library

Browse available models

Build from a Safetensors model

```
FROM <model directory>
```

The model directory should contain the Safetensors weights for a supported architecture.

Build from a GGUF model

```
FROM ./ollama-model.gguf
```

The GGUF file location should be specified as an absolute path or relative to the `Modelfile` location.

For a split GGUF model, keep the original split filenames and use a wildcard that matches every shard. Shards can also be listed with separate `FROM` instructions.

```
FROM ./model-*.gguf
```

PARAMETER

The `PARAMETER` instruction defines a parameter that can be set when the model is run.

```
PARAMETER <parameter> <parametervalue>
```

Valid Parameters and Values

Parameter	Description	Value Type	Example Usage
num_ctx	Sets the size of the context window used to generate the next token. (Default: 2048)	int	num_ctx 4096
repeat_last_n	Sets how far back for the model to look back to prevent repetition. (Default: 64, 0 = disabled, -1 = num_ctx)	int	repeat_last_n 64
repeat_penalty	Sets how strongly to penalize repetitions. A higher value (e.g., 1.5) will penalize repetitions more strongly, while a lower value (e.g., 0.9) will be more lenient. (Default: 1.0, disabled)	float	repeat_penalty 1.1
temperature	The temperature of the model. Increasing the temperature will make the model answer more creatively. (Default: 0.8)	float	temperature 0.7
seed	Sets the random number seed to use for generation. Setting this to a specific number will make the model generate the same text for the same prompt. (Default: 0)	int	seed 42
stop	Sets the stop sequences to use. When this pattern is encountered the LLM will stop generating text and return.	string	stop "AI assistant:"

Parameter	Description	Value Type	Example Usage
	Multiple stop patterns may be set by specifying multiple separate `stop` parameters in a modelfile.		
num_predict	Maximum number of tokens to predict when generating text. (Default: -1, infinite generation)	int	num_predict 42
draft_num_predict	Maximum number of speculative draft tokens to predict per step when a draft model is available. Separate draft models default to 4; embedded MTP tensors require setting this parameter. Set to 0 to disable speculative drafting.	int	draft_num_predict 4
top_k	Reduces the probability of generating nonsense. A higher value (e.g. 100) will give more diverse answers, while a lower value (e.g. 10) will be more conservative. (Default: 40)	int	top_k 40
top_p	Works together with top-k. A higher value (e.g., 0.95) will lead to more diverse text, while a lower value (e.g., 0.5) will generate more focused and conservative text. (Default: 0.9)	float	top_p 0.9
min_p	Alternative to the top*p, and aims to ensure a balance of quality and variety. The parameter _p* represents the minimum probability for a token to be considered, relative to the probability of the most likely token. For example, with *p*=0.05 and the most likely token having a probability of 0.9, logits with a value less than 0.045 are filtered out. (Default: 0.0)	float	min_p 0.05

TEMPLATE

`TEMPLATE` of the full prompt template to be passed into the model. It may include (optionally) a system message, a user's message and the response from the model. Note: syntax may be model specific. Templates use Go template syntax.

Template Variables

Variable	Description
`{{ .System }}`	The system message used to specify custom behavior.
`{{ .Prompt }}`	The user prompt message.

Variable	Description
`{{ .Response }}`	The response from the model. When generating a response, text after this variable is omitted.

```
TEMPLATE """{{ if .System }}<|im_start|>system
{{ .System }}<|im_end|>
{{ end }}{{ if .Prompt }}<|im_start|>user
{{ .Prompt }}<|im_end|>
{{ end }}<|im_start|>assistant
"""
```

SYSTEM

The `SYSTEM` instruction specifies the system message to be used in the template, if applicable.

```
SYSTEM """<system message>"""
```

LICENSE

The `LICENSE` instruction allows you to specify the legal license under which the model used with this Modelfile is shared or distributed.

```
LICENSE """
<license text>
"""
```

MESSAGE

The `MESSAGE` instruction allows you to specify a message history for the model to use when responding. Use multiple iterations of the MESSAGE command to build up a conversation which will guide the model to answer in a similar way.

```
MESSAGE <role> <message>
```

Valid roles

Role	Description
system	Alternate way of providing the SYSTEM message for the model.
user	An example message of what the user could have asked.
assistant	An example message of how the model should respond.

Example conversation

```
MESSAGE user Is Toronto in Canada?
MESSAGE assistant yes
MESSAGE user Is Sacramento in Canada?
MESSAGE assistant no
MESSAGE user Is Ontario in Canada?
MESSAGE assistant yes
```

CAPABILITY

Adds a capability to a GGUF or MLX model without removing inherited or inferred capabilities. Repeat the instruction to add more than one. Declare `decision` only for weights trained for System One's decision prompts:

```
CAPABILITY decision
```

REQUIRES

The `REQUIRES` instruction allows you to specify the minimum version of Ollama required by the model.

```
REQUIRES <version>
```

The version should be a valid Ollama version (e.g. 0.14.0).

Notes

- the **`Modelfile` is not case sensitive**. In the examples, uppercase instructions are used to make it easier to distinguish it from arguments.
- Instructions can be in any order. In the examples, the `FROM` instruction is first to keep it easily readable.

Context length

Source: https://docs.ollama.com/context-length

Context length is the maximum number of tokens that the model has access to in memory.

> **Note:** Ollama defaults to the following context lengths based on VRAM: - < 24 GiB VRAM: 4k context - 24-48 GiB VRAM: 32k context - >= 48 GiB VRAM: 256k context

Tasks which require large context like web search, agents, and coding tools should be set to at least 64000 tokens.

Setting context length

Setting a larger context length will increase the amount of memory required to run a model. Ensure you have enough VRAM available to increase the context length.

Cloud models are set to their maximum context length by default.

App

Change the slider in the Ollama app under settings to your desired context length.

CLI

If editing the context length for Ollama is not possible, the context length can also be updated when serving Ollama.

```
OLLAMA_CONTEXT_LENGTH=64000 ollama serve
```

Check allocated context length and model offloading

For best performance, use the maximum context length for a model, and avoid offloading the model to CPU. Verify the split under `PROCESSOR` using `ollama ps`.

```
ollama ps
```

```
NAME             ID              SIZE      PROCESSOR    CONTEXT    UNTIL
gemma4:latest    c6eb396dbd59    9.6 GB    100% GPU     131072     2 minutes from now
```

Linux

Source: https://docs.ollama.com/linux

Install

To install Ollama, run the following command:

```
curl -fsSL https://ollama.com/install.sh | sh
```

Manual install

> **Note:** If you are upgrading from a prior version, you should remove the old libraries with `sudo rm -rf /usr/lib/ollama` first.

Download and extract the package:

```
curl -fsSL https://ollama.com/download/ollama-linux-amd64.tar.zst \
    | sudo tar x -C /usr
```

Start Ollama:

```
ollama serve
```

In another terminal, verify that Ollama is running:

```
ollama -v
```

AMD GPU install

If you have an AMD GPU, also download and extract the additional ROCm package:

```
curl -fsSL https://ollama.com/download/ollama-linux-amd64-rocm.tar.zst \
    | sudo tar x -C /usr
```

ARM64 install

Download and extract the ARM64-specific package:

```
curl -fsSL https://ollama.com/download/ollama-linux-arm64.tar.zst \
    | sudo tar x -C /usr
```

Adding Ollama as a startup service (recommended)

Create a user and group for Ollama:

```
sudo useradd -r -s /bin/false -U -m -d /usr/share/ollama ollama
sudo usermod -a -G ollama $(whoami)
```

Create a service file in `/etc/systemd/system/ollama.service`:

```
[Unit]
Description=Ollama Service
After=network-online.target

[Service]
ExecStart=/usr/bin/ollama serve
User=ollama
Group=ollama
Restart=always
RestartSec=3
Environment="PATH=$PATH"

[Install]
WantedBy=multi-user.target
```

Then start the service:

```
sudo systemctl daemon-reload
sudo systemctl enable ollama
```

Install CUDA drivers (optional)

Download and install CUDA.

Verify that the drivers are installed by running the following command, which should print details about your GPU:

```
nvidia-smi
```

Install AMD ROCm drivers (optional)

Download and Install ROCm v7.

Start Ollama

Start Ollama and verify it is running:

```
sudo systemctl start ollama
sudo systemctl status ollama
```

> **Note:** While AMD has contributed the `amdgpu` driver upstream to the official linux kernel source, the version is older and may not support all ROCm features. We recommend you install the latest driver from https://www.amd.com/en/support/download/linux-drivers.html for best support of your Radeon GPU.

Customizing

To customize the installation of Ollama, you can edit the systemd service file or the environment variables by running:

```
sudo systemctl edit ollama
```

Alternatively, create an override file manually in `/etc/systemd/system/ollama.service.d/override.conf`:

```
[Service]
Environment="OLLAMA_DEBUG=1"
```

Updating

Update Ollama by running the install script again:

```
curl -fsSL https://ollama.com/install.sh | sh
```

Or by re-downloading Ollama:

```
curl -fsSL https://ollama.com/download/ollama-linux-amd64.tar.zst \
    | sudo tar x -C /usr
```

Installing specific versions

Use `OLLAMA_VERSION` environment variable with the install script to install a specific version of Ollama, including pre-releases. You can find the version numbers in the releases page.

For example:

```
curl -fsSL https://ollama.com/install.sh | OLLAMA_VERSION=0.5.7 sh
```

Viewing logs

To view logs of Ollama running as a startup service, run:

```
journalctl -e -u ollama
```

Uninstall

Remove the ollama service:

```
sudo systemctl stop ollama
sudo systemctl disable ollama
sudo rm /etc/systemd/system/ollama.service
```

Remove ollama libraries from your lib directory (either `/usr/local/lib`, `/usr/lib`, or `/lib`):

```
sudo rm -r $(which ollama | tr 'bin' 'lib')
```

Remove the ollama binary from your bin directory (either `/usr/local/bin`, `/usr/bin`, or `/bin`):

```
sudo rm $(which ollama)
```

Remove the downloaded models and Ollama service user and group:

```
sudo userdel ollama
sudo groupdel ollama
```

```
sudo rm -r /usr/share/ollama
```

macOS

Source: https://docs.ollama.com/macos

System Requirements

- MacOS Sonoma (v14) or newer
- Apple M series (CPU and GPU support) or x86 (CPU only)

Filesystem Requirements

The preferred method of installation is to mount the `ollama.dmg` and drag-and-drop the Ollama application to the system-wide `Applications` folder. Upon startup, the Ollama app will verify the `ollama` CLI is present in your PATH, and if not detected, will prompt for permission to create a link in `/usr/local/bin`

Once you've installed Ollama, you'll need additional space for storing the Large Language models, which can be tens to hundreds of GB in size. If your home directory doesn't have enough space, you can change where the binaries are installed, and where the models are stored.

Changing Install Location

To install the Ollama application somewhere other than `Applications`, place the Ollama application in the desired location, and ensure the CLI `Ollama.app/Contents/Resources/ollama` or a sym-link to the CLI can be found in your path. Upon first start decline the "Move to Applications?" request.

Troubleshooting

Ollama on MacOS stores files in a few different locations. - `~/.ollama` contains models and configuration - `~/.ollama/logs` contains logs - *app.log* contains most recent logs from the GUI application - *server.log* contains the most recent server logs - `<install location>/Ollama.app/Contents/Resources/ollama` the CLI binary

Uninstall

To fully remove Ollama from your system, remove the following files and folders:

```
sudo rm -rf /Applications/Ollama.app
sudo rm /usr/local/bin/ollama
rm -rf "~/Library/Application Support/Ollama"
rm -rf "~/Library/Saved Application State/com.electron.ollama.savedState"
rm -rf ~/Library/Caches/com.electron.ollama/
rm -rf ~/Library/Caches/ollama
rm -rf ~/Library/WebKit/com.electron.ollama
rm -rf ~/.ollama
```

Docker

Source: https://docs.ollama.com/docker

CPU only

```
docker run -d -v ollama:/root/.ollama -p 11434:11434 --name ollama ollama/ollama
```

Nvidia GPU

Install the NVIDIA Container Toolkit.

Install with Apt

1. Configure the repository

```
curl -fsSL https://nvidia.github.io/libnvidia-container/gpgkey \
    | sudo gpg --dearmor -o /usr/share/keyrings/nvidia-container-toolkit-keyring.gpg
curl -fsSL https://nvidia.github.io/libnvidia-container/stable/deb/nvidia-container-
toolkit.list \
    | sed 's#deb https://#deb [signed-by=/usr/share/keyrings/nvidia-container-toolkit-
keyring.gpg] https://#g' \
    | sudo tee /etc/apt/sources.list.d/nvidia-container-toolkit.list
sudo apt-get update
```

2. Install the NVIDIA Container Toolkit packages

```
sudo apt-get install -y nvidia-container-toolkit
```

Install with Yum or Dnf

1. Configure the repository

```
curl -fsSL https://nvidia.github.io/libnvidia-container/stable/rpm/nvidia-container-
toolkit.repo \
    | sudo tee /etc/yum.repos.d/nvidia-container-toolkit.repo
```

2. Install the NVIDIA Container Toolkit packages

```
sudo yum install -y nvidia-container-toolkit
```

Configure Docker to use Nvidia driver

```
sudo nvidia-ctk runtime configure --runtime=docker
sudo systemctl restart docker
```

Start the container

```
docker run -d --gpus=all -v ollama:/root/.ollama -p 11434:11434 --name ollama ollama/ollama
```

> **Note:** If you're running on an NVIDIA JetPack system, Ollama can't automatically discover the correct JetPack version. Pass the environment variable `JETSON_JETPACK=5` or `JETSON_JETPACK=6` to the container to select version 5 or 6.

AMD GPU

To run Ollama using Docker with AMD GPUs, use the `rocm` tag and the following command:

```
docker run -d --device /dev/kfd --device /dev/dri -v ollama:/root/.ollama -p 11434:11434 --
name ollama ollama/ollama:rocm
```

Vulkan Support

Vulkan is bundled into the `ollama/ollama` image and is enabled by default when the container can access the GPU devices.

```
docker run -d --device /dev/kfd --device /dev/dri -v ollama:/root/.ollama -p 11434:11434 --
name ollama ollama/ollama
```

Use `OLLAMA_VULKAN=0` to disable Vulkan, or `GGML_VK_VISIBLE_DEVICES=<ids>` to select specific Vulkan devices.

Run model locally

Now you can run a model:

```
docker exec -it ollama ollama run llama3.2
```

Try different models

More models can be found on the Ollama library.

Importing a Model

Source: https://docs.ollama.com/import

Table of Contents

- Importing a Safetensors model
- Importing a GGUF model
- Sharing models on ollama.com

Importing a model from Safetensors weights

First, create a `Modelfile` with a `FROM` command which points to the directory containing your Safetensors weights:

```
FROM /path/to/safetensors/directory
```

If you create the Modelfile in the same directory as the weights, you can use the command `FROM .`.

Now run the `ollama create` command from the directory where you created the `Modelfile`:

```
ollama create my-model
```

Lastly, test the model:

```
ollama run my-model
```

Importing a GGUF model

Ollama does not quantize GGUF models during import. Prepare and quantize them first with a GGUF tool such as llama.cpp's `llama-quantize`.

To import a single-file GGUF model, create a `Modelfile` containing:

```
FROM /path/to/file.gguf
```

For a split GGUF model, keep the original split filenames and use a wildcard that matches every shard.

```
FROM /path/to/model-*.gguf
```

Once you have created your `Modelfile`, use the `ollama create` command to build the model.

```
ollama create my-model
```

Sharing your model on ollama.com

You can share any model you have created by pushing it to ollama.com so that other users can try it out.

First, use your browser to go to the Ollama Sign-Up page. If you already have an account, you can skip this step.

The `Username` field will be used as part of your model's name (e.g. `jmorganca/mymodel`), so make sure you are comfortable with the username that you have selected.

Now that you have created an account and are signed-in, go to the Ollama Keys Settings page.

Follow the directions on the page to determine where your Ollama Public Key is located.

Click on the `Add Ollama Public Key` button, and copy and paste the contents of your Ollama Public Key into the text field.

To push a model to ollama.com, first make sure that it is named correctly with your username. You may have to use the `ollama cp` command to copy your model to give it the correct name. Once you're happy with your model's name, use the `ollama push` command to push it to ollama.com.

```
ollama cp mymodel myuser/mymodel
ollama push myuser/mymodel
```

Once your model has been pushed, other users can pull and run it by using the command:

```
ollama run myuser/mymodel
```

FAQ

Source: https://docs.ollama.com/faq

How can I upgrade Ollama?

Ollama on macOS and Windows will automatically download updates. Click on the taskbar or menubar item and then click "Restart to update" to apply the update. Updates can also be installed by downloading the latest version manually.

On Linux, re-run the install script:

```
curl -fsSL https://ollama.com/install.sh | sh
```

How can I view the logs?

Review the Troubleshooting docs for more about using logs.

Is my GPU compatible with Ollama?

Please refer to the GPU docs.

How can I specify the context window size?

By default, Ollama uses a context window size of 4096 tokens.

This can be overridden with the `OLLAMA_CONTEXT_LENGTH` environment variable. For example, to set the default context window to 8K, use:

```
OLLAMA_CONTEXT_LENGTH=8192 ollama serve
```

To change this when using `ollama run`, use `/set parameter`:

```
/set parameter num_ctx 4096
```

When using the API, specify the `num_ctx` parameter:

```
curl http://localhost:11434/api/generate -d '{
  "model": "llama3.2",
  "prompt": "Why is the sky blue?",
  "options": {
    "num_ctx": 4096
  }
}'
```

How can I tell if my model was loaded onto the GPU?

Use the `ollama ps` command to see what models are currently loaded into memory.

```
ollama ps
```

> **Info:**

Output:

```
NAME        ID            SIZE    PROCESSOR   UNTIL
llama3:70b  bcfb190ca3a7  42 GB   100% GPU    4 minutes from now
```

The `Processor` column will show which memory the model was loaded into:

- `100% GPU` means the model was loaded entirely into the GPU
- `100% CPU` means the model was loaded entirely in system memory
- `48%/52% CPU/GPU` means the model was loaded partially onto both the GPU and into system memory

How do I configure Ollama server?

Ollama server can be configured with environment variables.

Setting environment variables on Mac

If Ollama is run as a macOS application, environment variables should be set using `launchctl`:

1. For each environment variable, call `launchctl setenv`.

```
launchctl setenv OLLAMA_HOST "0.0.0.0:11434"
```

2. Restart Ollama application.

Setting environment variables on Linux

If Ollama is run as a systemd service, environment variables should be set using `systemctl`:

1. Edit the systemd service by calling `systemctl edit ollama.service`. This will open an editor.
2. For each environment variable, add a line `Environment` under section `[Service]`:

```
[Service]
Environment="OLLAMA_HOST=0.0.0.0:11434"
```

3. Save and exit.
4. Reload `systemd` and restart Ollama:

```
systemctl daemon-reload
systemctl restart ollama
```

Setting environment variables on Windows

On Windows, Ollama inherits your user and system environment variables.

1. First Quit Ollama by clicking on it in the task bar.
2. Start the Settings (Windows 11) or Control Panel (Windows 10) application and search for *environment variables*.

3. Click on *Edit environment variables for your account*.
4. Edit or create a new variable for your user account for `OLLAMA_HOST`, `OLLAMA_MODELS`, etc.
5. Click OK/Apply to save.
6. Start the Ollama application from the Windows Start menu.

How do I use Ollama behind a proxy?

Ollama pulls models from the Internet and may require a proxy server to access the models. Use `HTTPS_PROXY` to redirect outbound requests through the proxy. Ensure the proxy certificate is installed as a system certificate. Refer to the section above for how to use environment variables on your platform.

> **Note:** Avoid setting `HTTP_PROXY`. Ollama does not use HTTP for model pulls, only HTTPS. Setting `HTTP_PROXY` may interrupt client connections to the server.

How do I use Ollama behind a proxy in Docker?

The Ollama Docker container image can be configured to use a proxy by passing `-e HTTPS_PROXY=https://proxy.example.com` when starting the container.

Alternatively, the Docker daemon can be configured to use a proxy. Instructions are available for Docker Desktop on macOS, Windows, and Linux, and Docker daemon with systemd.

Ensure the certificate is installed as a system certificate when using HTTPS. This may require a new Docker image when using a self-signed certificate.

```
FROM ollama/ollama
COPY my-ca.pem /usr/local/share/ca-certificates/my-ca.crt
RUN update-ca-certificates
```

Build and run this image:

```
docker build -t ollama-with-ca .
docker run -d -e HTTPS_PROXY=https://my.proxy.example.com -p 11434:11434 ollama-with-ca
```

Does Ollama send my prompts and answers back to ollama.com?

Ollama runs locally. We don't see your prompts or data when you run locally. When using cloud-hosted models, we process your prompts and responses to provide the service but do not store or log that content and never train on it. We collect basic account info and limited usage metadata to provide the service that does not include prompt or response content. We don't sell your data. You can delete your account anytime.

How do I disable Ollama's cloud features?

Ollama can run in local only mode by disabling Ollama's cloud features. By turning off Ollama's cloud features, you will lose the ability to use Ollama's cloud models and web search.

Set `disable_ollama_cloud` in `~/.ollama/server.json`:

```
{
  "disable_ollama_cloud": true
}
```

You can also set the environment variable:

```
OLLAMA_NO_CLOUD=1
```

Restart Ollama after changing configuration. Once disabled, Ollama's logs will show `Ollama cloud disabled: true`.

How can I expose Ollama on my network?

Ollama binds 127.0.0.1 port 11434 by default. Change the bind address with the `OLLAMA_HOST` environment variable.

Refer to the section above for how to set environment variables on your platform.

How can I use Ollama with a proxy server?

Ollama runs an HTTP server and can be exposed using a proxy server such as Nginx. To do so, configure the proxy to forward requests and optionally set required headers (if not exposing Ollama on the network). For example, with Nginx:

```
server {
    listen 80;
    server_name example.com;  # Replace with your domain or IP
    location / {
        proxy_pass http://localhost:11434;
        proxy_set_header Host localhost:11434;
    }
}
```

How can I use Ollama with ngrok?

Ollama can be accessed using a range of tunneling apps. For example with Ngrok:

```
ngrok http 11434 --host-header="localhost:11434"
```

How can I use Ollama with Cloudflare Tunnel?

To use Ollama with Cloudflare Tunnel, use the `--url` and `--http-host-header` flags:

```
cloudflared tunnel --url http://localhost:11434 --http-host-header="localhost:11434"
```

How can I allow additional web origins to access Ollama?

Ollama allows cross-origin requests from `127.0.0.1` and `0.0.0.0` by default. Additional origins can be configured with `OLLAMA_ORIGINS`.

For browser extensions, you'll need to explicitly allow the extension's origin pattern. Set `OLLAMA_ORIGINS` to include `chrome-extension://*`, `moz-extension://*`, and `safari-web-extension://*` if you wish to allow all browser extensions access, or specific extensions as needed:

```
# Allow all Chrome, Firefox, and Safari extensions
OLLAMA_ORIGINS=chrome-extension://*,moz-extension://*,safari-web-extension://* ollama serve
```

Refer to the section above for how to set environment variables on your platform.

Where are models stored?

- macOS: `~/.ollama/models`
- Linux: `/usr/share/ollama/.ollama/models`
- Windows: `C:\Users\%username%\.ollama\models`

How do I set them to a different location?

If a different directory needs to be used, set the environment variable `OLLAMA_MODELS` to the chosen directory.

> **Note:** On Linux using the standard installer, the `ollama` user needs read and write access to the specified directory. To assign the directory to the `ollama` user run `sudo chown -R ollama:ollama <directory>`.

Refer to the section above for how to set environment variables on your platform.

How can I use Ollama in Visual Studio Code?

Install the Ollama extension to use Ollama models in VS Code Chat. See the VS Code integration guide for setup and troubleshooting.

How do I use Ollama with GPU acceleration in Docker?

The Ollama Docker container can be configured with GPU acceleration in Linux or Windows (with WSL2). This requires the nvidia-container-toolkit. See ollama/ollama for more details.

GPU acceleration is not available for Docker Desktop in macOS due to the lack of GPU passthrough and emulation.

Why is networking slow in WSL2 on Windows 10?

This can impact both installing Ollama, as well as downloading models.

Open `Control Panel > Networking and Internet > View network status and tasks` and click on `Change adapter settings` on the left panel. Find the `vEthernet (WSL)` adapter, right click and select `Properties`. Click on `Configure` and open the `Advanced` tab. Search through each of the properties until you find `Large Send Offload Version 2 (IPv4)` and `Large Send Offload Version 2 (IPv6)`. *Disable* both of these properties.

How can I preload a model into Ollama to get faster response times?

If you are using the API you can preload a model by sending the Ollama server an empty request. This works with both the `/api/generate` and `/api/chat` API endpoints.

To preload the mistral model using the generate endpoint, use:

```
curl http://localhost:11434/api/generate -d '{"model": "mistral"}'
```

To use the chat completions endpoint, use:

```
curl http://localhost:11434/api/chat -d '{"model": "mistral"}'
```

To preload a model using the CLI, use the command:

```
ollama run llama3.2 ""
```

How do I keep a model loaded in memory or make it unload immediately?

By default models are kept in memory for 5 minutes before being unloaded. This allows for quicker response times if you're making numerous requests to the LLM. If you want to immediately unload a model from memory, use the `ollama stop` command:

```
ollama stop llama3.2
```

If you're using the API, use the `keep_alive` parameter with the `/api/generate` and `/api/chat` endpoints to set the amount of time that a model stays in memory. The `keep_alive` parameter can be set to:

- a duration string (such as "10m" or "24h")
- a number in seconds (such as 3600)
- any negative number which will keep the model loaded in memory (e.g. -1 or "-1m")
- '0' which will unload the model immediately after generating a response

For example, to preload a model and leave it in memory use:

```
curl http://localhost:11434/api/generate -d '{"model": "llama3.2", "keep_alive": -1}'
```

To unload the model and free up memory use:

```
curl http://localhost:11434/api/generate -d '{"model": "llama3.2", "keep_alive": 0}'
```

Alternatively, you can change the amount of time all models are loaded into memory by setting the `OLLAMA_KEEP_ALIVE` environment variable when starting the Ollama server. The `OLLAMA_KEEP_ALIVE` variable uses the same parameter types as the `keep_alive` parameter types mentioned above. Refer to the section explaining how to configure the Ollama server to correctly set the environment variable.

The `keep_alive` API parameter with the `/api/generate` and `/api/chat` API endpoints will override the `OLLAMA_KEEP_ALIVE` setting.

How do I manage the maximum number of requests the Ollama server can queue?

If too many requests are sent to the server, it will respond with a 503 error indicating the server is overloaded. You can adjust how many requests may be queued by setting `OLLAMA_MAX_QUEUE`.

How does Ollama handle concurrent requests?

Ollama supports two levels of concurrent processing. If your system has sufficient available memory (system memory when using CPU inference, or VRAM for GPU inference) then multiple models can be loaded at the same time. For a given model, if there is sufficient available memory when the model is loaded, it is configured to allow parallel request processing.

If there is insufficient available memory to load a new model request while one or more models are already loaded, all new requests will be queued until the new model can be loaded. As prior models become idle, one or more will be unloaded to make room for the new model. Queued requests will be processed in order. When using GPU inference new models must be able to completely fit in VRAM to allow concurrent model loads.

Parallel request processing for a given model results in increasing the context size by the number of parallel requests. For example, a 2K context with 4 parallel requests will result in an 8K context and additional memory allocation.

The following server settings may be used to adjust how Ollama handles concurrent requests on most platforms:

- `OLLAMA_MAX_LOADED_MODELS` - The maximum number of models that can be loaded concurrently provided they fit in available memory. The default is 3 * the number of GPUs or 3 for CPU inference.
- `OLLAMA_NUM_PARALLEL` - The maximum number of parallel requests each model will process at the same time, default 1. Required RAM will scale by `OLLAMA_NUM_PARALLEL` * `OLLAMA_CONTEXT_LENGTH`.
- `OLLAMA_MAX_QUEUE` - The maximum number of requests Ollama will queue when busy before rejecting additional requests. The default is 512

Note: Windows with Radeon GPUs currently default to 1 model maximum due to limitations in ROCm v5.7 for available VRAM reporting. Once ROCm v6.2 is available, Windows Radeon will follow the defaults above. You may enable concurrent model loads on Radeon on Windows, but ensure you don't load more models than will fit into your GPU's VRAM.

How does Ollama load models on multiple GPUs?

When loading a new model, Ollama evaluates the required VRAM for the model against what is currently available. If the model will entirely fit on any single GPU, Ollama will load the model on that GPU. This typically provides the best performance as it reduces the amount of data transferring across the PCI bus during inference. If the model does not fit entirely on one GPU, then it will be spread across all the available GPUs.

How can I enable Flash Attention?

Flash Attention is a feature of most modern models that can significantly reduce memory usage as the context size grows. Ollama uses Flash Attention automatically when the selected backend and devices support it. To force Flash Attention on, set `OLLAMA_FLASH_ATTENTION=1` when starting the Ollama server. To disable it, set `OLLAMA_FLASH_ATTENTION=0`.

How can I set the quantization type for the K/V cache?

The K/V context cache can be quantized to significantly reduce memory usage when Flash Attention is enabled.

To use quantized K/V cache with Ollama you can set the following environment variable:

- `OLLAMA_KV_CACHE_TYPE` - The quantization type for the K/V cache. Default is `f16`.

> **Note:** Currently this is a global option - meaning all models will run with the specified quantization type.

The currently available K/V cache quantization types are:

- `f16` - high precision and memory usage (default).
- `q8_0` - 8-bit quantization, uses approximately 1/2 the memory of `f16` with a very small loss in precision, this usually has no noticeable impact on the model's quality (recommended if not using f16).
- `q4_0` - 4-bit quantization, uses approximately 1/4 the memory of `f16` with a small-medium loss in precision that may be more noticeable at higher context sizes.

How much the cache quantization impacts the model's response quality will depend on the model and the task. Models that have a high GQA count (e.g. Qwen2) may see a larger impact on precision from quantization than models with a low GQA count.

You may need to experiment with different quantization types to find the best balance between memory usage and quality.

Where can I find my Ollama Public Key?

Your **Ollama Public Key** is the public part of the key pair that lets your local Ollama instance talk to ollama.com.

You'll need it to: * Push models to Ollama * Pull private models from Ollama to your machine * Run models hosted in Ollama's cloud

How to Add the Key

- **Sign-in via the Settings page** in the **Mac** and **Windows App**
- **Sign-in via CLI**

```
ollama signin
```

- **Manually copy & paste** the key on the **Ollama Keys** page: https://ollama.com/settings/keys

Where the Ollama Public Key lives

OS	Path to `id_ed25519.pub`
macOS	`~/.ollama/id_ed25519.pub`
Linux	`/usr/share/ollama/.ollama/id_ed25519.pub`
Windows	`C:\Users\<username>\.ollama\id_ed25519.pub`

> **Note:** Replace <username> with your actual Windows user name.

How can I stop Ollama from starting when I login to my computer?

Ollama for Windows and macOS register as a login item during installation. You can disable this if you prefer not to have Ollama automatically start. Ollama will respect this setting across upgrades, unless you uninstall the application.

Windows - In `Task Manager` go to the `Startup apps` tab, search for `ollama` then click `Disable`

MacOS - Open `Settings` and search for "Login Items", find the `Ollama` entry under `Allow in the Background`, then click the slider to disable.

Hardware support

Source: https://docs.ollama.com/gpu

Nvidia

Ollama supports Nvidia GPUs with compute capability 5.0+ and driver version 550 and newer. Nvidia GPUs with compute capability 5.0 through 6.2 require driver version 570 or newer.

Check your compute compatibility to see if your card is supported: https://developer.nvidia.com/cuda-gpus

Compute Capability	Family	Cards
12.1	NVIDIA	`GB10 (DGX Spark)`
12.0	GeForce RTX 50xx	`RTX 5060` `RTX 5060 Ti` `RTX 5070` `RTX 5070 Ti` `RTX 5080` `RTX 5090`
	NVIDIA Professional	`RTX PRO 4000 Blackwell` `RTX PRO 4500 Blackwell` `RTX PRO 5000 Blackwell` `RTX PRO 6000 Blackwell`
9.0	NVIDIA	`H200` `H100`
8.9	GeForce RTX 40xx	`RTX 4090` `RTX 4080 SUPER` `RTX 4080` `RTX 4070 Ti SUPER` `RTX 4070 Ti` `RTX 4070 SUPER` `RTX 4070` `RTX 4060 Ti` `RTX 4060`
	NVIDIA Professional	`L4` `L40` `RTX 6000`
8.6	GeForce RTX 30xx	`RTX 3090 Ti` `RTX 3090` `RTX 3080 Ti` `RTX 3080` `RTX 3070 Ti` `RTX 3070` `RTX 3060 Ti` `RTX 3060` `RTX 3050 Ti` `RTX 3050`
	NVIDIA Professional	`A40` `RTX A6000` `RTX A5000` `RTX A4000` `RTX A3000` `RTX A2000` `A10` `A16` `A2`
8.0	NVIDIA	`A100` `A30`
7.5	GeForce GTX/RTX	`GTX 1650 Ti` `TITAN RTX` `RTX 2080 Ti` `RTX 2080` `RTX 2070` `RTX 2060`
	NVIDIA Professional	`T4` `RTX 5000` `RTX 4000` `RTX 3000` `T2000` `T1200` `T1000` `T600` `T500`
	Quadro	`RTX 8000` `RTX 6000` `RTX 5000` `RTX 4000`
7.0	NVIDIA	`TITAN V` `V100` `Quadro GV100`
6.1	NVIDIA TITAN	`TITAN Xp` `TITAN X`
	GeForce GTX	`GTX 1080 Ti` `GTX 1080` `GTX 1070 Ti` `GTX 1070` `GTX 1060` `GTX 1050 Ti` `GTX 1050`
	Quadro	`P6000` `P5200` `P4200` `P3200` `P5000` `P4000` `P3000` `P2200` `P2000` `P1000` `P620` `P600` `P500` `P520`

Compute Capability	Family	Cards
	Tesla	`P40` `P4`
6.0	NVIDIA	`Tesla P100` `Quadro GP100`
5.2	GeForce GTX	`GTX TITAN X` `GTX 980 Ti` `GTX 980` `GTX 970` `GTX 960` `GTX 950`
	Quadro	`M6000 24GB` `M6000` `M5000` `M5500M` `M4000` `M2200` `M2000` `M620`
	Tesla	`M60` `M40`
5.0	GeForce GTX	`GTX 750 Ti` `GTX 750` `NVS 810`
	Quadro	`K2200` `K1200` `K620` `M1200` `M520` `M5000M` `M4000M` `M3000M` `M2000M` `M1000M` `K620M` `M600M` `M500M`

For building locally to support older GPUs, see development.

GPU Selection

If you have multiple NVIDIA GPUs in your system and want to limit Ollama to use a subset, you can set `CUDA_VISIBLE_DEVICES` to a comma separated list of GPUs. Numeric IDs may be used, however ordering may vary, so UUIDs are more reliable. You can discover the UUID of your GPUs by running `nvidia-smi -L` If you want to ignore the GPUs and force CPU usage, use an invalid GPU ID (e.g., "-1")

Linux Suspend Resume

On linux, after a suspend/resume cycle, sometimes Ollama will fail to discover your NVIDIA GPU, and fallback to running on the CPU. You can workaround this driver bug by reloading the NVIDIA UVM driver with `sudo rmmod nvidia_uvm && sudo modprobe nvidia_uvm`

AMD Radeon

Ollama supports the following AMD GPUs via the ROCm library:

> **NOTE:** Additional AMD GPU support is provided by the Vulkan Library - see below.

Linux Support

Ollama requires the AMD ROCm v7 driver on Linux. You can install or upgrade using the `amdgpu-install` utility from AMD's ROCm documentation.

Family	Cards and accelerators
AMD Radeon RX	`9070 XT` `9070 GRE` `9070` `9060 XT` `9060 XT LP` `9060` `7900 XTX` `7900 XT` `7900 GRE` `7800 XT` `7700 XT` `7700` `7600 XT` `7600` `6950 XT` `6900 XTX` `6900XT` `6800 XT` `6800`
AMD Radeon AI PRO	`R9700` `R9600D`
AMD Radeon PRO	`W7900` `W7800` `W7700` `W7600` `W7500` `W6900X` `W6800X Duo` `W6800X` `W6800` `V620`
AMD Ryzen AI	`Ryzen AI Max+ 395` `Ryzen AI Max 390` `Ryzen AI Max 385` `Ryzen AI 9 HX 475` `Ryzen AI 9 HX 470`

Family	Cards and accelerators
	`Ryzen AI 9 465` `Ryzen AI 9 HX 375` `Ryzen AI 9 HX 370` `Ryzen AI 9 365`
AMD Instinct	`MI350X` `MI300X` `MI300A` `MI250X` `MI250` `MI210` `MI100`

Windows Support

Ollama requires an AMD ROCm v7 / HIP7-capable driver stack on Windows.

Family	Cards and accelerators
AMD Radeon RX	`7900 XTX` `7900 XT` `7900 GRE` `7800 XT` `7700 XT` `7600 XT` `7600`
AMD Radeon PRO	`W7900` `W7800` `W7700` `W7600` `W7500`

Overrides on Linux

Ollama leverages the AMD ROCm library, which does not support all AMD GPUs. In some cases you can force the system to try to use a similar LLVM target that is close. For example The Radeon RX 5400 is `gfx1034` (also known as 10.3.4) however, ROCm does not currently support this target. The closest support is `gfx1030`. You can use the environment variable `HSA_OVERRIDE_GFX_VERSION` with `x.y.z` syntax. So for example, to force the system to run on the RX 5400, you would set `HSA_OVERRIDE_GFX_VERSION="10.3.0"` as an environment variable for the server. If you have an unsupported AMD GPU you can experiment using the list of supported types below.

If you have multiple GPUs with different GFX versions, append the numeric device number to the environment variable to set them individually. For example, `HSA_OVERRIDE_GFX_VERSION_0=10.3.0` and `HSA_OVERRIDE_GFX_VERSION_1=11.0.0`

At this time, the known supported GPU types on linux are the following LLVM Targets. This table shows some example GPUs that map to these LLVM targets: | **LLVM Target** | **An Example GPU** | |----------------|---------------------| | gfx908 | Radeon Instinct MI100 | | gfx90a | Radeon Instinct MI210/MI250 | | gfx942 | Radeon Instinct MI300X/MI300A | | gfx950 | Radeon Instinct MI350X | | gfx1030 | Radeon PRO V620 | | gfx1100 | Radeon PRO W7900 | | gfx1101 | Radeon PRO W7700 | | gfx1102 | Radeon RX 7600 | | gfx1150 | Ryzen AI 9 HX 375 | | gfx1151 | Ryzen AI Max+ 395 | | gfx1200 | Radeon RX 9070 | | gfx1201 | Radeon RX 9070 XT |

Reach out on Discord or file an issue for additional help.

GPU Selection

If you have multiple AMD GPUs in your system and want to limit Ollama to use a subset, you can set `ROCR_VISIBLE_DEVICES` to a comma separated list of GPUs. You can see the list of devices with `rocminfo`. If you want to ignore the GPUs and force CPU usage, use an invalid GPU ID (e.g., "-1"). When available, use the `Uuid` to uniquely identify the device instead of numeric value.

Container Permission

In some Linux distributions, SELinux can prevent containers from accessing the AMD GPU devices. On the host system you can run `sudo setsebool container_use_devices=1` to allow containers to use devices.

Metal (Apple GPUs)

Ollama supports GPU acceleration on Apple devices via the Metal API.

Vulkan GPU Support

Additional GPU support on Windows and Linux is provided via Vulkan. Vulkan is enabled by default when the backend is installed. On Windows most GPU vendors drivers come bundled with Vulkan support and require no additional setup steps. Most Linux distributions require installing additional components, and you may have multiple options for Vulkan drivers between Mesa and GPU Vendor specific packages

- Linux Intel GPU Instructions - https://dgpu-docs.intel.com/driver/client/overview.html
- Linux AMD GPU Instructions - https://amdgpu-install.readthedocs.io/en/latest/install-script.html#specifying-a-vulkan-implementation

For AMD GPUs on some Linux distributions, you may need to add the `ollama` user to the `render` group.

The Ollama scheduler leverages available VRAM data reported by the GPU libraries to make optimal scheduling decisions. Vulkan requires additional capabilities or running as root to expose this available VRAM data. If neither root access or this capability are granted, Ollama will use approximate sizes of the models to make best effort scheduling decisions.

```
sudo setcap cap_perfmon+ep /usr/local/bin/ollama
```

GPU Selection

To select specific Vulkan GPU(s), you can set the environment variable `GGML_VK_VISIBLE_DEVICES` to one or more numeric IDs on the Ollama server as described in the FAQ. If you encounter any problems with Vulkan based GPUs, you can disable all Vulkan GPUs by setting `OLLAMA_VULKAN=0` or `GGML_VK_VISIBLE_DEVICES=-1`.

On mixed iGPU/dGPU systems where the Vulkan iGPU is unstable, keep Vulkan enabled and set `GGML_VK_VISIBLE_DEVICES` to the discrete GPU index. For example, use `GGML_VK_VISIBLE_DEVICES=1` when `Vulkan1` is the discrete GPU.

Troubleshooting

Source: https://docs.ollama.com/troubleshooting

Sometimes Ollama may not perform as expected. One of the best ways to figure out what happened is to take a look at the logs. Find the logs on **Mac** by running the command:

```
cat ~/.ollama/logs/server.log
```

On **Linux** systems with systemd, the logs can be found with this command:

```
journalctl -u ollama --no-pager --follow --pager-end
```

When you run Ollama in a **container**, the logs go to stdout/stderr in the container:

```
docker logs <container-name>
```

(Use `docker ps` to find the container name)

If manually running `ollama serve` in a terminal, the logs will be on that terminal.

When you run Ollama on **Windows**, there are a few different locations. You can view them in the explorer window by hitting `<cmd>+R` and type in:

- `explorer %LOCALAPPDATA%\Ollama` to view logs. The most recent server logs will be in `server.log` and older logs will be in `server-#.log`
- `explorer %LOCALAPPDATA%\Programs\Ollama` to browse the binaries (The installer adds this to your user PATH)
- `explorer %HOMEPATH%\.ollama` to browse where models and configuration is stored
- `explorer %TEMP%` where temporary executable files are stored in one or more `ollama*` directories

To enable additional debug logging to help troubleshoot problems, first **Quit the running app from the tray menu** then in a powershell terminal

```
$env:OLLAMA_DEBUG="1"
& "ollama app.exe"
```

Join the Discord for help interpreting the logs.

LLM libraries

Ollama includes multiple LLM libraries compiled for different GPUs and CPU vector features. Ollama tries to pick the best one based on the capabilities of your system. If this autodetection has problems, or you run into other problems (e.g. crashes in your GPU) you can workaround this by forcing a specific LLM library. `cpu_avx2` will perform the best, followed by `cpu_avx` an the slowest but most compatible is `cpu`. Rosetta emulation under MacOS will work with the `cpu` library.

In the server log, you will see a message that looks something like this (varies from release to release):

```
Dynamic LLM libraries [rocm_v6 cpu cpu_avx cpu_avx2 cuda_v11 rocm_v5]
```

Experimental LLM Library Override

You can set OLLAMA_LLM_LIBRARY to any of the available LLM libraries to bypass autodetection, so for example, if you have a CUDA card, but want to force the CPU LLM library with AVX2 vector support, use:

```
OLLAMA_LLM_LIBRARY="cpu_avx2" ollama serve
```

You can see what features your CPU has with the following.

```
cat /proc/cpuinfo| grep flags | head -1
```

Installing older or pre-release versions on Linux

If you run into problems on Linux and want to install an older version, or you'd like to try out a pre-release before it's officially released, you can tell the install script which version to install.

```
curl -fsSL https://ollama.com/install.sh | OLLAMA_VERSION=0.5.7 sh
```

Linux tmp noexec

If your system is configured with the "noexec" flag where Ollama stores its temporary executable files, you can specify an alternate location by setting OLLAMA_TMPDIR to a location writable by the user ollama runs as. For example OLLAMA_TMPDIR=/usr/share/ollama/

Linux docker

If Ollama initially works on the GPU in a docker container, but then switches to running on CPU after some period of time with errors in the server log reporting GPU discovery failures, this can be resolved by disabling systemd cgroup management in Docker. Edit `/etc/docker/daemon.json` on the host and add `"exec-opts": ["native.cgroupdriver=cgroupfs"]` to the docker configuration.

NVIDIA GPU Discovery

When Ollama starts up, it takes inventory of the GPUs present in the system to determine compatibility and how much VRAM is available. Sometimes this discovery can fail to find your GPUs. In general, running the latest driver will yield the best results.

Linux NVIDIA Troubleshooting

If you are using a container to run Ollama, make sure you've set up the container runtime first as described in docker

Sometimes the Ollama can have difficulties initializing the GPU. When you check the server logs, this can show up as various error codes, such as "3" (not initialized), "46" (device unavailable), "100" (no device), "999" (unknown), or others. The following troubleshooting techniques may help resolve the problem

- If you are using a container, is the container runtime working? Try `docker run --gpus all ubuntu nvidia-smi` - if this doesn't work, Ollama won't be able to see your NVIDIA GPU.
- Is the uvm driver loaded? `sudo nvidia-modprobe -u`
- Try reloading the nvidia_uvm driver - `sudo rmmod nvidia_uvm` then `sudo modprobe nvidia_uvm`

- Try rebooting
- Make sure you're running the latest nvidia drivers

If none of those resolve the problem, gather additional information and file an issue:

- Set `CUDA_ERROR_LEVEL=50` and try again to get more diagnostic logs
- Check dmesg for any errors `sudo dmesg | grep -i nvrm` and `sudo dmesg | grep -i nvidia`

AMD GPU Discovery

On linux, AMD GPU access typically requires `video` and/or `render` group membership to access the `/dev/kfd` device. If permissions are not set up correctly, Ollama will detect this and report an error in the server log.

When running in a container, in some Linux distributions and container runtimes, the ollama process may be unable to access the GPU. Use `ls -lnd /dev/kfd /dev/dri /dev/dri/*` on the host system to determine the **numeric** group IDs on your system, and pass additional `--group-add ...` arguments to the container so it can access the required devices. For example, in the following output `crw-rw---- 1 0 44 226, 0 Sep 16 16:55 /dev/dri/card0` the group ID column is `44`

If you are experiencing problems getting Ollama to correctly discover or use your GPU for inference, the following may help isolate the failure.

- `AMD_LOG_LEVEL=3` Enable info log levels in the AMD HIP/ROCm libraries. This can help show more detailed error codes that can help troubleshoot problems
- `OLLAMA_DEBUG=1` During GPU discovery additional information will be reported
- Check dmesg for any errors from amdgpu or kfd drivers `sudo dmesg | grep -i amdgpu` and `sudo dmesg | grep -i kfd`

AMD Driver Version Mismatch

If your AMD GPU is not detected on Linux and the server logs contain messages like:

```
msg="failure during GPU discovery" ... error="failed to finish discovery before timeout"
msg="bootstrap discovery took" duration=30s ...
```

This typically means the system's AMD GPU driver is too old. Ollama bundles ROCm 7 linux libraries which require a compatible ROCm 7 kernel driver. If the system is running an older driver (ROCm 6.x or earlier), GPU initialization will hang during device discovery and eventually time out, causing Ollama to fall back to CPU.

To resolve this, upgrade to the ROCm v7 driver using the `amdgpu-install` utility from AMD's ROCm documentation. After upgrading, reboot and restart Ollama.

Multiple AMD GPUs

If you experience gibberish responses when models load across multiple AMD GPUs on Linux, see the following guide.

- https://rocm.docs.amd.com/projects/radeon-ryzen/en/latest/docs/install/installrad/native_linux/mgpu.html#mgpu-known-issues-and-limitations

Windows Terminal Errors

Older versions of Windows 10 (e.g., 21H1) are known to have a bug where the standard terminal program does not display control characters correctly. This can result in a long string of strings like `←[?25h←[?25l` being displayed, sometimes erroring with `The parameter is incorrect` To resolve this problem, please update to Win 10 22H1 or newer.

Introduction

Source: https://docs.ollama.com/api/introduction

Use Ollama's API to build applications with cloud or local models.

Get started

Call cloud models with an API key. No Ollama installation required. Start with the API quickstart.

To run models on your computer, install Ollama and run a local model.

Base URLs

API	Direct cloud access	Local Ollama server
Ollama	`https://ollama.com/api`	`http://localhost:11434/api`
OpenAI compatibility	`https://ollama.com/v1`	`http://localhost:11434/v1`
Anthropic client base URL	`https://ollama.com`	`http://localhost:11434`

The Anthropic client appends `/v1/messages` to its base URL.

Cloud requests need an API key. Local requests do not. To use cloud models through your local server, sign in to Ollama.

Creating or deleting models requires a local Ollama server. For supported API features, see OpenAI compatibility and Anthropic compatibility.

Ollama API example

Set `OLLAMA_API_KEY` to your API key, then call a cloud model:

```
curl https://ollama.com/api/chat \
  -H "Authorization: Bearer $OLLAMA_API_KEY" \
  -H "Content-Type: application/json" \
  -d '{
    "model": "gemma4:31b",
    "messages": [{"role": "user", "content": "Why is the sky blue?"}],
    "stream": false
  }'
```

Read the answer from `message.content`. For local inference, use `http://localhost:11434/api/chat` with a downloaded model and omit the authorization header.

Libraries

Use Ollama's official libraries with either a local server or the hosted API:

- Python
- JavaScript

For direct cloud access, set the host to `https://ollama.com` and supply your API key in the `Authorization: Bearer` header.

Browse community libraries.

Versioning

Ollama's API isn't strictly versioned, but the API is expected to be stable and backwards compatible. Deprecations are rare and will be announced in the release notes.

Streaming

Source: https://docs.ollama.com/api/streaming

Certain API endpoints stream responses by default, such as `/api/generate`. These responses are provided in the newline-delimited JSON format (i.e. the `application/x-ndjson` content type). For example:


```
{"model":"gemma4","created_at":"2025-10-26T17:15:24.097767Z","response":"That","done":false}
{"model":"gemma4","created_at":"2025-10-26T17:15:24.109172Z","response":"'","done":false}
{"model":"gemma4","created_at":"2025-10-26T17:15:24.121485Z","response":"s","done":false}
{"model":"gemma4","created_at":"2025-10-26T17:15:24.132802Z","response":" a","done":false}
{"model":"gemma4","created_at":"2025-10-26T17:15:24.143931Z","response":"
fantastic","done":false}
{"model":"gemma4","created_at":"2025-10-26T17:15:24.155176Z","response":"
question","done":false}
{"model":"gemma4","created_at":"2025-10-26T17:15:24.166576Z","response":"!","done":true,
"done_reason": "stop"}
```


Disabling streaming

Streaming can be disabled by providing `{"stream": false}` in the request body for any endpoint that support streaming. This will cause responses to be returned in the `application/json` format instead:


```
{"model":"gemma4","created_at":"2025-10-26T17:15:24.166576Z","response":"That's a fantastic
question!","done":true}
```


When to use streaming vs non-streaming

Streaming (default): - Real-time response generation - Lower perceived latency - Better for long generations

Non-streaming: - Simpler to process - Better for short responses, or structured outputs - Easier to handle in some applications

Usage

Source: https://docs.ollama.com/api/usage

Ollama's API responses include metrics that can be used for measuring performance and model usage:

- `total_duration`: How long the response took to generate
- `load_duration`: How long the model took to load
- `prompt_eval_count`: How many input tokens were in the prompt
- `prompt_eval_cached_count`: How many prompt tokens were read from the cache
- `prompt_eval_duration`: How long it took to evaluate the uncached prompt tokens
- `eval_count`: How many output tokens were processes
- `eval_duration`: How long it took to generate the output tokens

All timing values are measured in nanoseconds.

Example response

For endpoints that return usage metrics, the response body will include the usage fields. For example, a non-streaming call to `/api/generate` may return the following response:

```
{
  "model": "gemma4",
  "created_at": "2025-10-17T23:14:07.414671Z",
  "response": "Hello! How can I help you today?",
  "done": true,
  "done_reason": "stop",
  "total_duration": 174560334,
  "load_duration": 101397084,
  "prompt_eval_count": 11,
  "prompt_eval_cached_count": 8,
  "prompt_eval_duration": 13074791,
  "eval_count": 18,
  "eval_duration": 52479709
}
```

For endpoints that return **streaming responses**, usage fields are included as part of the final chunk, where `done` is `true`.

Errors

Source: https://docs.ollama.com/api/errors

Status codes

Endpoints return appropriate HTTP status codes based on the success or failure of the request in the HTTP status line (e.g. `HTTP/1.1 200 OK` or `HTTP/1.1 400 Bad Request`). Common status codes are:

- `200`: Success
- `400`: Bad Request (missing parameters, invalid JSON, etc.)
- `404`: Not Found (model doesn't exist, etc.)
- `429`: Too Many Requests (e.g. when a rate limit is exceeded)
- `500`: Internal Server Error
- `502`: Bad Gateway (e.g. when a cloud model cannot be reached)

Error messages

Errors are returned in the `application/json` format with the following structure, with the error message in the `error` property:

```
{
  "error": "the model failed to generate a response"
}
```

Errors that occur while streaming

If an error occurs mid-stream, the error will be returned as an object in the `application/x-ndjson` format with an `error` property. Since the response has already started, the status code of the response will not be changed.

```
{"model":"gemma4","created_at":"2025-10-26T17:21:21.196249Z","response":" Yes","done":false}
{"model":"gemma4","created_at":"2025-10-26T17:21:21.207235Z","response":".","done":false}
{"model":"gemma4","created_at":"2025-10-26T17:21:21.219166Z","response":"I","done":false}
{"model":"gemma4","created_at":"2025-10-26T17:21:21.231094Z","response":"can","done":false}
{"error":"an error was encountered while running the model"}
```

OpenAI compatibility

Source: https://docs.ollama.com/api/openai-compatibility

Connect OpenAI clients to Ollama. Ollama supports a subset of the OpenAI API.

Direct cloud access

Set your API key in `OLLAMA_API_KEY`. Install the client with `pip install openai`. No Ollama installation required.

```python
from openai import OpenAI

client = OpenAI(
    base_url="https://ollama.com/v1",
    api_key=os.environ["OLLAMA_API_KEY"],
)

response = client.chat.completions.create(
    model="gemma4:31b",
    messages=[{"role": "user", "content": "Say hello in one sentence."}],
)
print(response.choices[0].message.content)
```

The same client supports stateless Responses requests:

```python
response = client.responses.create(
    model="gemma4:31b",
    input="Say hello in one sentence.",
)
print(response.output_text)
```

Choose a cloud model. The cloud API does not support stateful Responses, built-in web search through `/v1/responses`, or custom/freeform tool-call replay.

Local server usage

These examples connect to your local Ollama server. The client requires an API key value, but Ollama ignores it. To use cloud models through this server, sign in to Ollama.

Simple `/v1/chat/completions` example

basic.py

```python
from openai import OpenAI

client = OpenAI(
    base_url='http://localhost:11434/v1/',
    api_key='ollama',  # required but ignored
)

chat_completion = client.chat.completions.create(
    messages=[
        {
            'role': 'user',
            'content': 'Say this is a test',
        }
    ],
    model='gpt-oss:20b',
```

```
)
print(chat_completion.choices[0].message.content)
```

basic.sh

```
curl -X POST http://localhost:11434/v1/chat/completions \
-H "Content-Type: application/json" \
-d '{
  "model": "gpt-oss:20b",
  "messages": [{ "role": "user", "content": "Say this is a test" }]
}'
```

Simple `/v1/responses` example

responses.py

```
from openai import OpenAI

client = OpenAI(
    base_url='http://localhost:11434/v1/',
    api_key='ollama',  # required but ignored
)

responses_result = client.responses.create(
  model='qwen3:8b',
  input='Write a short poem about the color blue',
)
print(responses_result.output_text)
```

responses.sh

```
curl -X POST http://localhost:11434/v1/responses \
-H "Content-Type: application/json" \
-d '{
  "model": "qwen3:8b",
  "input": "Write a short poem about the color blue"
}'
```

`/v1/chat/completions` with vision example

vision.py

```
from openai import OpenAI

client = OpenAI(
    base_url='http://localhost:11434/v1/',
    api_key='ollama',  # required but ignored
)

response = client.chat.completions.create(
    model='qwen3-vl:8b',
    messages=[
        {
            'role': 'user',
            'content': [
                {'type': 'text', 'text': "What's in this image?"},
                {
                    'type': 'image_url',
                    'image_url':
'
wYAAAAAXNSR0IArs4c6QAAAARnQU1BAACxjwv8YQUAAA3VSURBVHgB7Z27r0zdG8fX743i1bi1ikMoFMQloXRpKFFIqI7
LH4BEQ+NWIkjQuSWCRIEoULk0gsK1kCBI0IhrQVT7tz/7zZo888yz1r7MnDl7z5xvsjkzm1r7nS5fvxjzs2Asm1nNfa7lkAsm7d+++
ffv2JHP79ey6dq1ars/yL4/
v27S0ejqwv+cUOGEGGpKHR37tzJCEpHV9tnT58+dXXXCJDdECBE2OjrqjH071hpNECjx4cMHVycM1Uhbv359B2F79+5158
6daxN/+pyRkRFXKyRDAqxEp4yMlDDzXG1NPnnyJKkThoK0VFd1ELZu3TrzXKxKfW7dMBQ6bcuWLW2v0VlHjx41z717927
ba22U9APcw7Nnz1oGEPeL3m3p2mTAYYnFmMOMXybPPXv2bNIPpFZr1NHn4HMw0KRBjg9NuRw95s8PEcz/6DZELQd/09C9
QGq5RsmSRybqkwHGjh07OsJSsYYm3ijPpyHzoiacg35MLdDSIS/O1yM778jOTwYUkKNHWUzUWaOsylE00MyI0fcnOwIdj
vtNdW/HZwNLGg+sR1kMepSNJXmIwxBZiG8tDTpEZzKg0GItNsosY8USkxDhD0Rinuiko2gfL/RbiD2LZAjU9zKQJj8RDR
```

```
0vJBR1/Phx9+PHj9Z7REF4nTZkxzX4LCXHrV271qXkBAPGfP/atWvu/PnzHe4C97F48eIsRLZ9+3a3f/9+87dwP1JxaF7
/3r17ba+5l4EcaVo0lj3SBq5kGTJSQmLWMjgYNei2GPT1MuMqGTDEFHzeQSP2wi/jGnkmPJ/nhccs44jvDAxpVcxnq0F6
eT8h4ni/iIWpR5lPyA6ETkNXoSukvpJAD3AsXLiwpZs49+fPn5ke4j10TqYvegSfn0OnafC+Tv9ooA/JPkgQysqQNBzag
XY55nO/oa1F7qvIPWkRL12WRpMWUvpVDYmxAPehxWSe8ZEXL20sadYIozfmNch4QJPAfeJgW3rNsnzphBKNJM2KKODo1r
VOMRYik5ETy3ix4qWNI81qAAirizgMIc+yhTytx0JWZuNI03qsrgWlGtwjoS9XwgUhWGyhUaRZZQNNIEwCiXD16tXcAHU
s79co0vSD8rrJCIW98pzvxpAWyyo3HYwqS0+H0BjStClcZJT5coMm6D2LOF8TolGJtK9fvyZpyiC5ePFi9nc/oJU4eiEP
0jVoAnHa9wyJycITMP78+eMeP37sXrx44d6+fdt6f82aNdkx1pg9e3Zb5W+RSRE+n+VjksQWifvVaTKFhn5O8my63K8Qa
bdv33b379/PiAP//vuvW7BggZszZ072/+TJk91YgkafPn166zXB1rQHFvouAWHq9z3SEevSUerqCn2/dDCeta2jxYbr69
evk4MHDyY7d+7MjhMnTiTPnz9Pfv/+nfQT2ggpO2dMF8cghuoM7Ygj5iWCqRlGFml0QC/ftGmTmzt3rmsaKDsgBSPh0/8
yPeLLBihLkOKJc0jp8H8vUzcxIA1k6QJ/c78tWEyj5P3o4u9+jywNPdJi5rAH9x0KHcl4Hg570eQp3+vHXGyrmEeigzQs
QsjavXt38ujRo44LQuDDhw+TW7duRS1HGgMxhNXHgflaNTOsHyKvHK5Ijo2jbFjJBQK9YwFd6RVMzfgRBmEfP37suBBm/
p49e1qjEP2mwTViNRo0VJWH1deMXcNK08uUjVUu7s/zRaL+oLNxz1bpANco4npUgX4G2eFbpDFyQoQxojBCpEGSytmOH8
qrH5Q9vuzD6ofQylkCUmh8DBAr+q8JCyVNtWQIidKQE9wNtLSQnS4jDSsxNHogzFuQBw4cyM61UKVsjfr3ooBkPSqqQHe
sUPWVtzi9/vQi1T+rJj7WiTz4Pt/l3LxUkr5P2VYZaZ4URpsE+st/dujQoaBBYokbrz/8TJNQYLSonrPS9kUaSkPeZyj1
AWSj+d+VBoy1pIWVNed8P0Ll/ee5HdGRhrHhR5GGN0r4LGZBaj8oFDJitBTJzIZgFcmU0Y8ytWMZMzJOaXUSrUs5RxKnr
xmbb5YXO9VGUhtpXldhEUogFr3IzIsvlpmdosVcGVGXFWp2oU9kLFL3dEkSz6NHEY1sjSRdIuDFWEhd8KxFqsRi1uM/nz
9/zpxnwlESONdg6dKlbsaMGS4EHFHtjFIDHwKOo46l4TxSuxgDzi+rE2jg+BaFruOX4HXa0Nnf1lwAPufZeF8/r6zD97W
K2qFnGjBxTw5qNGPxT+5T/r7/7RawFC3j4vTp09koCxkeHjqbHJqArmH5UrFKKksnxrK7FuRIs8STfBZv+luugXZ2pR/p
P9Ois4z+TiMzUUkUjD0iEi1fzX8GmXyuxUBRcaUfykV0YZnlJGKQpOiGB76x5GeWkWWJc3mOrK6S7xdND+W5N6XyaRgtW
JFe13GkaZnKOsYqGdOVVVbGupsyA/l7emTLHi7vwTdirNEt0qxnzAvBFcnQF16xh/TMpUuXHDowhlA9vQVraQhkudRdzO
nK+04ZSP3DUhVSP61YsaLtd/ks7ZgtPcXqPqEafHkdqa84X6aCeL7YWlv6edGFHb+ZFICPlljHhg0bKuk0CSvVznWsotR
u433alNdFrqG45ejoaPCaUkWERpLXjzFL2Rpllp7PJU2a/v7Ab8N05/9t27Z16KUqoFGsxnI9EosS2niSYg9SpU6B4JgT
rvVW1flt1sT+0ADIJU2maXzcUTraGCRaL1Wp9rUMk16PMom8QhruxzvZIegJjFU7LLCePfS8uaQdPny4jTTL0dbee5mYo
kQsXTIWNY46kuMbnt8Kmec+LGWtOVIl9cT1rCB0V8WqkjAsRwta93TbwNYoGKsUSChN44lgBNCoHLHzquYKrU6qZ8lolC
IN0Rh6cP0Q3U6I6IXILYOQI513hJaSKAorFpuHXJNfVlpRtmYBk1Su1obZr5dnKAO+L10Hrj3WZW+E3qh6IszE37F6EB+
68mGpvKm4eb9bFrlzrok7fvr0Kfv727dvWRmdVTJHw0qiiCUSZ6wCK+7XL/AcsgNyL74DQQ730sv78Su7+t/A36MdY0sW
5o40ahslXr58aZ5HtZB8GH64m9EmMZ7FpYw4T6QnrZfgenrhFxaSiSGXtPnz57e9TkNZLvTjeqhr734CNtrK41L40sUQc
kmj1lGKQ0rC37x544r8eNXRpnVE3ZZY7zXo8NomiO0ZUCj2uHz58rbXoZ6gc0uA+F6ZeKS/jhRDUq8MKrTho9fEkihMmh
xtBI1DxKFY9XLpVcSkfoi8JGnToZO5sU5aiDQIW716ddt7ZLYtMQlhECdBGXZZMWldY5BHm5xgAroWj4C0hbYkSc/jBmg
gIrXJWlZM6pSETsEPGqZOndr2uuuR5rF169a2HoHPdurUKZM4CO1WTPqaDaAd+GFGKdIQkxAn9RuEWcTRyN2KSUgiSgF5
aWzPTeA/lN5rZubMmR2bE4SIC4nJoltgAV/dVefZm72AtctUCJU2CMJ327hxY9t7EHbkyJFseq+EJSY16RPo3Dkq1kkr7
+q0bNmyDuLQcZBEPYmHVdOBiJyIlrRDq41YPWfXOxUysi5fvtyaj+2BpcnsUV/oSoEMOk2CQGlr4ckhBwaetBhjCwH0ZH
tJROPJkyc7UjcYLDjmrH7ADTEBXFfOYmB0k9oYBOjJ8b4aOYSe7QkKcYhFlq3QYLQhSidNmtS2RATwy8YOM3EQJsUjKia
WZ+vZToUQgzhkHXudb/PW5YMHD9yZM2faPsMwoc7RciYJXbGuBqJ1UIGKKLv915jsvgtJxCZDubdXr165mzdvtr1Hz5LO
NA8jrUwKPqsmVesKa49S3Q4WxmRPUEYdTjgiUcfUwLx589ySJUva3oMkP6IYddq6HMS4o55xBJBUeRjzfa4Zdeg56QZ43
LhxoyPo7Lf1kNt7oO8wWAbNwaYjIv5lhyS7kRf96dvm5Jah8vfvX3flyhX35cuX6HfzFHOToS1H4BenCaHvO8pr8iDuwo
UL7tevX+b5ZdbBair0xkFIlFDlW4ZknEClsp/TzXyAKVOmmHWFVSbDNw1l1+4f90U6IY/q4V27dpnE9bJ+v87QEydjqx/
UamVVPRG+mwkNTYN+9tjkwzEx+atCm/X9WvWtDtAb68Wy9LXa1UmvCDDIpPkyOQ5ZwSzJ4jMrvFcr0rSjOUh+GcT4LSg5
ugkW1Io0/SCDQBojh0hPlaJdah+tkVYrnTZowP8iq1F1TgMBBauufyB33x1v+NWFYmT5KmppgHC+NkAgbmRkpD3yn9QIs
eXymoTQFGQmIOKTxiZIWpvAatenVqRVXf2nTrAWMsPnKrMZHz6bJq5jvce6QK8J1cQNgKxlJapMPdZSR64/UivS9Nztpk
VEdKcrs5alhhWP9NeqlfWopzhZScI6QxseegZRGeg5a8C3Re1Mfl1ScP36ddcUaMuv24iOJtz7sbUjTS4qBvKmstYJoUa
uiuD3k5qhyr7QdUHMeCgLa1Ear9NquemdXgmum4fvJ6w1lqsuDhNrg1qSpleJK7K3TF0Q2jSd94uSZ60kK1e3qyVpQK6P
VWXp2/FC3mp6jBhKKOiY2h3gtUV64TWM6wDETRPLDfSakXmH3w8g9Jlug8ZtTt4kVF0kLUYYmCCtD/DrQ5YhMGbA9L3uc
djh0y8kOHW5gU/VEEmJTcL4Pz/f7mgoAbYkAAAAAElFTkSuQmCC',
                    },
                ],
            }
    ],
    max_tokens=300,
)
print(response.choices[0].message.content)
```

vision.sh

```
curl -X POST http://localhost:11434/v1/chat/completions \
-H "Content-Type: application/json" \
-d '{
  "model": "qwen3-vl:8b",
  "messages": [{ "role": "user", "content": [{"type": "text", "text": "What is this an image
of?"}, {"type": "image_url", "image_url":
"
wYAAAAAXNSR0IArs4c6QAAAARnQU1BAACxjwv8YQUAAA3VSURBVHgB7Z27r0zdG8fX743i1bi1ikMoFMQloXRpKFFIqI7
LH4BEQ+NWIkjQuSWCRIEoUlk0gsK1kCBI0IhrQVT7tz/7zZo888yz1ikMoFMQloXRpKFFIqI7
Sffft38ujRo44LQuDDhw+TW7dbuWLWMjgYNei2GPT1MuMqGTDEFHzeQSP2wi/jGnkmPJ/nhccs44jvDAxpVcxnq0F6
eT8h4ni/iIWpR5lPyA6ETkNXoSukvpJAD3AsXLiwpZs49+fPn5ke4j10TqYvegSfn0OnafC+Tv9ooA/JPkgQysqQNBzag
XY55nO/oa1F7qvIPWkRL12WRpMWUvpVDYmxAPehxWSe8ZEXL20sadYIozfmNch4QJPAfeJgW3rNsnzphBKNJM2KKODo1r
VOMRYik5ETy3ix4qWNI81qAAirizgMIc+yhTytx0JWZuNI03qsrgWlGtwjoS9XwgUhWGyhUaRZZQNNIEwCiXD16tXcAHU
s79co0vSD8rrJCIW98pzvxpAWyyo3HYwqS0+H0BjStClcZJT5coMm6D2LOF8TolGJtK9fvyZpyiC5ePFi9nc/oJU4eiEP
0jVoAnHa9wyJycITMP78+eMeP37sXrx44d6+fdt6f82aNdkx1pg9e3Zb5W+RSRE+n+VjksQWifvVaTKFhn5O8my63K8Qa
bdv33b379/PiAP//vuvW7BggZszZ072/+TJk91YgkafPn166zXB1rQHFvouAWHq9z3SEevSUerqCn2/dDCeta2jxYbr69
evk4MHDyY7d+7MjhMnTiTPnz9Pfv/+nfQT2ggpO2dMF8cghuoM7Ygj5iWCqRlGFml0QC/ftGmTmzt3rmsaKDsgBSPh0/8
yPeLLBihLkOKJc0jp8H8vUzcxIA1k6QJ/c78tWEyj5P3o4u9+jywNPdJi5rAH9x0KHcl4Hg570eQp3+vHXGyrmEeigzQs
QsjavXt38ujRo44LQuDDhw+TW7duRS1HGgMxhNXHgflaNTOsHyKvHK5Ijo2jbFjJBQK9YwFd6RVMzfgRBmEfP37suBBm/
p49e1qjEP2mwTViNRo0VJWH1deMXcNK08uUjVUu7s/zRaL+oLNxz1bpANco4npUgX4G2eFbpDFyQoQxojBCpEGSytmOH8
qrH5Q9vuzD6ofQylkCUmh8DBAr+q8JCyVNtWQIidKQE9wNtLSQnS4jDSsxNHogzFuQBw4cyM61UKVsjfr3ooBkPSqqQHe
sUPWVtzi9/vQi1T+rJj7WiTz4Pt/l3LxUkr5P2VYZaZ4URpsE+st/dujQoaBBYokbrz/8TJNQYLSonrPS9kUaSkPeZyj1
AWSj+d+VBoy1pIWVNed8P0Ll/ee5HdGRhrHhR5GGN0r4LGZBaj8oFDJitBTJzIZgFcmU0Y8ytWMZMzJOaXUSrUs5RxKnr
xmbb5YXO9VGUhtpXldhEUogFr3IzIsvlpmdosVcGVGXFWp2oU9kLFL3dEkSz6NHEY1sjSRdIuDFWEhd8KxFqsRi1uM/nz
9/zpxnwlESONdg6dKlbsaMGS4EHFHtjFIDHwKOo46l4TxSuxgDzi+rE2jg+BaFruOX4HXa0Nnf1lwAPufZeF8/r6zD97W
K2qFnGjBxTw5qNGPxT+5T/r7/7RawFC3j4vTp09koCxkeHjqbHJqArmH5UrFKKksnxrK7FuRIs8STfBZv+luugXZ2pR/p
P9Ois4z+TiMzUUkUjD0iEi1fzX8GmXyuxUBRcaUfykV0YZnlJGKQpOiGB76x5GeWkWWJc3mOrK6S7xdND+W5N6XyaRgtW
JFe13GkaZnKOsYqGdOVVVbGupsyA/l7emTLHi7vwTdirNEt0qxnzAvBFcnQF16xh/TMpUuXHDowhlA9vQVraQhkudRdzO
nK+04ZSP3DUhVSP61YsaLtd/ks7ZgtPcXqPqEafHkdqa84X6aCeL7YWlv6edGFHb+ZFICPlljHhg0bKuk0CSvVznWsotR
u433alNdFrqG45ejoaPCaUkWERpLXjzFL2Rpllp7PJU2a/v7Ab8N05/9t27Z16KUqoFGsxnI9EosS2niSYg9SpU6B4JgT
rvVW1flt1sT+0ADIJU2maXzcUTraGCRaL1Wp9rUMk16PMom8QhruxzvZIegJjFU7LLCePfS8uaQdPny4jTTL0dbee5mYo
kQsXTIWNY46kuMbnt8Kmec+LGWtOVIl9cT1rCB0V8WqkjAsRwta93TbwNYoGKsUSChN44lgBNCoHLHzquYKrU6qZ8lolC
IN0Rh6cP0Q3U6I6IXILYOQI513hJaSKAorFpuHXJNfVlpRtmYBk1Su1obZr5dnKAO+L10Hrj3WZW+E3qh6IszE37F6EB+
68mGpvKm4eb9bFrlzrok7fvr0Kfv727dvWRmdVTJHw0qiiCUSZ6wCK+7XL/AcsgNyL74DQQ730sv78Su7+t/A36MdY0sW
5o40ahslXr58aZ5HtZB8GH64m9EmMZ7FpYw4T6QnrZfgenrhFxaSiSGXtPnz57e9TkNZLvTjeqhr734CNtrK41L40sUQc
kmj1lGKQ0rC37x544r8eNXRpnVE3ZZY7zXo8NomiO0ZUCj2uHz58rbXoZ6gc0uA+F6ZeKS/jhRDUq8MKrTho9fEkihMmh
xtBI1DxKFY9XLpVcSkfoi8JGnToZO5sU5aiDQIW716ddt7ZLYtMQlhECdBGXZZMWldY5BHm5xgAroWj4C0hbYkSc/jBmg
gIrXJWlZM6pSETsEPGqZOndr2uuuR5rF169a2HoHPdurUKZM4CO1WTPqaDaAd+GFGKdIQkxAn9RuEWcTRyN2KSUgiSgF5
aWzPTeA/lN5rZubMmR2bE4SIC4nJoltgAV/dVefZm72AtctUCJU2CMJ327hxY9t7EHbkyJFseq+EJSY16RPo3Dkq1kkr7
+q0bNmyDuLQcZBEPYmHVdOBiJyIlrRDq41YPWfXOxUysi5fvtyaj+2BpcnsUV/oSoEMOk2CQGlr4ckhBwaetBhjCwH0ZH
tJROPJkyc7UjcYLDjmrH7ADTEBXFfOYmB0k9oYBOjJ8b4aOYSe7QkKcYhFlq3QYLQhSidNmtS2RATwy8YOM3EQJsUjKia
WZ+vZToUQgzhkHXudb/PW5YMHD9yZM2faPsMwoc7RciYJXbGuBqJ1UIGKKLv915jsvgtJxCZDubdXr165mzdvtr1Hz5LO
NA8jrUwKPqsmVesKa49S3Q4WxmRPUEYdTjgiUcfUwLx589ySJUva3oMkP6IYddq6HMS4o55xBJBUeRjzfa4Zdeg56QZ43
LhxoyPo7Lf1kNt7oO8wWAbNwaYjIv5lhyS7kRf96dvm5Jah8vfvX3flyhX35cuX6HfzFHOToS1H4BenCaHvO8pr8iDuwo
UL7tevX+b5ZdbBair0xkFIlFDlW4ZknEClsp/TzXyAKVOmmHWFVSbDNw1l1+4f90U6IY/q4V27dpnE9bJ+v87QEydjqx/
UamVVPRG+mwkNTYN+9tjkwzEx+atCm/X9WvWtDtAb68Wy9LXa1UmvCDDIpPkyOQ5ZwSzJ4jMrvFcr0rSjOUh+GcT4LSg5
ugkW1Io0/SCDQBojh0hPlaJdah+tkVYrnTZowP8iq1F1TgMBBauufyB33x1v+NWFYmT5KmppgHC+NkAgbmRkpD3yn9QIs
eXymoTQFGQmIOKTxiZIWpvAatenVqRVXf2nTrAWMsPnKrMZHz6bJq5jvce6QK8J1cQNgKxlJapMPdZSR64/UivS9Nztpk
VEdKcrs5alhhWP9NeqlfWopzhZScI6QxseegZRGeg5a8C3Re1Mfl1ScP36ddcUaMuv24iOJtz7sbUjTS4qBvKmstYJoUa
uiuD3k5qhyr7QdUHMeCgLa1Ear9NquemdXgmum4fvJ6w1lqsuDhNrg1qSpleJK7K3TF0Q2jSd94uSZ60kK1e3qyVpQK6P
VWXp2/FC3mp6jBhKKOiY2h3gtUV64TWM6wDETRPLDfSakXmH3w8g9Jlug8ZtTt4kVF0kLUYYmCCtD/DrQ5YhMGbA9L3uc
djh0y8kOHW5gU/VEEmJTcL4Pz/f7mgoAbYkAAAAAElFTkSuQmCC"
  }]
}'
```

```
VOMRYik5ETy3ix4qWNI81qAAirizgMIc+yhTytx0JWZuNI03qsrgWlGtwjoS9XwgUhWGyhUaRZZQNNIEwCiXD16tXcAHU
s79co0vSD8rrJCIW98pzvxpAWyyo3HYwqS0+H0BjStClcZJT5coMm6D2LOF8TolGJtK9fvyZpyiC5ePFi9nc/oJU4eiEP
0jVoAnHa9wyJycITMP78+eMeP37sXrx44d6+fdt6f82aNdkx1pg9e3Zb5W+RSRE+n+VjksQWifvVaTKFhn5O8my63K8Qa
bdv33b379/PiAP//vuvW7BggZszZ072/+TJk91YgkafPn166zXB1rQHFvouAWHq9z3SEevSUerqCn2/dDCeta2jxYbr69
evk4MHDyY7d+7MjhMnTiTPnz9Pfv/+nfQT2ggpO2dMF8cghuoM7Ygj5iWCqRlGFml0QC/ftGmTmzt3rmsaKDsgBSPh0/8
yPeLLBihLkOKJc0jp8H8vUzcxIA1k6QJ/c78tWEyj5P3o4u9+jywNPdJi5rAH9x0KHcl4Hg570eQp3+vHXGyrmEeigzQs
QsjavXt38ujRo44LQuDDhw+TW7duRS1HGgMxhNXHgflaNTOsHyKvHK5Ijo2jbFjJBQK9YwFd6RVMzfgRBmEfP37suBBm/
p49e1qjEP2mwTViNRo0VJWH1deMXcNK08uUjVUu7s/zRaL+oLNxz1bpANco4npUgX4G2eFbpDFyQoQxojBCpEGSytmOH8
qrH5Q9vuzD6ofQylkCUmh8DBAr+q8JCyVNtWQIidKQE9wNtLSQnS4jDSsxNHogzFuQBw4cyM61UKVsjfr3ooBkPSqqQHe
sUPWVtzi9/vQi1T+rJj7WiTz4Pt/l3LxUkr5P2VYZaZ4URpsE+st/dujQoaBBYokbrz/8TJNQYLSonrPS9kUaSkPeZyj1
AWSj+d+VBoy1pIWVNed8P0Ll/ee5HdGRhrHhR5GGN0r4LGZBaj8oFDJitBTJzIZgFcmU0Y8ytWMZMzJOaXUSrUs5RxKnr
xmbb5YXO9VGUhtpXldhEUogFr3IzIsvlpmdosVcGVGXFWp2oU9kLFL3dEkSz6NHEY1sjSRdIuDFWEhd8KxFqsRi1uM/nz
9/zpxnwlESONdg6dKlbsaMGS4EHFHtjFIDHwKOo46l4TxSuxgDzi+rE2jg+BaFruOX4HXa0Nnf1lwAPufZeF8/r6zD97W
K2qFnGjBxTw5qNGPxT+5T/r7/7RawFC3j4vTp09koCxkeHjqbHJqArmH5UrFKKksnxrK7FuRIs8STfBZv+luugXZ2pR/p
P9Ois4z+TiMzUUkUjD0iEi1fzX8GmXyuxUBRcaUfykV0YZnlJGKQpOiGB76x5GeWkWWJc3mOrK6S7xdND+W5N6XyaRgtW
JFe13GkaZnKOsYqGdOVVVbGupsyA/l7emTLHi7vwTdirNEt0qxnzAvBFcnQF16xh/TMpUuXHDowhlA9vQVraQhkudRdzO
nK+04ZSP3DUhVSP61YsaLtd/ks7ZgtPcXqPqEafHkdqa84X6aCeL7YWlv6edGFHb+ZFICPlljHhg0bKuk0CSvVznWsotR
u433alNdFrqG45ejoaPCaUkWERpLXjzFL2Rpllp7PJU2a/v7Ab8N05/9t27Z16KUqoFGsxnI9EosS2niSYg9SpU6B4JgT
rvVW1flt1sT+0ADIJU2maXzcUTraGCRaL1Wp9rUMk16PMom8QhruxzvZIegJjFU7LLCePfS8uaQdPny4jTTL0dbee5mYo
kQsXTIWNY46kuMbnt8Kmec+LGWtOVIl9cT1rCB0V8WqkjAsRwta93TbwNYoGKsUSChN44lgBNCoHLHzquYKrU6qZ8lolC
IN0Rh6cP0Q3U6I6IXILYOQI513hJaSKAorFpuHXJNfVlpRtmYBk1Su1obZr5dnKAO+L10Hrj3WZW+E3qh6IszE37F6EB+
68mGpvKm4eb9bFrlzrok7fvr0Kfv727dvWRmdVTJHw0qiiCUSZ6wCK+7XL/AcsgNyL74DQQ730sv78Su7+t/A36MdY0sW
5o40ahslXr58aZ5HtZB8GH64m9EmMZ7FpYw4T6QnrZfgenrhFxaSiSGXtPnz57e9TkNZLvTjeqhr734CNtrK41L40sUQc
kmj1lGKQ0rC37x544r8eNXRpnVE3ZZY7zXo8NomiO0ZUCj2uHz58rbXoZ6gc0uA+F6ZeKS/jhRDUq8MKrTho9fEkihMmh
xtBI1DxKFY9XLpVcSkfoi8JGnToZO5sU5aiDQIW716ddt7ZLYtMQlhECdBGXZZMWldY5BHm5xgAroWj4C0hbYkSc/jBmg
gIrXJWlZM6pSETsEPGqZOndr2uuuR5rF169a2HoHPdurUKZM4CO1WTPqaDaAd+GFGKdIQkxAn9RuEWcTRyN2KSUgiSgF5
aWzPTeA/lN5rZubMmR2bE4SIC4nJoltgAV/dVefZm72AtctUCJU2CMJ327hxY9t7EHbkyJFseq+EJSY16RPo3Dkq1kkr7
+q0bNmyDuLQcZBEPYmHVdOBiJyIlrRDq41YPWfXOxUysi5fvtyaj+2BpcnsUV/oSoEMOk2CQGlr4ckhBwaetBhjCwH0ZH
tJROPJkyc7UjcYLDjmrH7ADTEBXFfOYmB0k9oYBOjJ8b4aOYSe7QkKcYhFlq3QYLQhSidNmtS2RATwy8YOM3EQJsUjKia
WZ+vZToUQgzhkHXudb/PW5YMHD9yZM2faPsMwoc7RciYJXbGuBqJ1UIGKKLv915jsvgtJxCZDubdXr165mzdvtr1Hz5LO
NA8jrUwKPqsmVesKa49S3Q4WxmRPUEYdTjgiUcfUwLx589ySJUva3oMkP6IYddq6HMS4o55xBJBUeRjzfa4Zdeg56QZ43
LhxoyPo7Lf1kNt7oO8wWAbNwaYjIv5lhyS7kRf96dvm5Jah8vfvX3flyhX35cuX6HfzFHOToS1H4BenCaHvO8pr8iDuwo
UL7tevX+b5ZdbBair0xkFIlFDlW4ZknEClsp/TzXyAKVOmmHWFVSbDNw1l1+4f90U6IY/q4V27dpnE9bJ+v87QEydjqx/
UamVVPRG+mwkNTYN+9tjkwzEx+atCm/X9WvWtDtAb68Wy9LXa1UmvCDDIpPkyOQ5ZwSzJ4jMrvFcr0rSjOUh+GcT4LSg5
ugkW1Io0/SCDQBojh0hPlaJdah+tkVYrnTZowP8iq1F1TgMBBauufyB33x1v+NWFYmT5KmppgHC+NkAgbmRkpD3yn9QIs
eXymoTQFGQmIOKTxiZIWpvAatenVqRVXf2nTrAWMsPnKrMZHz6bJq5jvce6QK8J1cQNgKxlJapMPdZSR64/UivS9Nztpk
VEdKcrs5alhhWP9NeqlfWopzhZScI6QxseegZRGeg5a8C3Re1Mfl1ScP36ddcUaMuv24iOJtz7sbUjTS4qBvKmstYJoUa
uiuD3k5qhyr7QdUHMeCgLa1Ear9NquemdXgmum4fvJ6w1lqsuDhNrg1qSpleJK7K3TF0Q2jSd94uSZ60kK1e3qyVpQK6P
VWXp2/FC3mp6jBhKKOiY2h3gtUV64TWM6wDETRPLDfSakXmH3w8g9Jlug8ZtTt4kVF0kLUYYmCCtD/DrQ5YhMGbA9L3uc
djh0y8kOHW5gU/VEEmJTcL4Pz/f7mgoAbYkAAAAAElFTkSuQmCC"}]}]
}'
```

Endpoints

`/v1/chat/completions`

Supported features

- [x] Chat completions
- [x] Streaming
- [x] JSON mode
- [x] Reproducible outputs
- [x] Vision
- [x] Tools
- [x] Reasoning/thinking control (for thinking models)
- [ ] Logprobs

Supported request fields

- [x] `model`
- [x] `messages`
- [x] Text `content`

- [x] Image `content`
  - [x] Base64 encoded image
  - [ ] Image URL
- [x] Array of `content` parts
- [x] `frequency_penalty`
- [x] `presence_penalty`
- [x] `response_format`
- [x] `seed`
- [x] `stop`
- [x] `stream`
- [x] `stream_options`
- [x] `include_usage`
- [x] `temperature`
- [x] `top_p`
- [x] `max_tokens`
- [x] `tools`
- [x] `reasoning_effort` (model-defined names and compatibility aliases)
- [x] `reasoning`
- [x] `effort` (model-defined names and compatibility aliases)
- [ ] `tool_choice`
- [ ] `logit_bias`
- [ ] `user`
- [ ] `n`

`reasoning_effort` and `reasoning.effort` control model thinking. Use `/api/show` to discover each model's supported values and default. When a model uses its metadata to resolve named levels, supported names are applied exactly and unsupported names resolve to the model default. Boolean-only models that support enabling thinking map recognized compatibility efforts to `true`, while `"none"` requests `false`. Models without metadata retain the existing compatibility aliases: `"minimal"` maps to `"low"`, and `"xhigh"` or `"ultra"` map to `"max"`. GPT-OSS keeps its existing OpenAI compatibility conversion: `"minimal"` uses `"low"`, `"xhigh"` or `"ultra"` use `"high"`, and `"none"` requests no thinking output.

`/v1/completions`

Supported features

- [x] Completions
- [x] Streaming
- [x] JSON mode
- [x] Reproducible outputs
- [ ] Logprobs

Supported request fields

- [x] `model`
- [x] `prompt`
- [x] `frequency_penalty`
- [x] `presence_penalty`
- [x] `seed`

- [x] `stop`
- [x] `stream`
- [x] `stream_options`
- [x] `include_usage`
- [x] `temperature`
- [x] `top_p`
- [x] `max_tokens`
- [x] `suffix`
- [ ] `best_of`
- [ ] `echo`
- [ ] `logit_bias`
- [ ] `user`
- [ ] `n`

Notes

- `prompt` currently only accepts a string

`/v1/models`

Notes

- `created` corresponds to when the model was last modified
- `owned_by` corresponds to the ollama username, defaulting to `"library"`

`/v1/models/{model}`

Notes

- `created` corresponds to when the model was last modified
- `owned_by` corresponds to the ollama username, defaulting to `"library"`

`/v1/embeddings`

Supported request fields

- [x] `model`
- [x] `input`
- [x] string
- [x] array of strings
- [ ] array of tokens
- [ ] array of token arrays
- [x] `encoding format`
- [x] `dimensions`
- [ ] `user`

`/v1/audio/speech`

> Note: Added in Ollama v0.40.8

Generates audio with speech models. The response streams while audio is generated.

Supported request fields

- [x] `model`
- [x] `input` (up to 4,096 characters)
- [x] `voice` (one of the voices `ollama show` lists, in any case and with or without accents, such as `Freya`; omit it for the model's `default` voice)
- [x] `response_format`
- [x] `wav` (default)
- [x] `pcm`
- [ ] `mp3`, `opus`, `aac`, `flac`
- [x] `speed` (`1` only)
- [x] `stream_format` (`audio` only)
- [ ] `instructions`

Notes

- Audio is 16-bit, mono, 24 kHz. `pcm` returns raw little-endian samples with no header.
- The default `response_format` is `wav`, not `mp3`.
- Unsupported values return a `400` error instead of being ignored.

`/v1/audio/transcriptions`

Transcribes speech with models that accept audio input.

Supported request fields

- [x] `file` (WAV at up to 384 kHz; up to 512 MB and 2 hours)
- [x] `model`
- [x] `language` (ISO 639-1 code such as `fr`, or an English language name; omit it to let the model detect the language)
- [x] `prompt` (used by models without speech-specific instructions)
- [x] `response_format`
- [x] `json` (default)
- [x] `text`
- [x] `verbose_json` (one segment per transcribed piece of audio; `language` is the output language for translations; for transcriptions it is set only when given in the request)
- [ ] `srt`, `vtt`
- [x] `temperature` (default `0`)
- [x] `stream` (server-sent `transcript.text.delta` and `transcript.text.done` events)
- [ ] `timestamp_granularities`

Notes

- Audio longer than the model takes at once (30 seconds for current models) is split at pauses and transcribed piece by piece. Streaming sends each piece as it finishes.
- Each model gets the transcription instructions its publisher documents.

`/v1/audio/translations`

> Note: Added in Ollama v0.40.12

Translates speech into text in another language.

Supported request fields

- [x] `file`
- [x] `model`
- [x] `to_language` (Ollama extension; ISO 639-1 code of the output language, default `en`)
- [x] `language` (Ollama extension; the spoken language, if known)
- [x] `prompt` (used by models without speech-specific instructions)
- [x] `response_format`, `temperature` and `stream`, as for transcriptions

Responses API

`POST /v1/responses`

> Note: Added in Ollama v0.13.3

Ollama supports the OpenAI Responses API. Only the non-stateful flavor is supported (i.e., there is no `previous_response_id` or `conversation` support).

Supported features

- [x] Streaming
- [x] Tools (function calling)
- [x] Reasoning summaries (for thinking models)
- [ ] Stateful requests

Supported request fields

- [x] `model`
- [x] `input`
- [x] `instructions`
- [x] `tools`
- [x] `stream`
- [x] `temperature`
- [x] `top_p`
- [x] `max_output_tokens`
- [x] `reasoning`
- [x] `effort` (model-defined names and compatibility aliases)
- [x] `think` (Ollama extension; boolean, model-defined string, or `null`)
- [ ] `previous_response_id` (stateful v1/responses not supported)
- [ ] `conversation` (stateful v1/responses not supported)
- [ ] `truncation`

Models

For local inference, download a model before using it:

```
ollama pull llama3.2
```

Direct cloud requests use the identifiers from `https://ollama.com/api/tags` and do not require a pull. Through a signed-in local server, select a cloud model such as `gemma4:cloud`.

Default model names on a local server

For tooling that relies on default OpenAI model names such as `gpt-3.5-turbo`, use `ollama cp` to copy an existing model name to a temporary name:

```
ollama cp llama3.2 gpt-3.5-turbo
```

Afterwards, this new model name can be specified the `model` field:

```
curl http://localhost:11434/v1/chat/completions \
    -H "Content-Type: application/json" \
    -d '{
        "model": "gpt-3.5-turbo",
        "messages": [
            {
                "role": "user",
                "content": "Hello!"
            }
        ]
    }'
```

Setting the local context size

The OpenAI API does not have a way of setting the context size for a model. If you need to change the context size, create a `Modelfile` which looks like:

```
FROM <some model>
PARAMETER num_ctx <context size>
```

Use the `ollama create mymodel` command to create a new model with the updated context size. Call the API with the updated model name:

```
curl http://localhost:11434/v1/chat/completions \
    -H "Content-Type: application/json" \
    -d '{
        "model": "mymodel",
        "messages": [
            {
                "role": "user",
                "content": "Hello!"
            }
        ]
    }'
```

API

Source: https://github.com/ollama/ollama/blob/main/docs/api.md (full endpoint reference)

> Note: Ollama's API docs are moving to https://docs.ollama.com/api

Endpoints

- Generate a completion
- Generate a chat completion
- Create a Model
- List Local Models
- Show Model Information
- Copy a Model
- Delete a Model
- Pull a Model
- Push a Model
- Generate Embeddings
- List Running Models
- Version

Conventions

Model names

Model names follow a `model:tag` format, where `model` can have an optional namespace such as `example/model`. Some examples are `orca-mini:3b-q8_0` and `llama3:70b`. The tag is optional and, if not provided, will default to `latest`. The tag is used to identify a specific version.

Durations

All durations are returned in nanoseconds.

Streaming responses

Certain endpoints stream responses as JSON objects. Streaming can be disabled by providing `{"stream": false}` for these endpoints.

Generate a completion

```
POST /api/generate
```

Generate a response for a given prompt with a provided model. This is a streaming endpoint, so there will be a series of responses. The final response object will include statistics and additional data from the request.

Parameters

- `model`: (required) the model name
- `prompt`: the prompt to generate a response for

- `suffix`: the text after the model response
- `images`: (optional) a list of base64-encoded images (for multimodal models such as `llava`)
- `think`: (for thinking models) should the model think before responding? Can be a boolean or a thinking level (`"low"`, `"medium"`, `"high"`, or `"max"`).

Advanced parameters (optional):

- `format`: the format to return a response in. Format can be `json` or a JSON schema
- `options`: additional model parameters listed in the documentation for the Modelfile such as `temperature`
- `system`: system message to (overrides what is defined in the `Modelfile`)
- `template`: the prompt template to use (overrides what is defined in the `Modelfile`)
- `stream`: if `false` the response will be returned as a single response object, rather than a stream of objects
- `raw`: if `true` no formatting will be applied to the prompt. You may choose to use the `raw` parameter if you are specifying a full templated prompt in your request to the API
- `keep_alive`: controls how long the model will stay loaded into memory following the request (default: `5m`)
- `context` (deprecated): the context parameter returned from a previous request to `/generate`, this can be used to keep a short conversational memory

Structured outputs

Structured outputs are supported by providing a JSON schema in the `format` parameter. The model will generate a response that matches the schema. See the structured outputs example below.

JSON mode

Enable JSON mode by setting the `format` parameter to `json`. This will structure the response as a valid JSON object. See the JSON mode example below.

> [!IMPORTANT] It's important to instruct the model to use JSON in the `prompt`.
> Otherwise, the model may generate large amounts whitespace.

Examples

Generate request (Streaming)

Request

```
curl http://localhost:11434/api/generate -d '{
  "model": "llama3.2",
  "prompt": "Why is the sky blue?"
}'
```

Response

A stream of JSON objects is returned:

```
{
  "model": "llama3.2",
  "created_at": "2023-08-04T08:52:19.385406455-07:00",
  "response": "The",
  "done": false
}
```

The final response in the stream also includes additional data about the generation:

- `total_duration`: time spent generating the response
- `load_duration`: time spent in nanoseconds loading the model
- `prompt_eval_count`: number of tokens in the prompt
- `prompt_eval_cached_count`: number of prompt tokens read from the cache
- `prompt_eval_duration`: time spent in nanoseconds evaluating uncached prompt tokens
- `eval_count`: number of tokens in the response
- `eval_duration`: time in nanoseconds spent generating the response
- `context`: an encoding of the conversation used in this response, this can be sent in the next request to keep a conversational memory
- `response`: empty if the response was streamed, if not streamed, this will contain the full response

To calculate how fast the response is generated in tokens per second (token/s), divide `eval_count` / `eval_duration` * `10^9`.

```
{
  "model": "llama3.2",
  "created_at": "2023-08-04T19:22:45.499127Z",
  "response": "",
  "done": true,
  "context": [1, 2, 3],
  "total_duration": 10706818083,
  "load_duration": 6338219291,
  "prompt_eval_count": 26,
  "prompt_eval_duration": 130079000,
  "eval_count": 259,
  "eval_duration": 4232710000
}
```

Request (No streaming)

Request

A response can be received in one reply when streaming is off.

```
curl http://localhost:11434/api/generate -d '{
  "model": "llama3.2",
  "prompt": "Why is the sky blue?",
  "stream": false
}'
```

Response

If `stream` is set to `false`, the response will be a single JSON object:

```
{
  "model": "llama3.2",
  "created_at": "2023-08-04T19:22:45.499127Z",
  "response": "The sky is blue because it is the color of the sky.",
  "done": true,
  "context": [1, 2, 3],
  "total_duration": 5043500667,
  "load_duration": 5025959,
  "prompt_eval_count": 26,
  "prompt_eval_duration": 325953000,
  "eval_count": 290,
  "eval_duration": 4709213000
}
```

Request (with suffix)

Request

```
curl http://localhost:11434/api/generate -d '{
  "model": "codellama:code",
  "prompt": "def compute_gcd(a, b):",
  "suffix": "    return result",
  "options": {
    "temperature": 0
  },
  "stream": false
}'
```

Response

```
{
  "model": "codellama:code",
  "created_at": "2024-07-22T20:47:51.147561Z",
  "response": "\n  if a == 0:\n    return b\n  else:\n    return compute_gcd(b % a, a)\n\ndef
compute_lcm(a, b):\n  result = (a * b) / compute_gcd(a, b)\n",
  "done": true,
  "done_reason": "stop",
  "context": [...],
  "total_duration": 1162761250,
  "load_duration": 6683708,
  "prompt_eval_count": 17,
  "prompt_eval_duration": 201222000,
  "eval_count": 63,
  "eval_duration": 953997000
}
```

Request (Structured outputs)

Request

```
curl -X POST http://localhost:11434/api/generate -H "Content-Type: application/json" -d '{
  "model": "llama3.1:8b",
  "prompt": "Ollama is 22 years old and is busy saving the world. Respond using JSON",
  "stream": false,
  "format": {
    "type": "object",
```

```
    "properties": {
      "age": {
        "type": "integer"
      },
      "available": {
        "type": "boolean"
      }
    },
    "required": [
      "age",
      "available"
    ]
  }
}'
```

Response

```
{
  "model": "llama3.1:8b",
  "created_at": "2024-12-06T00:48:09.983619Z",
  "response": "{\n  \"age\": 22,\n  \"available\": true\n}",
  "done": true,
  "done_reason": "stop",
  "context": [1, 2, 3],
  "total_duration": 1075509083,
  "load_duration": 567678166,
  "prompt_eval_count": 28,
  "prompt_eval_duration": 236000000,
  "eval_count": 16,
  "eval_duration": 269000000
}
```

Request (JSON mode)

> [!IMPORTANT] When `format` is set to `json`, the output will always be a well-formed JSON object. It's important to also instruct the model to respond in JSON.

Request

```
curl http://localhost:11434/api/generate -d '{
  "model": "llama3.2",
  "prompt": "What color is the sky at different times of the day? Respond using JSON",
  "format": "json",
  "stream": false
}'
```

Response

```
{
  "model": "llama3.2",
  "created_at": "2023-11-09T21:07:55.186497Z",
  "response": "{\n\"morning\": {\n\"color\": \"blue\"\n},\n\"noon\": {\n\"color\": \"blue-
gray\"\n},\n\"afternoon\": {\n\"color\": \"warm gray\"\n},\n\"evening\": {\n\"color\":
\"orange\"\n}\n}\n",
  "done": true,
  "context": [1, 2, 3],
```

```
  "total_duration": 4648158584,
  "load_duration": 4071084,
  "prompt_eval_count": 36,
  "prompt_eval_duration": 439038000,
  "eval_count": 180,
  "eval_duration": 4196918000
}
```

The value of `response` will be a string containing JSON similar to:

```
{
  "morning": {
    "color": "blue"
  },
  "noon": {
    "color": "blue-gray"
  },
  "afternoon": {
    "color": "warm gray"
  },
  "evening": {
    "color": "orange"
  }
}
```

Request (with images)

To submit images to multimodal models such as `llava` or `bakllava`, provide a list of base64-encoded `images`:

Request

```
curl http://localhost:11434/api/generate -d '{
  "model": "llava",
  "prompt":"What is in this picture?",
  "stream": false,
  "images":
["iVBORw0KGgoAAAANSUhEUgAAAG0AAABmCAYAAADBPx+VAAAACXBIWXMAAAsTAAALEwEAmpwYAAAAAXNSR0IArs4c6QA
AAARnQU1BAACxjwv8YQUAAA3VSURBVHgB7Z27r0zdG8fX743i1bi1ikMoFMQloXRpKFFIqI7LH4BEQ+NWIkjQuSWCRIEo
ULk0gsK1kCBI0IhrQVT7tz/7zZo888yz1r7MnDl7z5xvsjkzs2fP3uu71nNfa7lkAsm7d++/cqq1ars/yL4/v27S0ejqwv+cUOGEGGpKH
R37tzJCEpHV9tnT58+dXXCJDdECBE2Ojrqjh071hpNECjx4cMHVycM1Uhbv359B2F79+51586daxN/+pyRkRFXKyRDAqx
Ep4yMlDDzXG1NPnnyJKkThoK0VFd1ELZu3TrzXKxKfW7dMBQ6bcuWLW2v0VlHjx41z717927ba22U9APcw7Nnz1oGEPeL
3m3p2mTAYYnFmMOMXybPPXv2bNIPpFZr1NHn4HMw0KRBjg9NuRw95s8PEcz/6DZELQd/09C9QGq5RsmSRybqkwHGjh07O
sJSsYYm3ijPpyHzoiacg35MLdDSIS/O1yM778jOTwYUkKNHWUzUWaOsylE00MyI0fcnOwIdjvtNdW/HZwNLGg+sR1kMep
SNJXmIwxBZiG8tDTpEZzKg0GItNsosY8USkxDhD0Rinuiko2gfL/RbiD2LZAjU9zKQJj8RDR0vJBR1/Phx9+PHj9Z7REF
4nTZkxzX4LCXHrV271qXkBAPGfP/atWvu/PnzHe4C97F48eIsRLZ9+3a3f/9+87dwP1JxaF7/3r17ba+5l4EcaVo0lj3S
Bq5kGTJSQmLWMjgYNei2GPT1MuMqGTDEFHzeQSP2wi/jGnkmPJ/nhccs44jvDAxpVcxnq0F6eT8h4ni/iIWpR5lPyA6ET
kNXoSukvpJAD3AsXLiwpZs49+fPn5ke4j10TqYvegSfn0OnafC+Tv9ooA/JPkgQysqQNBzagXY55nO/oa1F7qvIPWkRL1
2WRpMWUvpVDYmxAPehxWSe8ZEXL20sadYIozfmNch4QJPAfeJgW3rNsnzphBKNJM2KKODo1rVOMRYik5ETy3ix4qWNI81
qAAirizgMIc+yhTytx0JWZuNI03qsrgWlGtwjoS9XwgUhWGyhUaRZZQNNIEwCiXD16tXcAHUs79co0vSD8rrJCIW98pzv
xpAWyyo3HYwqS0+H0BjStClcZJT5coMm6D2LOF8TolGJtK9fvyZpyiC5ePFi9nc/oJU4eiEP0jVoAnHa9wyJycITMP78+
eMeP37sXrx44d6+fdt6f82aNdkx1pg9e3Zb5W+RSRE+n+VjksQWifvVaTKFhn5O8my63K8Qabdv33b379/PiAP//vuvW7
BggZszZ072/+TJk91YgkafPn166zXB1rQHFvouAWHq9z3SEevSUerqCn2/dDCeta2jxYbr69evk4MHDyY7d+7MjhMnTiT
Pnz9Pfv/+nfQT2ggpO2dMF8cghuoM7Ygj5iWCqRlGFml0QC/ftGmTmzt3rmsaKDsgBSPh0/8yPeLLBihLkOKJc0jp8H8v
UzcxIA1k6QJ/c78tWEyj5P3o4u9+jywNPdJi5rAH9x0KHcl4Hg570eQp3+vHXGyrmEeigzQsQsjavXt38ujRo44LQuDDh
w+TW7duRS1HGgMxhNXHgflaNTOsHyKvHK5Ijo2jbFjJBQK9YwFd6RVMzfgRBmEfP37suBBm/p49e1qjEP2mwTViNRo0VJ
WH1deMXcNK08uUjVUu7s/zRaL+oLNxz1bpANco4npUgX4G2eFbpDFyQoQxojBCpEGSytmOH8qrH5Q9vuzD6ofQylkCUmh
8DBAr+q8JCyVNtWQIidKQE9wNtLSQnS4jDSsxNHogzFuQBw4cyM61UKVsjfr3ooBkPSqqQHesUPWVtzi9/vQi1T+rJj7W
iTz4Pt/l3LxUkr5P2VYZaZ4URpsE+st/dujQoaBBYokbrz/8TJNQYLSonrPS9kUaSkPeZyj1AWSj+d+VBoy1pIWVNed8P
0Ll/ee5HdGRhrHhR5GGN0r4LGZBaj8oFDJitBTJzIZgFcmU0Y8ytWMZMzJOaXUSrUs5RxKnrxmbb5YXO9VGUhtpXldhEU
ogFr3IzIsvlpmdosVcGVGXFWp2oU9kLFL3dEkSz6NHEY1sjSRdIuDFWEhd8KxFqsRi1uM/nz9/zpxnwlESONdg6dKlbsa
MGS4EHFHtjFIDHwKOo46l4TxSuxgDzi+rE2jg+BaFruOX4HXa0Nnf1lwAPufZeF8/r6zD97WK2qFnGjBxTw5qNGPxT+5T
/r7/7RawFC3j4vTp09koCxkeHjqbHJqArmH5UrFKKksnxrK7FuRIs8STfBZv+luugXZ2pR/pP9Ois4z+TiMzUUkUjD0iE
i1fzX8GmXyuxUBRcaUfykV0YZnlJGKQpOiGB76x5GeWkWWJc3mOrK6S7xdND+W5N6XyaRgtWJFe13GkaZnKOsYqGdOVVV
```

```
bGupsyA/l7emTLHi7vwTdirNEt0qxnzAvBFcnQF16xh/TMpUuXHDowhlA9vQVraQhkudRdzOnK+04ZSP3DUhVSP61YsaL
td/ks7ZgtPcXqPqEafHkdqa84X6aCeL7YWlv6edGFHb+ZFICPlljHhg0bKuk0CSvVznWsotRu433alNdFrqG45ejoaPCa
UkWERpLXjzFL2Rpllp7PJU2a/v7Ab8N05/9t27Z16KUqoFGsxnI9EosS2niSYg9SpU6B4JgTrvVW1flt1sT+0ADIJU2ma
XzcUTraGCRaL1Wp9rUMk16PMom8QhruxzvZIegJjFU7LLCePfS8uaQdPny4jTTL0dbee5mYokQsXTIWNY46kuMbnt8Kme
c+LGWtOVIl9cT1rCB0V8WqkjAsRwta93TbwNYoGKsUSChN44lgBNCoHLHzquYKrU6qZ8lolCIN0Rh6cP0Q3U6I6IXILYO
QI513hJaSKAorFpuHXJNfVlpRtmYBk1Su1obZr5dnKAO+L10Hrj3WZW+E3qh6IszE37F6EB+68mGpvKm4eb9bFrlzrok7
fvr0Kfv727dvWRmdVTJHw0qiiCUSZ6wCK+7XL/AcsgNyL74DQQ730sv78Su7+t/A36MdY0sW5o40ahslXr58aZ5HtZB8G
H64m9EmMZ7FpYw4T6QnrZfgenrhFxaSiSGXtPnz57e9TkNZLvTjeqhr734CNtrK41L40sUQckmj1lGKQ0rC37x544r8eN
XRpnVE3ZZY7zXo8NomiO0ZUCj2uHz58rbXoZ6gc0uA+F6ZeKS/jhRDUq8MKrTho9fEkihMmhxtBI1DxKFY9XLpVcSkfoi
8JGnToZO5sU5aiDQIW716ddt7ZLYtMQlhECdBGXZZMWldY5BHm5xgAroWj4C0hbYkSc/jBmggIrXJWlZM6pSETsEPGqZO
ndr2uuuR5rF169a2HoHPdurUKZM4CO1WTPqaDaAd+GFGKdIQkxAn9RuEWcTRyN2KSUgiSgF5aWzPTeA/lN5rZubMmR2bE
4SIC4nJoltgAV/dVefZm72AtctUCJU2CMJ327hxY9t7EHbkyJFseq+EJSY16RPo3Dkq1kkr7+q0bNmyDuLQcZBEPYmHVd
OBiJyIlrRDq41YPWfXOxUysi5fvtyaj+2BpcnsUV/oSoEMOk2CQGlr4ckhBwaetBhjCwH0ZHtJROPJkyc7UjcYLDjmrH7
ADTEBXFfOYmB0k9oYBOjJ8b4aOYSe7QkKcYhFlq3QYLQhSidNmtS2RATwy8YOM3EQJsUjKiaWZ+vZToUQgzhkHXudb/PW
5YMHD9yZM2faPsMwoc7RciYJXbGuBqJ1UIGKKLv915jsvgtJxCZDubdXr165mzdvtr1Hz5LONA8jrUwKPqsmVesKa49S3
Q4WxmRPUEYdTjgiUcfUwLx589ySJUva3oMkP6IYddq6HMS4o55xBJBUeRjzfa4Zdeg56QZ43LhxoyPo7Lf1kNt7oO8wWA
bNwaYjIv5lhyS7kRf96dvm5Jah8vfvX3flyhX35cuX6HfzFHOToS1H4BenCaHvO8pr8iDuwoUL7tevX+b5ZdbBair0xkF
IlFDlW4ZknEClsp/TzXyAKVOmmHWFVSbDNw1l1+4f90U6IY/q4V27dpnE9bJ+v87QEydjqx/UamVVPRG+mwkNTYN+9tjk
wzEx+atCm/X9WvWtDtAb68Wy9LXa1UmvCDDIpPkyOQ5ZwSzJ4jMrvFcr0rSjOUh+GcT4LSg5ugkW1Io0/SCDQBojh0hPl
aJdah+tkVYrnTZowP8iq1F1TgMBBauufyB33x1v+NWFYmT5KmppgHC+NkAgbmRkpD3yn9QIseXymoTQFGQmIOKTxiZIWp
vAatenVqRVXf2nTrAWMsPnKrMZHz6bJq5jvce6QK8J1cQNgKxlJapMPdZSR64/UivS9NztpkVEdKcrs5alhhWP9NeqlfW
opzhZScI6QxseegZRGeg5a8C3Re1Mfl1ScP36ddcUaMuv24iOJtz7sbUjTS4qBvKmstYJoUauiuD3k5qhyr7QdUHMeCgL
a1Ear9NquemdXgmum4fvJ6w1lqsuDhNrg1qSpleJK7K3TF0Q2jSd94uSZ60kK1e3qyVpQK6PVWXp2/FC3mp6jBhKKOiY2
h3gtUV64TWM6wDETRPLDfSakXmH3w8g9Jlug8ZtTt4kVF0kLUYYmCCtD/DrQ5YhMGbA9L3ucdjh0y8kOHW5gU/VEEmJTc
L4Pz/f7mgoAbYkAAAAAElFTkSuQmCC"]
}'
```

Response

```
{
  "model": "llava",
  "created_at": "2023-11-03T15:36:02.583064Z",
  "response": "A happy cartoon character, which is cute and cheerful.",
  "done": true,
  "context": [1, 2, 3],
  "total_duration": 2938432250,
  "load_duration": 2559292,
  "prompt_eval_count": 1,
  "prompt_eval_duration": 2195557000,
  "eval_count": 44,
  "eval_duration": 736432000
}
```

Request (Raw Mode)

In some cases, you may wish to bypass the templating system and provide a full prompt. In this case, you can use the `raw` parameter to disable templating. Also note that raw mode will not return a context.

Request

```
curl http://localhost:11434/api/generate -d '{
  "model": "mistral",
  "prompt": "[INST] why is the sky blue? [/INST]",
  "raw": true,
  "stream": false
}'
```

Request (Reproducible outputs)

For reproducible outputs, set `seed` to a number:

Request

```
curl http://localhost:11434/api/generate -d '{
  "model": "mistral",
  "prompt": "Why is the sky blue?",
  "options": {
    "seed": 123
  }
}'
```

Response

```
{
  "model": "mistral",
  "created_at": "2023-11-03T15:36:02.583064Z",
  "response": " The sky appears blue because of a phenomenon called Rayleigh scattering.",
  "done": true,
  "total_duration": 8493852375,
  "load_duration": 6589624375,
  "prompt_eval_count": 14,
  "prompt_eval_duration": 119039000,
  "eval_count": 110,
  "eval_duration": 1779061000
}
```

Generate request (With options)

If you want to set custom options for the model at runtime rather than in the Modelfile, you can do so with the `options` parameter. This example sets every available option, but you can set any of them individually and omit the ones you do not want to override.

`typical_p` is deprecated and may be removed in a future release.

Request

```
curl http://localhost:11434/api/generate -d '{
  "model": "llama3.2",
  "prompt": "Why is the sky blue?",
  "stream": false,
  "options": {
    "num_keep": 5,
    "seed": 42,
    "num_predict": 100,
    "draft_num_predict": 4,
    "top_k": 20,
    "top_p": 0.9,
    "min_p": 0.0,
    "repeat_last_n": 33,
    "temperature": 0.8,
    "repeat_penalty": 1.2,
    "presence_penalty": 1.5,
    "frequency_penalty": 1.0,
    "penalize_newline": true,
    "stop": ["\n", "user:"],
```

```
    "numa": false,
    "num_ctx": 1024,
    "num_batch": 2,
    "num_gpu": 1,
    "main_gpu": 0,
    "use_mmap": true,
    "num_thread": 8
  }
}'
```

Response

```
{
  "model": "llama3.2",
  "created_at": "2023-08-04T19:22:45.499127Z",
  "response": "The sky is blue because it is the color of the sky.",
  "done": true,
  "context": [1, 2, 3],
  "total_duration": 4935886791,
  "load_duration": 534986708,
  "prompt_eval_count": 26,
  "prompt_eval_duration": 107345000,
  "eval_count": 237,
  "eval_duration": 4289432000
}
```

Load a model

If an empty prompt is provided, the model will be loaded into memory.

Request

```
curl http://localhost:11434/api/generate -d '{
  "model": "llama3.2"
}'
```

Response

A single JSON object is returned:

```
{
  "model": "llama3.2",
  "created_at": "2023-12-18T19:52:07.071755Z",
  "response": "",
  "done": true
}
```

Unload a model

If an empty prompt is provided and the `keep_alive` parameter is set to `0`, a model will be unloaded from memory.

Request

```
curl http://localhost:11434/api/generate -d '{
  "model": "llama3.2",
  "keep_alive": 0
}'
```

Response

A single JSON object is returned:

```
{
  "model": "llama3.2",
  "created_at": "2024-09-12T03:54:03.516566Z",
  "response": "",
  "done": true,
  "done_reason": "unload"
}
```

Generate a chat completion

```
POST /api/chat
```

Generate the next message in a chat with a provided model. This is a streaming endpoint, so there will be a series of responses. Streaming can be disabled using `"stream": false`. The final response object will include statistics and additional data from the request.

Parameters

- `model`: (required) the model name
- `messages`: the messages of the chat, this can be used to keep a chat memory
- `tools`: list of tools in JSON for the model to use if supported
- `think`: (for thinking models) should the model think before responding? Can be a boolean or a thinking level (`"low"`, `"medium"`, `"high"`, or `"max"`).

The `message` object has the following fields:

- `role`: the role of the message, either `system`, `user`, `assistant`, or `tool`
- `content`: the content of the message
- `thinking`: (for thinking models) the model's thinking process
- `images` (optional): a list of images to include in the message (for multimodal models such as `llava`)
- `tool_calls` (optional): a list of tools in JSON that the model wants to use
- `tool_name` (optional): add the name of the tool that was executed to inform the model of the result

Advanced parameters (optional):

- `format`: the format to return a response in. Format can be `json` or a JSON schema.
- `options`: additional model parameters listed in the documentation for the Modelfile such as `temperature`

- `stream`: if `false` the response will be returned as a single response object, rather than a stream of objects
- `keep_alive`: controls how long the model will stay loaded into memory following the request (default: `5m`)

Tool calling

Tool calling is supported by providing a list of tools in the `tools` parameter. The model will generate a response that includes a list of tool calls. See the Chat request (Streaming with tools) example below.

Models can also explain the result of the tool call in the response. See the Chat request (With history, with tools) example below.

See models with tool calling capabilities.

Structured outputs

Structured outputs are supported by providing a JSON schema in the `format` parameter. The model will generate a response that matches the schema. See the Chat request (Structured outputs) example below.

Examples

Chat request (Streaming)

Request

Send a chat message with a streaming response.

```
curl http://localhost:11434/api/chat -d '{
  "model": "llama3.2",
  "messages": [
    {
      "role": "user",
      "content": "why is the sky blue?"
    }
  ]
}'
```

Response

A stream of JSON objects is returned:

```
{
  "model": "llama3.2",
  "created_at": "2023-08-04T08:52:19.385406455-07:00",
  "message": {
    "role": "assistant",
    "content": "The",
    "images": null
  },
```

```
  "done": false
}
```

Final response:

```
{
  "model": "llama3.2",
  "created_at": "2023-08-04T19:22:45.499127Z",
  "message": {
    "role": "assistant",
    "content": ""
  },
  "done": true,
  "total_duration": 4883583458,
  "load_duration": 1334875,
  "prompt_eval_count": 26,
  "prompt_eval_duration": 342546000,
  "eval_count": 282,
  "eval_duration": 4535599000
}
```

Chat request (Streaming with tools)

Request

```
curl http://localhost:11434/api/chat -d '{
  "model": "llama3.2",
  "messages": [
    {
      "role": "user",
      "content": "what is the weather in tokyo?"
    }
  ],
  "tools": [
    {
      "type": "function",
      "function": {
        "name": "get_weather",
        "description": "Get the weather in a given city",
        "parameters": {
          "type": "object",
          "properties": {
            "city": {
              "type": "string",
              "description": "The city to get the weather for"
            }
          },
          "required": ["city"]
        }
      }
    }
  ],
  "stream": true
}'
```

Response

A stream of JSON objects is returned:

```
{
  "model": "llama3.2",
  "created_at": "2025-07-07T20:22:19.184789Z",
  "message": {
    "role": "assistant",
    "content": "",
    "tool_calls": [
      {
        "function": {
          "name": "get_weather",
          "arguments": {
            "city": "Tokyo"
          }
        }
      }
    ]
  },
  "done": false
}
```


Final response:


```
{
  "model": "llama3.2",
  "created_at": "2025-07-07T20:22:19.19314Z",
  "message": {
    "role": "assistant",
    "content": ""
  },
  "done_reason": "stop",
  "done": true,
  "total_duration": 182242375,
  "load_duration": 41295167,
  "prompt_eval_count": 169,
  "prompt_eval_duration": 24573166,
  "eval_count": 15,
  "eval_duration": 115959084
}
```


Chat request (No streaming)

Request

```
curl http://localhost:11434/api/chat -d '{
  "model": "llama3.2",
  "messages": [
    {
      "role": "user",
      "content": "why is the sky blue?"
    }
  ],
  "stream": false
}'
```

Response


```
{
  "model": "llama3.2",
  "created_at": "2023-12-12T14:13:43.416799Z",
  "message": {
```

```
    "role": "assistant",
    "content": "Hello! How are you today?"
  },
  "done": true,
  "total_duration": 5191566416,
  "load_duration": 2154458,
  "prompt_eval_count": 26,
  "prompt_eval_duration": 383809000,
  "eval_count": 298,
  "eval_duration": 4799921000
}
```

Chat request (No streaming, with tools)

Request

```
curl http://localhost:11434/api/chat -d '{
  "model": "llama3.2",
  "messages": [
    {
      "role": "user",
      "content": "what is the weather in tokyo?"
    }
  ],
  "tools": [
    {
      "type": "function",
      "function": {
        "name": "get_weather",
        "description": "Get the weather in a given city",
        "parameters": {
          "type": "object",
          "properties": {
            "city": {
              "type": "string",
              "description": "The city to get the weather for"
            }
          },
          "required": ["city"]
        }
      }
    }
  ],
  "stream": false
}'
```

Response

```
{
  "model": "llama3.2",
  "created_at": "2025-07-07T20:32:53.844124Z",
  "message": {
    "role": "assistant",
    "content": "",
    "tool_calls": [
      {
        "function": {
          "name": "get_weather",
          "arguments": {
            "city": "Tokyo"
          }
        }
      }
```

```
    ]
  },
  "done_reason": "stop",
  "done": true,
  "total_duration": 3244883583,
  "load_duration": 2969184542,
  "prompt_eval_count": 169,
  "prompt_eval_duration": 141656333,
  "eval_count": 18,
  "eval_duration": 133293625
}
```

Chat request (Structured outputs)

Request

```
curl -X POST http://localhost:11434/api/chat -H "Content-Type: application/json" -d '{
  "model": "llama3.1",
  "messages": [{"role": "user", "content": "Ollama is 22 years old and busy saving the world.
Return a JSON object with the age and availability."}],
  "stream": false,
  "format": {
    "type": "object",
    "properties": {
      "age": {
        "type": "integer"
      },
      "available": {
        "type": "boolean"
      }
    },
    "required": [
      "age",
      "available"
    ]
  },
  "options": {
    "temperature": 0
  }
}'
```

Response

```
{
  "model": "llama3.1",
  "created_at": "2024-12-06T00:46:58.265747Z",
  "message": {
    "role": "assistant",
    "content": "{\"age\": 22, \"available\": false}"
  },
  "done_reason": "stop",
  "done": true,
  "total_duration": 2254970291,
  "load_duration": 574751416,
  "prompt_eval_count": 34,
  "prompt_eval_duration": 1502000000,
  "eval_count": 12,
  "eval_duration": 175000000
}
```

Chat request (With History)

Send a chat message with a conversation history. You can use this same approach to start the conversation using multi-shot or chain-of-thought prompting.

Request

```
curl http://localhost:11434/api/chat -d '{
  "model": "llama3.2",
  "messages": [
    {
      "role": "user",
      "content": "why is the sky blue?"
    },
    {
      "role": "assistant",
      "content": "due to rayleigh scattering."
    },
    {
      "role": "user",
      "content": "how is that different than mie scattering?"
    }
  ]
}'
```

Response

A stream of JSON objects is returned:

```
{
  "model": "llama3.2",
  "created_at": "2023-08-04T08:52:19.385406455-07:00",
  "message": {
    "role": "assistant",
    "content": "The"
  },
  "done": false
}
```

Final response:

```
{
  "model": "llama3.2",
  "created_at": "2023-08-04T19:22:45.499127Z",
  "done": true,
  "total_duration": 8113331500,
  "load_duration": 6396458,
  "prompt_eval_count": 61,
  "prompt_eval_duration": 398801000,
  "eval_count": 468,
  "eval_duration": 7701267000
}
```

Chat request (With history, with tools)

Request

```
curl http://localhost:11434/api/chat -d '{
  "model": "llama3.2",
  "messages": [
    {
      "role": "user",
      "content": "what is the weather in Toronto?"
    },
    // the message from the model appended to history
    {
      "role": "assistant",
      "content": "",
      "tool_calls": [
        {
          "function": {
            "name": "get_weather",
            "arguments": {
              "city": "Toronto"
            }
          }
        }
      ]
    },
    // the tool call result appended to history
    {
      "role": "tool",
      "content": "11 degrees celsius",
      "tool_name": "get_weather"
    }
  ],
  "stream": false,
  "tools": [
    {
      "type": "function",
      "function": {
        "name": "get_weather",
        "description": "Get the weather in a given city",
        "parameters": {
          "type": "object",
          "properties": {
            "city": {
              "type": "string",
              "description": "The city to get the weather for"
            }
          },
          "required": ["city"]
        }
      }
    }
  ]
}'
```

Response

```
{
  "model": "llama3.2",
  "created_at": "2025-07-07T20:43:37.688511Z",
  "message": {
    "role": "assistant",
    "content": "The current temperature in Toronto is 11°C."
  },
  "done_reason": "stop",
  "done": true,
  "total_duration": 890771750,
  "load_duration": 707634750,
  "prompt_eval_count": 94,
  "prompt_eval_duration": 91703208,
  "eval_count": 11,
```

```
  "eval_duration": 90282125
}
```

Chat request (with images)

Request

Send a chat message with images. The images should be provided as an array, with the individual images encoded in Base64.

```
curl http://localhost:11434/api/chat -d '{
  "model": "llava",
  "messages": [
    {
      "role": "user",
      "content": "what is in this image?",
      "images":
["iVBORw0KGgoAAAANSUhEUgAAAG0AAABmCAYAAADBPx+VAAAACXBIWXMAAAsTAAALEwEAmpwYAAAAAXNSR0IArs4c6QA
AAARnQU1BAACxjwv8YQUAAA3VSURBVHgB7Z27r0zdG8fX743i1bi1ikMoFMQloXRpKFFIqI7LH4BEQ+NWIkjQuSWCRIEo
ULk0gsK1kCBI0IhrQVT7tz/7zZo888yz1r7MnDl7z5xvsjkzs2fP3uu71nNfa7lkAsm7d++Sffv2JbJbNmzUqcc8m0adOSz
Zs3Z+/XES4ZckAWJEGWPiCxjsQNLWmQsWjRIpMseaxcuTKpG/7HP27I8P79e7dq1ars/yL4/v27S0ejqwv+cUOGEGGpKH
R37tzJCEpHV9tnT58+dXXCJDdECBE2Ojrqjh071hpNECjx4cMHVycM1Uhbv359B2F79+51586daxN/+pyRkRFXKyRDAqx
Ep4yMlDDzXG1NPnnyJKkThoK0VFd1ELZu3TrzXKxKfW7dMBQ6bcuWLW2v0VlHjx41z717927ba22U9APcw7Nnz1oGEPeL
3m3p2mTAYYnFmMOMXybPPXv2bNIPpFZr1NHn4HMw0KRBjg9NuRw95s8PEcz/6DZELQd/09C9QGq5RsmSRybqkwHGjh07O
sJSsYYm3ijPpyHzoiacg35MLdDSIS/O1yM778jOTwYUkKNHWUzUWaOsylE00MyI0fcnOwIdjvtNdW/HZwNLGg+sR1kMep
SNJXmIwxBZiG8tDTpEZzKg0GItNsosY8USkxDhD0Rinuiko2gfL/RbiD2LZAjU9zKQJj8RDR0vJBR1/Phx9+PHj9Z7REF
4nTZkxzX4LCXHrV271qXkBAPGfP/atWvu/PnzHe4C97F48eIsRLZ9+3a3f/9+87dwP1JxaF7/3r17ba+5l4EcaVo0lj3S
Bq5kGTJSQmLWMjgYNei2GPT1MuMqGTDEFHzeQSP2wi/jGnkmPJ/nhccs44jvDAxpVcxnq0F6eT8h4ni/iIWpR5lPyA6ET
kNXoSukvpJAD3AsXLiwpZs49+fPn5ke4j10TqYvegSfn0OnafC+Tv9ooA/JPkgQysqQNBzagXY55nO/oa1F7qvIPWkRL1
2WRpMWUvpVDYmxAPehxWSe8ZEXL20sadYIozfmNch4QJPAfeJgW3rNsnzphBKNJM2KKODo1rVOMRYik5ETy3ix4qWNI81
qAAirizgMIc+yhTytx0JWZuNI03qsrgWlGtwjoS9XwgUhWGyhUaRZZQNNIEwCiXD16tXcAHUs79co0vSD8rrJCIW98pzv
xpAWyyo3HYwqS0+H0BjStClcZJT5coMm6D2LOF8TolGJtK9fvyZpyiC5ePFi9nc/oJU4eiEP0jVoAnHa9wyJycITMP78+
eMeP37sXrx44d6+fdt6f82aNdkx1pg9e3Zb5W+RSRE+n+VjksQWifvVaTKFhn5O8my63K8Qabdv33b379/PiAP//vuvW7
BggZszZ072/+TJk91YgkafPn166zXB1rQHFvouAWHq9z3SEevSUerqCn2/dDCeta2jxYbr69evk4MHDyY7d+7MjhMnTiT
Pnz9Pfv/+nfQT2ggpO2dMF8cghuoM7Ygj5iWCqRlGFml0QC/ftGmTmzt3rmsaKDsgBSPh0/8yPeLLBihLkOKJc0jp8H8v
UzcxIA1k6QJ/c78tWEyj5P3o4u9+jywNPdJi5rAH9x0KHcl4Hg570eQp3+vHXGyrmEeigzQsQsjavXt38ujRo44LQuDDh
w+TW7duRS1HGgMxhNXHgflaNTOsHyKvHK5Ijo2jbFjJBQK9YwFd6RVMzfgRBmEfP37suBBm/p49e1qjEP2mwTViNRo0VJ
WH1deMXcNK08uUjVUu7s/zRaL+oLNxz1bpANco4npUgX4G2eFbpDFyQoQxojBCpEGSytmOH8qrH5Q9vuzD6ofQylkCUmh
8DBAr+q8JCyVNtWQIidKQE9wNtLSQnS4jDSsxNHogzFuQBw4cyM61UKVsjfr3ooBkPSqqQHesUPWVtzi9/vQi1T+rJj7W
iTz4Pt/l3LxUkr5P2VYZaZ4URpsE+st/dujQoaBBYokbrz/8TJNQYLSonrPS9kUaSkPeZyj1AWSj+d+VBoy1pIWVNed8P
0Ll/ee5HdGRhrHhR5GGN0r4LGZBaj8oFDJitBTJzIZgFcmU0Y8ytWMZMzJOaXUSrUs5RxKnrxmbb5YXO9VGUhtpXldhEU
ogFr3IzIsvlpmdosVcGVGXFWp2oU9kLFL3dEkSz6NHEY1sjSRdIuDFWEhd8KxFqsRi1uM/nz9/zpxnwlESONdg6dKlbsa
MGS4EHFHtjFIDHwKOo46l4TxSuxgDzi+rE2jg+BaFruOX4HXa0Nnf1lwAPufZeF8/r6zD97WK2qFnGjBxTw5qNGPxT+5T
/r7/7RawFC3j4vTp09koCxkeHjqbHJqArmH5UrFKKksnxrK7FuRIs8STfBZv+luugXZ2pR/pP9Ois4z+TiMzUUkUjD0iE
i1fzX8GmXyuxUBRcaUfykV0YZnlJGKQpOiGB76x5GeWkWWJc3mOrK6S7xdND+W5N6XyaRgtWJFe13GkaZnKOsYqGdOVVV
bGupsyA/l7emTLHi7vwTdirNEt0qxnzAvBFcnQF16xh/TMpUuXHDowhlA9vQVraQhkudRdzOnK+04ZSP3DUhVSP61YsaL
td/ks7ZgtPcXqPqEafHkdqa84X6aCeL7YWlv6edGFHb+ZFICPlljHhg0bKuk0CSvVznWsotRu433alNdFrqG45ejoaPCa
UkWERpLXjzFL2Rpllp7PJU2a/v7Ab8N05/9t27Z16KUqoFGsxnI9EosS2niSYg9SpU6B4JgTrvVW1flt1sT+0ADIJU2ma
XzcUTraGCRaL1Wp9rUMk16PMom8QhruxzvZIegJjFU7LLCePfS8uaQdPny4jTTL0dbee5mYokQsXTIWNY46kuMbnt8Kme
c+LGWtOVIl9cT1rCB0V8WqkjAsRwta93TbwNYoGKsUSChN44lgBNCoHLHzquYKrU6qZ8lolCIN0Rh6cP0Q3U6I6IXILYO
QI513hJaSKAorFpuHXJNfVlpRtmYBk1Su1obZr5dnKAO+L10Hrj3WZW+E3qh6IszE37F6EB+68mGpvKm4eb9bFrlzrok7
fvr0Kfv727dvWRmdVTJHw0qiiCUSZ6wCK+7XL/AcsgNyL74DQQ730sv78Su7+t/A36MdY0sW5o40ahslXr58aZ5HtZB8G
H64m9EmMZ7FpYw4T6QnrZfgenrhFxaSiSGXtPnz57e9TkNZLvTjeqhr734CNtrK41L40sUQckmj1lGKQ0rC37x544r8eN
XRpnVE3ZZY7zXo8NomiO0ZUCj2uHz58rbXoZ6gc0uA+F6ZeKS/jhRDUq8MKrTho9fEkihMmhxtBI1DxKFY9XLpVcSkfoi
8JGnToZO5sU5aiDQIW716ddt7ZLYtMQlhECdBGXZZMWldY5BHm5xgAroWj4C0hbYkSc/jBmggIrXJWlZM6pSETsEPGqZO
ndr2uuuR5rF169a2HoHPdurUKZM4CO1WTPqaDaAd+GFGKdIQkxAn9RuEWcTRyN2KSUgiSgF5aWzPTeA/lN5rZubMmR2bE
4SIC4nJoltgAV/dVefZm72AtctUCJU2CMJ327hxY9t7EHbkyJFseq+EJSY16RPo3Dkq1kkr7+q0bNmyDuLQcZBEPYmHVd
OBiJyIlrRDq41YPWfXOxUysi5fvtyaj+2BpcnsUV/oSoEMOk2CQGlr4ckhBwaetBhjCwH0ZHtJROPJkyc7UjcYLDjmrH7
ADTEBXFfOYmB0k9oYBOjJ8b4aOYSe7QkKcYhFlq3QYLQhSidNmtS2RATwy8YOM3EQJsUjKiaWZ+vZToUQgzhkHXudb/PW
5YMHD9yZM2faPsMwoc7RciYJXbGuBqJ1UIGKKLv915jsvgtJxCZDubdXr165mzdvtr1Hz5LONA8jrUwKPqsmVesKa49S3
Q4WxmRPUEYdTjgiUcfUwLx589ySJUva3oMkP6IYddq6HMS4o55xBJBUeRjzfa4Zdeg56QZ43LhxoyPo7Lf1kNt7oO8wWA
bNwaYjIv5lhyS7kRf96dvm5Jah8vfvX3flyhX35cuX6HfzFHOToS1H4BenCaHvO8pr8iDuwoUL7tevX+b5ZdbBair0xkF
IlFDlW4ZknEClsp/TzXyAKVOmmHWFVSbDNw1l1+4f90U6IY/q4V27dpnE9bJ+v87QEydjqx/UamVVPRG+mwkNTYN+9tjk
wzEx+atCm/X9WvWtDtAb68Wy9LXa1UmvCDDIpPkyOQ5ZwSzJ4jMrvFcr0rSjOUh+GcT4LSg5ugkW1Io0/SCDQBojh0hPl
aJdah+tkVYrnTZowP8iq1F1TgMBBauufyB33x1v+NWFYmT5KmppgHC+NkAgbmRkpD3yn9QIseXymoTQFGQmIOKTxiZIWp
vAatenVqRVXf2nTrAWMsPnKrMZHz6bJq5jvce6QK8J1cQNgKxlJapMPdZSR64/UivS9NztpkVEdKcrs5alhhWP9NeqlfW
opzhZScI6QxseegZRGeg5a8C3Re1Mfl1ScP36ddcUaMuv24iOJtz7sbUjTS4qBvKmstYJoUauiuD3k5qhyr7QdUHMeCgL
```

```
a1Ear9NquemdXgmum4fvJ6w1lqsuDhNrg1qSpleJK7K3TF0Q2jSd94uSZ60kK1e3qyVpQK6PVWXp2/FC3mp6jBhKKOiY2
h3gtUV64TWM6wDETRPLDfSakXmH3w8g9Jlug8ZtTt4kVF0kLUYYmCCtD/DrQ5YhMGbA9L3ucdjh0y8kOHW5gU/VEEmJTc
L4Pz/f7mgoAbYkAAAAAElFTkSuQmCC"]
    }
  ]
}'
```

Response

```
{
  "model": "llava",
  "created_at": "2023-12-13T22:42:50.203334Z",
  "message": {
    "role": "assistant",
    "content": " The image features a cute, little pig with an angry facial expression. It's
wearing a heart on its shirt and is waving in the air. This scene appears to be part of a
drawing or sketching project.",
    "images": null
  },
  "done": true,
  "total_duration": 1668506709,
  "load_duration": 1986209,
  "prompt_eval_count": 26,
  "prompt_eval_duration": 359682000,
  "eval_count": 83,
  "eval_duration": 1303285000
}
```

Chat request (Reproducible outputs)

Request

```
curl http://localhost:11434/api/chat -d '{
  "model": "llama3.2",
  "messages": [
    {
      "role": "user",
      "content": "Hello!"
    }
  ],
  "options": {
    "seed": 101,
    "temperature": 0
  }
}'
```

Response

```
{
  "model": "llama3.2",
  "created_at": "2023-12-12T14:13:43.416799Z",
  "message": {
    "role": "assistant",
    "content": "Hello! How are you today?"
  },
  "done": true,
  "total_duration": 5191566416,
  "load_duration": 2154458,
```

```
  "prompt_eval_count": 26,
  "prompt_eval_duration": 383809000,
  "eval_count": 298,
  "eval_duration": 4799921000
}
```

Chat request (with tools)

Request

```
curl http://localhost:11434/api/chat -d '{
  "model": "llama3.2",
  "messages": [
    {
      "role": "user",
      "content": "What is the weather today in Paris?"
    }
  ],
  "stream": false,
  "tools": [
    {
      "type": "function",
      "function": {
        "name": "get_current_weather",
        "description": "Get the current weather for a location",
        "parameters": {
          "type": "object",
          "properties": {
            "location": {
              "type": "string",
              "description": "The location to get the weather for, e.g. San Francisco, CA"
            },
            "format": {
              "type": "string",
              "description": "The format to return the weather in, e.g. 'celsius' or
'fahrenheit'",
              "enum": ["celsius", "fahrenheit"]
            }
          },
          "required": ["location", "format"]
        }
      }
    }
  ]
}'
```

Response

```
{
  "model": "llama3.2",
  "created_at": "2024-07-22T20:33:28.123648Z",
  "message": {
    "role": "assistant",
    "content": "",
    "tool_calls": [
      {
        "function": {
          "name": "get_current_weather",
          "arguments": {
            "format": "celsius",
            "location": "Paris, FR"
          }
        }
```

```
      }
    ]
  },
  "done_reason": "stop",
  "done": true,
  "total_duration": 885095291,
  "load_duration": 3753500,
  "prompt_eval_count": 122,
  "prompt_eval_duration": 328493000,
  "eval_count": 33,
  "eval_duration": 552222000
}
```

Load a model

If the messages array is empty, the model will be loaded into memory.

Request

```
curl http://localhost:11434/api/chat -d '{
  "model": "llama3.2",
  "messages": []
}'
```

Response

```
{
  "model": "llama3.2",
  "created_at": "2024-09-12T21:17:29.110811Z",
  "message": {
    "role": "assistant",
    "content": ""
  },
  "done_reason": "load",
  "done": true
}
```

Unload a model

If the messages array is empty and the `keep_alive` parameter is set to `0`, a model will be unloaded from memory.

Request

```
curl http://localhost:11434/api/chat -d '{
  "model": "llama3.2",
  "messages": [],
  "keep_alive": 0
}'
```

Response

A single JSON object is returned:

```
{
  "model": "llama3.2",
  "created_at": "2024-09-12T21:33:17.547535Z",
  "message": {
    "role": "assistant",
    "content": ""
  },
  "done_reason": "unload",
  "done": true
}
```

Create a Model

```
POST /api/create
```

Create a model from:

- another model;
- a safetensors directory; or
- one or more GGUF files.

Use GGUF community tools to prepare and quantize GGUF files before importing.

If you are creating a model from a safetensors directory or from GGUF files, you must push a blob for each file and then use its file name and SHA256 digest in the `files` field.

Parameters

- `model`: name of the model to create
- `from`: (optional) name of an existing model to create the new model from
- `files`: (optional) a dictionary of file names to SHA256 digests of blobs to create the model from
- `template`: (optional) the prompt template for the model
- `renderer`: (optional) the name of the renderer for the model
- `parser`: (optional) the name of the parser for the model
- `license`: (optional) a string or list of strings containing the license or licenses for the model
- `system`: (optional) a string containing the system prompt for the model
- `parameters`: (optional) a dictionary of parameters for the model (see Modelfile for a list of parameters)
- `messages`: (optional) a list of message objects used to create a conversation
- `capabilities`: (optional) capabilities to add without removing inherited or inferred capabilities, such as `["decision"]` for compatible decision models
- `stream`: (optional) if `false` the response will be returned as a single response object, rather than a stream of objects
- `quantize`: (optional) quantize safetensors model weights for MLX during import

Quantization types

Type	Recommended
nvfp4	*
mxfp8	*

Type	Recommended
mxfp4	
int4	
int8	

Examples

Create a new model

Create a new model from an existing model.

Request

```
curl http://localhost:11434/api/create -d '{
  "model": "mario",
  "from": "llama3.2",
  "system": "You are Mario from Super Mario Bros."
}'
```

Response

A stream of JSON objects is returned:

```
{"status":"reading model metadata"}
{"status":"creating system layer"}
{"status":"using already created layer
sha256:22f7f8ef5f4c791c1b03d7eb414399294764d7cc82c7e94aa81a1feb80a983a2"}
{"status":"using already created layer
sha256:8c17c2ebb0ea011be9981cc3922db8ca8fa61e828c5d3f44cb6ae342bf80460b"}
{"status":"using already created layer
sha256:7c23fb36d80141c4ab8cdbb61ee4790102ebd2bf7aeff414453177d4f2110e5d"}
{"status":"using already created layer
sha256:2e0493f67d0c8c9c68a8aeacdf6a38a2151cb3c4c1d42accf296e19810527988"}
{"status":"using already created layer
sha256:2759286baa875dc22de5394b4a925701b1896a7e3f8e53275c36f75a877a82c9"}
{"status":"writing layer
sha256:df30045fe90f0d750db82a058109cecd6d4de9c90a3d75b19c09e5f64580bb42"}
{"status":"writing layer
sha256:f18a68eb09bf925bb1b669490407c1b1251c5db98dc4d3d81f3088498ea55690"}
{"status":"writing manifest"}
{"status":"success"}
```

Create a model from GGUF

Create a model from GGUF files. The `files` parameter should contain the file name and SHA256 digest of each GGUF file. For a split GGUF model, include each shard under its original split filename. Use /api/blobs/:digest to push each file before calling this API.

Request

```
curl http://localhost:11434/api/create -d '{
  "model": "my-gguf-model",
  "files": {
    "test.gguf": "sha256:432f310a77f4650a88d0fd59ecdd7cebed8d684bafea53cbff0473542964f0c3"
  }
}'
```

Response

A stream of JSON objects is returned:

```
{"status":"parsing GGUF"}
{"status":"using existing layer
sha256:432f310a77f4650a88d0fd59ecdd7cebed8d684bafea53cbff0473542964f0c3"}
{"status":"writing manifest"}
{"status":"success"}
```

Create a model from a Safetensors directory

The `files` parameter should include the file name and SHA256 digest of each file in the Safetensors model. Use /api/blobs/:digest to push each file before calling this API. Files remain in the cache until the Ollama server is restarted.

Request

```
curl http://localhost:11434/api/create -d '{
  "model": "fred",
  "files": {
    "config.json": "sha256:dd3443e529fb2290423a0c65c2d633e67b419d273f170259e27297219828e389",
    "generation_config.json":
"sha256:88effbb63300dbbc7390143fbbdd9d9fa50587b37e8bfd16c8c90d4970a74a36",
    "special_tokens_map.json":
"sha256:b7455f0e8f00539108837bfa586c4fbf424e31f8717819a6798be74bef813d05",
    "tokenizer.json":
"sha256:bbc1904d35169c542dffbe1f7589a5994ec7426d9e5b609d07bab876f32e97ab",
    "tokenizer_config.json":
"sha256:24e8a6dc2547164b7002e3125f10b415105644fcf02bf9ad8b674c87b1eaaed6",
    "model.safetensors":
"sha256:1ff795ff6a07e6a68085d206fb84417da2f083f68391c2843cd2b8ac6df8538f"
  }
}'
```

Response

A stream of JSON objects is returned:

```
{"status":"importing fred (2 tensors)"}
{"status":"writing manifest for fred"}
{"status":"success"}
```

Check if a Blob Exists

```
HEAD /api/blobs/:digest
```

Ensures that the file blob (Binary Large Object) used with create a model exists on the server. This checks your Ollama server and not ollama.com.

Query Parameters

- `digest`: the SHA256 digest of the blob

Examples

Request

```
curl -I
http://localhost:11434/api/blobs/sha256:29fdb92e57cf0827ded04ae6461b5931d01fa595843f55d36f5b2
75a52087dd2
```

Response

Return 200 OK if the blob exists, 404 Not Found if it does not.

Push a Blob

```
POST /api/blobs/:digest
```

Push a file to the Ollama server to create a "blob" (Binary Large Object).

Query Parameters

- `digest`: the expected SHA256 digest of the file

Examples

Request

```
curl -T model.gguf -X POST
http://localhost:11434/api/blobs/sha256:29fdb92e57cf0827ded04ae6461b5931d01fa595843f55d36f5b2
75a52087dd2
```

Response

Return 201 Created if the blob was successfully created, 400 Bad Request if the digest used is not expected.

List Local Models

```
GET /api/tags
```

List models that are available locally.

Examples

Request

```
curl http://localhost:11434/api/tags
```

Response

A single JSON object will be returned.


```
{
  "models": [
    {
      "name": "deepseek-r1:latest",
      "model": "deepseek-r1:latest",
      "modified_at": "2025-05-10T08:06:48.639712648-07:00",
      "size": 4683075271,
      "digest": "0a8c266910232fd3291e71e5ba1e058cc5af9d411192cf88b6d30e92b6e73163",
      "details": {
        "parent_model": "",
        "format": "gguf",
        "family": "qwen2",
        "families": ["qwen2"],
        "parameter_size": "7.6B",
        "quantization_level": "Q4_K_M"
      }
    },
    {
      "name": "llama3.2:latest",
      "model": "llama3.2:latest",
      "modified_at": "2025-05-04T17:37:44.706015396-07:00",
      "size": 2019393189,
      "digest": "a80c4f17acd55265feec403c7aef86be0c25983ab279d83f3bcd3abbcb5b8b72",
      "details": {
        "parent_model": "",
        "format": "gguf",
        "family": "llama",
        "families": ["llama"],
        "parameter_size": "3.2B",
        "quantization_level": "Q4_K_M"
      }
    }
  ]
}
```


Show Model Information

```
POST /api/show
```

Show information about a model including details, modelfile, template, parameters, license, system prompt.

Parameters

- `model`: name of the model to show
- `verbose`: (optional) if set to `true`, returns full data for verbose response fields

Examples

Request

```
curl http://localhost:11434/api/show -d '{
  "model": "llava"
}'
```

Response

```
{
  modelfile: '# Modelfile generated by "ollama show"\n# To build a new Modelfile based on
this one, replace the FROM line with:\n# FROM llava:latest\n\nFROM
/Users/matt/.ollama/models/blobs/sha256:200765e1283640ffbd013184bf496e261032fa75b99498a9613be
4e94d63ad52\nTEMPLATE """{{ .System }}\nUSER: {{ .Prompt }}\nASSISTANT: """\nPARAMETER
num_ctx 4096\nPARAMETER stop "\u003c/s\u003e"\nPARAMETER stop "USER:"\nPARAMETER stop
"ASSISTANT:"',
  parameters: 'num_keep                       24\nstop                           "
<|start_header_id|>"\nstop                             "<|end_header_id|>"\nstop
"<|eot_id|>"',
  template: "{{ if .System }}<|start_header_id|>system<|end_header_id|>\n\n{{ .System }}
<|eot_id|>{{ end }}{{ if .Prompt }}<|start_header_id|>user<|end_header_id|>\n\n{{ .Prompt }}
<|eot_id|>{{ end }}<|start_header_id|>assistant<|end_header_id|>\n\n{{ .Response }}
<|eot_id|>",
  details: {
    parent_model: "",
    format: "gguf",
    family: "llama",
    families: ["llama"],
    parameter_size: "8.0B",
    quantization_level: "Q4_0",
  },
  model_info: {
    "general.architecture": "llama",
    "general.file_type": 2,
    "general.parameter_count": 8030261248,
    "general.quantization_version": 2,
    "llama.attention.head_count": 32,
    "llama.attention.head_count_kv": 8,
    "llama.attention.layer_norm_rms_epsilon": 0.00001,
    "llama.block_count": 32,
    "llama.context_length": 8192,
    "llama.embedding_length": 4096,
    "llama.feed_forward_length": 14336,
    "llama.rope.dimension_count": 128,
    "llama.rope.freq_base": 500000,
    "llama.vocab_size": 128256,
    "tokenizer.ggml.bos_token_id": 128000,
    "tokenizer.ggml.eos_token_id": 128009,
    "tokenizer.ggml.merges": [], // populates if `verbose=true`
    "tokenizer.ggml.model": "gpt2",
    "tokenizer.ggml.pre": "llama-bpe",
    "tokenizer.ggml.token_type": [], // populates if `verbose=true`
    "tokenizer.ggml.tokens": [], // populates if `verbose=true`
  },
  capabilities: ["completion", "vision"],
}
```

Copy a Model

```
POST /api/copy
```

Copy a model. Creates a model with another name from an existing model.

Examples

Request

```
curl http://localhost:11434/api/copy -d '{
  "source": "llama3.2",
  "destination": "llama3-backup"
}'
```

Response

Returns a 200 OK if successful, or a 404 Not Found if the source model doesn't exist.

```
{
  "status": "success"
}
```

Delete a Model

```
DELETE /api/delete
```

Delete a model and its data.

Parameters

- `model`: model name to delete

Examples

Request

```
curl -X DELETE http://localhost:11434/api/delete -d '{
  "model": "llama3:13b"
}'
```

Response

Returns a 200 OK if successful, 404 Not Found if the model to be deleted doesn't exist.

Pull a Model

```
POST /api/pull
```

Download a model from the ollama library. Cancelled pulls are resumed from where they left off, and multiple calls will share the same download progress.

Parameters

- `model`: name of the model to pull

- `insecure`: (optional) allow insecure connections to the library. Only use this if you are pulling from your own library during development.
- `stream`: (optional) if `false` the response will be returned as a single response object, rather than a stream of objects

Examples

Request

```
curl http://localhost:11434/api/pull -d '{
  "model": "llama3.2"
}'
```

Response

If `stream` is not specified, or set to `true`, a stream of JSON objects is returned:

The first object is the manifest:

```
{
  "status": "pulling manifest"
}
```

Then there is a series of downloading responses. Until any of the download is completed, the `completed` key may not be included. The number of files to be downloaded depends on the number of layers specified in the manifest.

```
{
  "status": "pulling digestname",
  "digest": "digestname",
  "total": 2142590208,
  "completed": 241970
}
```

After all the files are downloaded, the final responses are:

```
{
    "status": "verifying sha256 digest"
}
{
    "status": "writing manifest"
}
{
    "status": "removing any unused layers"
}
{
    "status": "success"
}
```

if `stream` is set to false, then the response is a single JSON object:

```
{
  "status": "success"
}
```

Push a Model

```
POST /api/push
```

Upload a model to a model library. Requires registering for ollama.ai and adding a public key first.

Parameters

- `model`: name of the model to push in the form of `<namespace>/<model>:<tag>`
- `insecure`: (optional) allow insecure connections to the library. Only use this if you are pushing to your library during development.
- `stream`: (optional) if `false` the response will be returned as a single response object, rather than a stream of objects

Examples

Request

```
curl http://localhost:11434/api/push -d '{
  "model": "mattw/pygmalion:latest"
}'
```

Response

If `stream` is not specified, or set to `true`, a stream of JSON objects is returned:

```
{ "status": "retrieving manifest" }
```

and then:

```
{
  "status": "starting upload",
  "digest": "sha256:bc07c81de745696fdf5afca05e065818a8149fb0c77266fb584d9b2cba3711ab",
  "total": 1928429856
}
```

Then there is a series of uploading responses:

```
{
  "status": "starting upload",
  "digest": "sha256:bc07c81de745696fdf5afca05e065818a8149fb0c77266fb584d9b2cba3711ab",
  "total": 1928429856
}
```

Finally, when the upload is complete:

```
{"status":"pushing manifest"}
{"status":"success"}
```

If `stream` is set to `false`, then the response is a single JSON object:

```
{ "status": "success" }
```

Generate Embeddings

```
POST /api/embed
```

Generate embeddings from a model

Parameters

- `model`: name of model to generate embeddings from
- `input`: text or list of text to generate embeddings for. To embed media (images or audio) alongside text on models that accept multimodal embeddings, each entry may instead be an object with any of `text` (string), `image` (base64-encoded bytes), `audio` (base64-encoded bytes). In a list, entries can mix plain strings and such objects; each object owns its own media. `video` is not currently accepted.

Advanced parameters:

- `truncate`: truncates the end of each input to fit within context length. Returns error if `false` and context length is exceeded. Defaults to `true`
- `options`: additional model parameters listed in the documentation for the Modelfile such as `temperature`
- `keep_alive`: controls how long the model will stay loaded into memory following the request (default: `5m`)
- `dimensions`: number of dimensions for the embedding

Examples

Request

```
curl http://localhost:11434/api/embed -d '{
  "model": "all-minilm",
  "input": "Why is the sky blue?"
}'
```

Response

```
{
  "model": "all-minilm",
  "embeddings": [
    [
      0.010071029, -0.0017594862, 0.05007221, 0.04692972, 0.054916814,
      0.008599704, 0.105441414, -0.025878139, 0.12958129, 0.031952348
    ]
  ],
  "total_duration": 14143917,
  "load_duration": 1019500,
  "prompt_eval_count": 8
}
```

Request (Multiple input)

```
curl http://localhost:11434/api/embed -d '{
  "model": "all-minilm",
  "input": ["Why is the sky blue?", "Why is the grass green?"]
}'
```

Response

```
{
  "model": "all-minilm",
  "embeddings": [
    [
      0.010071029, -0.0017594862, 0.05007221, 0.04692972, 0.054916814,
      0.008599704, 0.105441414, -0.025878139, 0.12958129, 0.031952348
    ],
    [
      -0.0098027075, 0.06042469, 0.025257962, -0.006364387, 0.07272725,
      0.017194884, 0.09032035, -0.051705178, 0.09951512, 0.09072481
    ]
  ]
}
```

Request (With image)

```
curl http://localhost:11434/api/embed -d '{
  "model": "embeddinggemma2",
  "input": {"text": "a cat sitting on a windowsill", "image": "iVBORw0KGgo..."}
}'
```

Request (Batched, mixed modalities)

```
curl http://localhost:11434/api/embed -d '{
  "model": "embeddinggemma2",
  "input": [
    "a plain text query",
    {"image": "iVBORw0KGgo..."},
    {"audio": "UklGRiQAAABXQVZF..."}
  ]
}'
```

List Running Models

```
GET /api/ps
```

List models that are currently loaded into memory.

Examples

Request

```
curl http://localhost:11434/api/ps
```

Response

A single JSON object will be returned.

```
{
  "models": [
    {
      "name": "mistral:latest",
```

```
      "model": "mistral:latest",
      "size": 5137025024,
      "digest": "2ae6f6dd7a3dd734790bbbf58b8909a606e0e7e97e94b7604e0aa7ae4490e6d8",
      "details": {
        "parent_model": "",
        "format": "gguf",
        "family": "llama",
        "families": ["llama"],
        "parameter_size": "7.2B",
        "quantization_level": "Q4_0"
      },
      "expires_at": "2024-06-04T14:38:31.83753-07:00",
      "size_vram": 5137025024
    }
  ]
}
```

Generate Embedding

> Note: this endpoint has been superseded by `/api/embed`

```
POST /api/embeddings
```

Generate embeddings from a model

Parameters

- `model`: name of model to generate embeddings from
- `prompt`: text to generate embeddings for

Advanced parameters:

- `options`: additional model parameters listed in the documentation for the Modelfile such as `temperature`
- `keep_alive`: controls how long the model will stay loaded into memory following the request (default: `5m`)

Examples

Request

```
curl http://localhost:11434/api/embeddings -d '{
  "model": "all-minilm",
  "prompt": "Here is an article about llamas..."
}'
```

Response

```
{
  "embedding": [
    0.5670403838157654, 0.009260174818336964, 0.23178744316101074,
    -0.2916173040866852, -0.8924556970596313, 0.8785552978515625,
    -0.34576427936553955, 0.5742510557174683, -0.04222835972905159,
    -0.137906014919281
  ]
}
```

Version

```
GET /api/version
```

Retrieve the Ollama version

Examples

Request

```
curl http://localhost:11434/api/version
```

Response

```
{
  "version": "0.5.1"
}
```

Ollama Python Library

Source: https://github.com/ollama/ollama-python

The Ollama Python library provides the easiest way to integrate Python 3.8+ projects with Ollama.

Prerequisites

- Ollama should be installed and running
- Pull a model to use with the library: `ollama pull <model>` e.g. `ollama pull gemma4`
- See Ollama.com for more information on the models available.

Install

```
pip install ollama
```

Usage

```
from ollama import chat
from ollama import ChatResponse

response: ChatResponse = chat(
  model='gemma4',
  messages=[
    {
      'role': 'user',
      'content': 'Why is the sky blue?',
    },
  ],
)
print(response['message']['content'])
# or access fields directly from the response object
print(response.message.content)
```

See _types.py for more information on the response types.

Streaming responses

Response streaming can be enabled by setting `stream=True`.

```
from ollama import chat

stream = chat(
  model='gemma4',
  messages=[{'role': 'user', 'content': 'Why is the sky blue?'}],
  stream=True,
)

for chunk in stream:
  print(chunk['message']['content'], end='', flush=True)
```

Cloud Models

Run larger models by offloading to Ollama’s cloud while keeping your local workflow.

- Supported models: `deepseek-v3.1:671b-cloud`, `gpt-oss:20b-cloud`, `gpt-oss:120b-cloud`, `kimi-k2:1t-cloud`, `qwen3-coder:480b-cloud`, `kimi-k2-thinking` See Ollama Models - Cloud for more

information

Run via local Ollama

1) Sign in (one-time):

```
ollama signin
```

2) Pull a cloud model:

```
ollama pull gpt-oss:120b-cloud
```

3) Make a request:

```
from ollama import Client

client = Client()

messages = [
  {
    'role': 'user',
    'content': 'Why is the sky blue?',
  },
]

for part in client.chat('gpt-oss:120b-cloud', messages=messages, stream=True):
  print(part.message.content, end='', flush=True)
```

Cloud API (ollama.com)

Access cloud models directly by pointing the client at `https://ollama.com`.

1) Create an API key from ollama.com , then set:

```

```

2) (Optional) List models available via the API:

```
curl https://ollama.com/api/tags
```

3) Generate a response via the cloud API:

```
from ollama import Client

client = Client(host='https://ollama.com', headers={'Authorization': 'Bearer ' +
os.environ.get('OLLAMA_API_KEY')})

messages = [
  {
    'role': 'user',
    'content': 'Why is the sky blue?',
  },
]

for part in client.chat('gpt-oss:120b', messages=messages, stream=True):
  print(part.message.content, end='', flush=True)
```

Custom client

A custom client can be created by instantiating `Client` or `AsyncClient` from `ollama`.

All extra keyword arguments are passed into the `httpx.Client`.

```
from ollama import Client

client = Client(host='http://localhost:11434', headers={'x-some-header': 'some-value'})
response = client.chat(
  model='gemma4',
  messages=[
    {
      'role': 'user',
      'content': 'Why is the sky blue?',
    },
  ],
)
```

Async client

The `AsyncClient` class is used to make asynchronous requests. It can be configured with the same fields as the `Client` class.

```
from ollama import AsyncClient


async def chat():
  message = {'role': 'user', 'content': 'Why is the sky blue?'}
  response = await AsyncClient().chat(model='gemma4', messages=[message])


asyncio.run(chat())
```

Setting `stream=True` modifies functions to return a Python asynchronous generator:

```
from ollama import AsyncClient


async def chat():
  message = {'role': 'user', 'content': 'Why is the sky blue?'}
  async for part in await AsyncClient().chat(model='gemma4', messages=[message],
stream=True):
    print(part['message']['content'], end='', flush=True)


asyncio.run(chat())
```

API

The Ollama Python library's API is designed around the Ollama REST API

Chat

```
ollama.chat(model='gemma4', messages=[{'role': 'user', 'content': 'Why is the sky blue?'}])
```

Generate

```
ollama.generate(model='gemma4', prompt='Why is the sky blue?')
```

List

```
ollama.list()
```

Show

```
ollama.show('gemma4')
```

Create

```
ollama.create(model='example', from_='gemma4', system='You are Mario from Super Mario Bros.')
```

Copy

```
ollama.copy('gemma4', 'user/gemma4')
```

Delete

```
ollama.delete('gemma4')
```

Pull

```
ollama.pull('gemma4')
```

Push

```
ollama.push('user/gemma4')
```

Embed

```
ollama.embed(model='gemma4', input='The sky is blue because of rayleigh scattering')
```

Embed (batch)

```
ollama.embed(model='gemma4', input=['The sky is blue because of rayleigh scattering', 'Grass is green because of chlorophyll'])
```

Ps

```
ollama.ps()
```

System One

```
response = ollama.systemone(
  model='nimble',
```

```
  state='Our checkout has returned 500 errors since 9am.',
  questions={
    'team': {
      'type': 'choice',
      'instructions': 'Which team should handle this ticket?',
      'criteria': {'billing': 'Payments and refunds', 'technical': 'Software errors'},
    },
  },
)
print(response.answers['team'])
```

System One uses `POST /v1/systemone` and returns one JSON response; streaming is not supported. It runs locally with decision models such as `nimble` (Ollama v0.35.0 or later) or `clef` (v0.35.1 or later), and in the cloud with `clef:cloud` on a paid plan (through Ollama v0.40.5 or later, or directly against ollama.com).

Clef also reads images (v0.40.0 or later locally): pass `images` as file paths, bytes, or base64 strings, shared by all questions. Cloud requests accept up to 4 PNG, JPEG, or WebP images (each at most 4 MiB and 16 megapixels, 8 MiB total).

`state` and question `instructions` accept text, JSON objects, or arrays. Questions are evaluated in their supplied order:

- `choice`: 2–26 option keys mapped to descriptions (up to 255 in the cloud); `None` uses the key as its description.
- `noul`: probability of true, with optional `{"false": "No", "true": "Yes"}` descriptions.
- `score`: 2–10 descriptions ordered lowest to highest; returns a potentially fractional, zero-based score.

In the cloud, question ids may only contain ASCII letters, digits, `_`, `.`, and `-`, up to 100 characters, and a request may ask up to 64 questions.

Responses contain `model`, `answers`, and `usage.input_tokens` / `usage.output_tokens`. Confidence measures probability concentration, not calibrated correctness. Token usage comes from the server; output tokens are not necessarily zero. Optional `keep_alive` accepts seconds or a duration string. Requests use the client's existing host, headers, and HTTP error handling. The server validates its body and model context limits without truncating input.

Available as `ollama.systemone`, `Client.systemone`, and `await AsyncClient.systemone`. Question and answer types, including `SystemOneResponse`, are exported from `ollama`. See the combined question example.

Errors

Errors are raised if requests return an error status or if an error is detected while streaming.

```
model = 'does-not-yet-exist'

try:
  ollama.chat(model)
except ollama.ResponseError as e:
  print('Error:', e.error)
  if e.status_code == 404:
    ollama.pull(model)
```